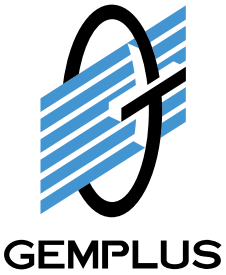
GemSafe v2 Applet

Reference Manual



All information herein is either public information or is the property of and owned solely by Gemplus S.A. who shall have and keep the sole right to file patent applications or any other kind of intellectual property protection in connection with such information.

Nothing herein shall be construed as implying or granting to you any rights, by license, grant or otherwise, under any intellectual and/or industrial property rights of or concerning any of Gemplus' information.

This document can be used for informational, non-commercial, internal and personal use only provided that:

- The copyright notice below, the confidentiality and proprietary legend and this full warning notice appear in all copies.
- This document shall not be posted on any network computer or broadcast in any media and no modification of any part of this document shall be made.

Use for any other purpose is expressly prohibited and may result in severe civil and criminal liabilities.

The information contained in this document is provided "AS IS" without any warranty of any kind. Unless otherwise expressly agreed in writing, Gemplus makes no warranty as to the value or accuracy of information contained herein. The document could include technical inaccuracies or typographical errors. Changes are periodically added to the information herein. Furthermore, Gemplus reserves the right to make any change or improvement in the specifications data, information, and the like described herein, at any time.

Gemplus hereby disclaims all warranties and conditions with regard to the information contained herein, including all implied warranties of merchantability, fitness for a particular purpose, title and non-infringement. In no event shall Gemplus be liable, whether in contract, tort or otherwise, for any indirect, special or consequential damages or any damages whatsoever including but not limited to damages resulting from loss of use, data, profits, revenues, or customers, arising out of or in connection with the use or performance of information contained in this document.

Gemplus does not and shall not warrant that this product will be resistant to all possible attacks and shall not incur, and disclaims, any liability in this respect. Even if each product is compliant with current security standards in force on the date of their design, security mechanisms' resistance necessarily evolves according to the state of the art in security and notably under the emergence of new attacks. Under no circumstances, shall Gemplus be held liable for any third party actions and in particular in case of any successful attack against systems or equipment incorporating Gemplus products. Gemplus disclaims any liability with respect to security for direct, indirect, incidental or consequential damages that result from any use of its products. It is further stressed that independent testing and verification by the person using the product is particularly encouraged, especially in any application in which defective, incorrect or insecure functioning could result in damage to persons or property, denial of service or loss of privacy.

© Copyright 2004–6 Gemplus S.A. All rights reserved. Gemplus and the Gemplus logo are trademarks and service marks of Gemplus S.A. and are registered in certain countries. All other trademarks and service marks, whether registered or not in specific countries, are the property of their respective owners.

GEMPLUS, B.P. 100, 13881 GEMENOS CEDEX, FRANCE.

Tel: +33 (0)4.42.36.50.00 Fax: +33 (0)4.42.36.50.90

Printed in France.

Document Reference: DOC112007G

Document Version: 7

June 26, 2006

Contents

Introduction xi

Who Should Read This Book . . . xi
Conventions . . . xi
Standard Terms . . . xiii
Applet, Application . . . xiii
Card Acceptance Device, Interface Device, Client or Host Terminal . . . xiii
Java Card . . . xiii
Contact Our Hotline . . . xiii

Chapter 1 Overview 1

What Is the GemSafe v2 Applet? . . . 1
Security Throughout the Life of the GemSafe v2 Applet . . . 1
Security During Installation . . . 1
Security During the Personalization Phase . . . 1
Security During the Application Phase . . . 1
GemSafe v2 Applet Features . . . 2
Data Structure . . . 3
Data Access Management . . . 3
Command Set . . . 3
Compliance with Standards . . . 4
Java Platform Compliance . . . 4

Chapter 2 Files and Data Objects 5

File Structure . . . 5
Root . . . 5
Dedicated Files . . . 6
File Control Parameters . . . 6
Elementary Files . . . 7
File Control Parameters . . . 7
EF Types . . . 8
File Referencing Methods . . . 8
Referencing by File Identifier . . . 8
Referencing by DF Name . . . 9
Referencing by Path . . . 9
Data Objects . . . 9

Chapter 3 Public Key Cryptographic Concepts 13

RSA Key Pairs 13
RSA Key Identification 13
Digital Signatures 14
Performing a Digital Signature 14
Verifying a Digital Signature 17
Encryption and Decryption 18
Encrypting a Message 18
Decrypting a Message 18
On Board Key Generation 19
Mutual Authentication 19

Chapter 4 Installing the GemSafe v2 Applet 21

Install Command Parameters 23
GemSafe v2–Specific Parameters 24

Chapter 5 Personalizing the GemSafe v2 Applet 27

Personalization According to Authentication Mechanism 27
Personalization Example for an Asymmetric Key Mutual Authentication Application 27
Personalization Example for a Symmetric Key Mutual Authentication Application . 30
PKCS#15 Personalization Profiles 32

Chapter 6 Security Features 33

Security Attributes 33
Access Mode Byte 33
Security Condition Byte 35
Security Attributes Example 36
Built–in Security Policy 37
PIN Management 37
PIN Contents 38
Verifying a PIN 39
Unblocking a PIN 39
Changing a PIN Value 40
Secure Messaging 40
Secure Messaging During the Personalization Phase 40
Secure Messaging During the Application Phase 43
Security Environments 47
Security Environments Structure 48

Chapter 7 Applet Life Cycle States 55

Chapter 8 Mutual Authentication 57

Asymmetric Key Mutual Authentication (with Privacy) 57
The Asymmetric Key Mutual Authentication Mechanism 58
Typical Command Sequence for Asymmetric Mutual Authentication 59
Symmetric Key Mutual Authentication 63
The Mutual Authentication Mechanism 63

Command Sequence for Mutual Authentication 64
Computation of Session Keys K_{SK}ENC & K_{SK}MAC 65
The Send Sequence Counter (SSC) 66

Chapter 9 APDU Command Formats 67

T=0 Protocol 67
T=1 Protocol 67
Command Format 67
Header Fields 68
Body Fields 68
Response Format 68
Command–Response Pairs 69
Case 1: No Input/No Output 69
Case 2s: No Input/Output of Expected Length 69
Case 3s: Input/No Output 69
Case 4s: Input/Output of Expected Length 69
T=0 Protocol 70
T=1 Protocol 70

Chapter 10 Personalization Phase Commands 71

Activate File 73
Create File 76
Deactivate File 80
End Personalization 82
Erase Binary 84
External Authenticate (Perso) 87
Get Data 89
Get Response 96
Initialize Update 97
Put Data 100
Put Data–Security Environments 101
Put Data–PINs 102
Put Data–Secret Keys 106
Put Data–Private Keys 108
Put Data–Public Keys 112
Put Data–DH Key Exchange Parameters 115
Read Binary 118
Select File 121
Update Binary 125

Chapter 11 Application Phase Commands 127

Activate File 129
Change Reference Data 132
Create File 134
Deactivate File 135
Erase Binary 138
External Authenticate (Application) 141
Generate Public Key Pair 144
Get Challenge 146

Get Data 147
Get Response 154
Internal Authenticate 155
MSE–Restore 157
MSE–Set 158
Mutual Authenticate 160
PSO–Compute Digital Signature 162
PSO–Decipher 165
PSO–Hash 168
PSO–Verify Certificate 172
Put Data 174
Put Data–Secret Keys 175
Put Data–Private Keys 177
Put Data–Public Keys 179
Put Data–DH Key Exchange Parameters 181
Read Binary 183
Reset Retry Counter 186
Select File 188
Update Binary 192
Verify 194

Appendix A Status Bytes 197

Appendix B Format of Authentication Certificates 199

C_CV.IFD.AUT 199
Certification Authority Reference (CAR) 200
Certificate Holder Reference 200
Certificate Holder Authorization 200
Hash 200
C.ICC.AUT 200

Appendix C Generating Session Keys with the Diffie–Hellman Algorithm 201

Deriving the Shared Secret Data, $K_{IFD/ICC}$ 201
Deriving the Session Keys 202
$K_{SK}ENC$ 202
$K_{SK}MAC$ 202

Appendix D Triple DES Computations 203

3DES in ECB Mode 203
3DES in CBC Mode 204

Appendix E Security Environment Coding Examples 205

Example 1 205
Example 2 206
Example 3 207
Example 4 207
Example 5 208
Example 6 209

Example 7 209
Example 8 210
Example 9 210
Example 10 211

Appendix F Common Criteria Certification Requirements 213

Applet Installation 213
Authentication Ratification Counter 213
Change PIN Before First Use Option 213
Application Personalization 214
PINs 214
RSA Keys 214
Signature Requirements 215

Terminology 217

Abbreviations 217
Glossary 220

For More Information 229

Standards and Specifications 229
Other Gemplus Documentation 230
Recommended Reading 231
Applet Development 231
Cryptography 231
Web References 231

Index 233

List of Figures

Figure 1 - Hierarchy of GemSafe v2 Files 5
Figure 2 - Performing a Digital Signature 14
Figure 3 - Verifying a Digital Signature 17
Figure 4 - Encrypting a Message 18
Figure 5 - Decrypting a Message 18
Figure 6 - Command Sequence to Install the GemSafe v2 Applet 22
Figure 7 - Example File Architecture After Personalization (1) 29
Figure 8 - Example File Architecture After Personalization (2) 31
Figure 9 - Computing a MAC 46
Figure 10 - Main Steps of Asymmetric Key Mutual Authentication 59
Figure 11 - Main Steps of Mutual Authentication 63
Figure 12 - 3DES Algorithm 203
Figure 13 - 3DES in CBC Mode 204

List of Tables

Table 1 - Key Notation xii
Table 2 - FCP for DFs 6
Table 3 - FCP for EFs 7
Table 4 - Life Cycle Status Byte Format 8
Table 5 - Private Key CRT Elements 10
Table 6 - Tags in RSA Public Key Data Objects 11
Table 7 - Tags in DH Key Exchange Parameters Data Objects 11
Table 8 - Algorithm ID Values 15
Table 9 - Algo ID = 01h: Hash Performed Externally; ISO 9796–2 Padding 15
Table 10 - Algo ID = 02h: Hash Performed Externally; PKCS #1 Padding 16
Table 11 - Algo ID = 03h: Hash Performed Externally; RFC 2409 Padding 16
Table 12 - Algo ID = 11h: Hash Performed by Card; ISO 9796–2 Padding 16
Table 13 - Algo ID = 11h: Hash Performed by Card; PKCS#1 Padding 17
Table 14 - Parameters for Install Command in Install Mode 23
Table 15 - Install Parameter Field 24
Table 16 - GemSafe v2–Specific Parameters 24
Table 17 - Authentication Key Option Byte Coding 24
Table 18 - Option Parameters 25
Table 19 - Application Options Byte Coding 25
Table 20 - Command Sequence to Personalize the GemSafe v2 Applet 28
Table 21 - Command Sequence to Personalize the GemSafe v2 Applet 30
Table 22 - Access Mode Byte Coding – For DFs 34
Table 23 - Access Mode Byte Coding – For EFs 34
Table 24 - Access Mode Byte Coding – For Data Objects 34
Table 25 - Security Attributes Coding Example 36
Table 26 - Built-in Minimum Levels of Secure Messaging 37
Table 27 - Structure of a PIN Data Object 38
Table 28 - PIN Attributes 39
Table 29 - Types of Session Key 41
Table 30 - Data Objects Used in Secure Messaging During Application Phase 44
Table 31 - SEID Values 48
Table 32 - LCS Values 48
Table 33 - CRT Values 49

Table 34 - Control Reference Data Objects 49
Table 35 - CRT Usage Qualifier Values 50
Table 36 - Authentication Template Coding for a PIN 50
Table 37 - Authentication Template Coding for a Key Reference 51
Table 38 - Authentication Template Coding for a CHA 51
Table 39 - Key Agreement Template Coding 52
Table 40 - Cryptographic Checksum Template Coding 52
Table 41 - Digital Signature Template Coding–For Signatures 53
Table 42 - Digital Signature Template Coding–For Certificate Verification 53
Table 43 - Coding for an Algorithm ID 53
Table 44 - Confidentiality Template Coding 54
Table 45 - Confidentiality Template Coding 54
Table 46 - Get Response when Licc > 256 Bytes 70
Table 47 - Response to Case 4 Command 70
Table 48 - Get Response when Licc > 256 Bytes 70
Table 49 - Personalization Commands 71
Table 50 - Data Elements for Create File Command 77
Table 51 - Example of DF Creation 78
Table 52 - Example of EF Creation 78
Table 53 - Get Data Values (Perso Phase) 89
Table 54 - Get Data Coding Structure – Public Key Element 90
Table 55 - Get Data Coding Structure – Entire Public Key 90
Table 56 - Get Data Coding Structure – K_{ICC} 91
Table 57 - Get Data Coding Structure –Diffie–Hellman Element 91
Table 58 - CPLC Data 93
Table 59 - Get Data Response Structure – Public Key Element 94
Table 60 - Get Data Response Structure – Entire Public Key 94
Table 61 - Get Data Response Structure – K_{ICC} 95
Table 62 - Get Data Response Structure for an SE 95
Table 63 - Structure of a PIN Data Object 103
Table 64 - Data Field Structure for Put Data–Secret Key Creation 107
Table 65 - Data Field Structure for Put Data–Secret Key Update 107
Table 66 - Example Data Field Structure for Put Data–Private Key Creation 110
Table 67 - Example Data Field Structure for Put Data–Writing Private Key Elements . . 111
Table 68 - Data Field Example for Put Data–Public Key Creation (not part of a Key Pair) 113
Table 69 - Data Field Structure for Put Data–Writing Exponent Value 114
Table 70 - Data Field Structure for Put Data–Writing Modulus Value 114
Table 71 - Data Field Example for Put Data–DH Key Exchange Parameters Creation . . 116
Table 72 - Data Field Structure for Put Data–Writing DH Key Exchange Parameters Value 117
Table 73 - FCI for DFs 123
Table 74 - FCI for EFs 124
Table 75 - Application Commands 127
Table 76 - Data Field for External Authenticate (Application) Command 142
Table 77 - Get Data Values (Perso Phase) 147
Table 78 - Get Data Coding Structure – Public Key Element 148
Table 79 - Get Data Coding Structure – Entire Public Key 148
Table 80 - Get Data Coding Structure – K_{ICC} 149
Table 81 - Get Data Coding Structure – Diffie–Hellman Element 149

Table 82 - Get Data Response Structure – CPLC Data 150
Table 83 - Get Data Response Structure – Public Key Element 151
Table 84 - Get Data Response Structure – Entire Public Key 152
Table 85 - Get Data Response Structure – K_{ICC} 152
Table 86 - Get Data Response Structure for an SE 152
Table 87 - Data Field for Internal Authenticate Command 156
Table 88 - PKCS #1 v2.x Format for Encrypting Data 166
Table 89 - PSO–Hash Input Data (Hash Performed Partially by the Card) 170
Table 90 - PSO–Hash–Input Data (Hash Performed Entirely Externally) 171
Table 91 - PSO–Hash–Length of Hash Input Data (External Hash) 171
Table 92 - Data Field Structure for Put Data–Secret Key Update 176
Table 93 - Example Data Field Structure for Put Data–Writing Private Key Elements . . 178
Table 94 - Example Data Field Structure for Put Data–Writing Exponent Value 180
Table 95 - Data Field Structure for Put Data–Writing Modulus Value 180
Table 96 - Data Field Structure for Put Data–Writing DH Key Exchange Parameters . . . 182
Table 97 - FCI for DFs 190
Table 98 - FCI for EFs 191
Table 99 - Response Status Bytes 197
Table 100 - C_CV.IFD.AUT Format 199

Introduction

This document provides information about the GemSafe v2 applet — an applet that can be used for public key infrastructure (PKI) applications such as identity cards and corporate security (closed user groups). The GemSafe v2 applet is fully compliant with digital signature law.

This reference manual provides a detailed description of the data structure, commands and security system and tells you how to load, install, personalize, and use the GemSafe v2 applet in personalization and application modes.

Who Should Read This Book

This reference manual is designed for developers building PKI applications such as identity, secure Signature Creation Applications (SCA), Certification Generation application (CGA), or closed user group applications.

Knowledge of the following is helpful:

- Smart card technology
- Java cards
- Cryptography
- PKI systems

Conventions

The following conventions are used in this document:

Numeric values

By default, numeric values are expressed in decimal notation.

- Binary numbers are followed by the ‘b’ character. For example, the decimal value 13 is expressed in binary as **1101b**.
- Hexadecimal numbers are followed by the ‘h’ character. For example, the decimal value 13 is expressed in hexadecimal as **0Dh**.

RFU values

The value 00h is assigned to each RFU (Reserved for Future Use) byte.

Bit Numbering

A byte consists of 8 bits, b_8 to b_1, where b_8 is the most significant bit and b_1 the least significant bit, as shown below:

One byte	b_8	b_7	b_6	b_5	b_4	b_3	b_2	b_1

Byte numbering

A string of n bytes consists of n number of concatenated bytes: $B_nB_{n-1}...B_2B_1$.
B_n is the most significant byte and B_1 is the least significant byte:

String on n bytes	B_n	B_{n-1}	...	B_4	B_3	B_2	B_1

Key Notation

To easily identify whether a key is public or private, whether it exists in the card or the terminal or is a CA key, and also the usage of a key, the following notation is used in this document.

KeyType.KeyOwner.KeyUsage

The possible values are shown in the following table:

Parameter	Value	Meaning
KeyType	PK	Public key
	SK	Private key (secret key)
KeyOwner	ICC	Card
	IFD	Terminal (that is interface with application)
	CA	Certification authority
KeyUsage	AUT	Authentication key
	DS	Digital signature key
	UT	Utility key

Table 1 - Key Notation

Algorithm Notation.

When describing a computation made by an algorithm that uses keys, the notation is written:

$ALGO_{MODE}$ [data] <key>

Mathematical Notation

<= means “less than or equal to”

XOR means an exclusive-OR operation.

Standard Terms

A complete list of abbreviations and terms is given in “Terminology” on page 217. The following terms used in this document may differ to those in associated documentation from Sun Microsystems and Visa.

Applet, Application

In Java Card terminology, an “applet” is an independent Java application loaded into a Java Card. The complementary application in the host terminal is referred to as the “host application”.

Card Acceptance Device, Interface Device, Client or Host Terminal

A card applet is always reactive, responding to function calls received from an external program. The card then reacts to commands sent from an external host. This program, the host application, is held on a terminal, which must incorporate some card acceptance device (CAD) or card reader to enable communication to and from the card. This terminal and CAD can be termed an interface device.

Java Card

This term refers to smart cards fully compliant both with the Java Card standards (currently Java Card 2.1.1) and with the GlobalPlatform 2.1.1 standard.

Contact Our Hotline

If you do not find the information you need in this manual, or if you find errors, contact the Gemplus hotline by phone, fax, or email. In your email, please include the document reference number, your job function, and the name of your company. (You will find the document reference number at the bottom of the legal notice on the inside front cover.)

EMEA (Europe, Middle East, and Africa) and ASPAC (Asia– Pacific)

Hotline: +33 (0)4 42 36 50 50

Hot Fax: +33 (0)4 42 36 64 00

Email: hotline@gemplus.com

Americas

Hotline: 1 (215) 390-2484

or Toll Free number: 1 (877) 436-7233

Hot Fax: 1 (215) 390-2920

Email: US.Support@gemplus.com

From Our Web Site

http://www.gemplus.com

1

Overview

What Is the GemSafe v2 Applet?

GemSafe v2 is a Java applet that provides all the necessary functions to integrate a smart card in a public key infrastructure (PKI) system, suitable for identity and corporate security applications. It is also useful for storing information about the cardholder and any sensitive data. GemSafe v2 implements state–of–the–art security and conforms to the latest standards for smart cards and PKI applications. It is also fully compliant with digital signature law.

Security Throughout the Life of the GemSafe v2 Applet

Security During Installation

Before the GemSafe v2 applet is installed on the card, the Card Manager opens a secure channel to protect the loading and installation of the GemSafe v2 applet. Once loaded, the applet life cycle state is SELECTABLE.

Security During the Personalization Phase

Once the GemSafe v2 applet is in the card, it needs to be personalized, that is, certain data must be written to the applet. Before the personalization device writes this data, it opens a secure channel that defines the level of security for the personalization commands. There are three modes, NONE, MAC, and MAC and ENCRYPTION. A MAC is a cryptographic checksum that guarantees the integrity of a message.

On the card side, the exchange is managed by the Card Manager and involves the computation of session keys using the 3DES algorithm.

After personalization, the applet life cycle state is PERSONALIZED.

Security During the Application Phase

During this phase, the terminal communicates directly with the GemSafe v2 applet. Again, the terminal and applet authenticate each other and open a secure channel. This process differs according to whether the application uses asymmetric key or symmetric key mutual authentication.

Asymmetric key mutual authentication uses the public key algorithm RSA, as recommended in the CWA 14890 standard.

Symmetric key mutual authentication consists of each entity verifying the presence of two 3DES secret keys in the other entity.

The two processes are described in detail in “Chapter 8 - Mutual Authentication”. In both cases, a ratification counter is decremented for failed mutual authentication, and if this counter reaches zero, the applet life cycle state changes to BLOCKED. Refer to “Chapter 7 - Applet Life Cycle States” for more details.

GemSafe v2 Applet Features

The main features of the GemSafe v2 applet are as follows:

- Full compliancy with digital signature law
  - CEN workshop agreement (CWA) 14890
  - Common criteria certification according to PP SSCD
- Digital signatures—these are used to ensure the integrity and authenticity of a message. The signatures are made by an RSA private key that is stored in the card.
- Storage of sensitive data based on security attributes, (PINs, mutual authentication, role ID and secure messaging).
- PIN management.
- Secure messaging based on the triple DES (3DES) algorithm.
- Public key cryptography, allowing for RSA keys of up to 2,048 bits
- Storage of digital certificates—these are issued by a trusted body known as a certification authority (CA) and are typically used in PKI authentication schemes.
- Decryption—data encryption guarantees the confidentiality of data being sent from the terminal to the card. The RSA mechanism enables a sender to send encrypted data, such as a triple DES session key for example, to the card. The card can then decrypt the data and return it to the application software.
- On board key generation—particularly useful for updating a key pair because the value of the private key never leaves the card and GemSafe v2 guarantees the security of the update.
- Mutual authentication between GemSafe v2 and the terminal based on either 3DES secret keys or RSA public keys with Diffie–Hellman key exchange mechanism as defined in the CWA 14890 standard.
- Compatible with Identrus requirements (as defined in *Key Storage Mechanism Requirements*, version 3.0, Identrus).
- Supports ISO 7816–15/PKCS#15 personalization profiles.
- Multi–instantiable, that is, it is possible to have several instances of the applet in the card.

Data Structure

GemSafe v2 provides specific file structures and data objects. Files are organized in a two-level hierarchy—the global level and local level. The top of the hierarchy is headed by a special dedicated file (DF), known as the Root. Below this are elementary files (EFs) and DFs (below which are more EFs).

The *global level* is the root and the EFs directly beneath it. The *local level* is formed by the DFs below the root and the EFs beneath them. A DF can also hold several EFs. Each EF contains data. For more information on how GemSafe v2 files are structured, see "Chapter 2 - Files and Data Objects".

In addition to files, GemSafe v2 supports data objects. These objects are PINs, security environments (SEs), RSA keys and 3DES keys. For more information about data objects, see "Data Objects" on page 9.

Files are created using the **Create File** command, while data objects are created using the **Put Data** command.

Data Access Management

Access to GemSafe v2 files and data objects is controlled by security attributes. These security attributes specify the protection given to a file or data object, which can be secure messaging (SM), mutual authentication, role or a PIN. More than one type of protection can be used for a given file.

Note: Secure messaging can be used only after a successful mutual authentication.

Security attributes refer to security environments (SEs). These security environments specify which PINs to use to access the file or data object and also the applet life cycle state(s) in which the file or data object can be accessed. The GemSafe v2 applet contains from 1 to 14 SEs.

For more information on accessing GemSafe v2 files and data objects, see "Chapter 2 - Files and Data Objects".

Command Set

The commands in GemSafe v2 broadly fall into two groups, according to whether they are used in the personalization phase or the application phase. They are described individually in "Chapter 10 - Personalization Phase Commands" and "Chapter 11 - Application Phase Commands". Some commands are available in both phases and appear in both chapters with their description corresponding to the appropriate phase.

Compliance with Standards

The GemSafe v2 applet features are based on the following standards:

- *ISO 7816 (parts 4, 6, 8, 9, and 15)*
  The compliant commands, data structures (multi-application) and return codes ensure a wide acceptance of the applet by application issuers and terminal manufacturers.

  Note: Part 15 is a smart card standard, itself based on the *PKCS#15 Cryptographic Token Information Syntax Standard*.

- *Java Card 2.1.1 Runtime Environment (JCRE) Specifications, Version 1.0, 18 May 2000.*
- *GlobalPlatform Card Specification, Version 2.1.1, March 2003.*
- *CWA 14890 2004—Application Interface for smart cards used as Secure Signature Creation Devices—Part 1 -Basic requirements. March 2004.*
- *Protection Profile—Secure Signature–Creation Device Type 2, version 1.04, EAL 4+*, 25 July 2001. Prepared for CEN/ISSS.
- *Protection Profile—Secure Signature–Creation Device Type 3, version 1.05, EAL 4+*, 25 July 2001. Prepared for CEN/ISSS.
- *Key Storage Mechanism Requirements*, version 3.0, Identrus

Java Platform Compliance

Although it can run on any smart card compliant with the Java Card 2.1.1 specification, it is recommended to use one of the cards in the GemXpresso Pro R3.x range from Gemplus. For details about the cards in this range, refer to the *GemXpresso Pro R3.x Reference Manual*.

2

Files and Data Objects

The GemSafe v2 applet contains files and data objects. This chapter describes the different types of both and the file structure in the applet.

File Structure

The files in GemSafe v2 applets are organized in a two-level hierarchy.

At the top is the root, which can be considered as particular dedicated file (DF) with its own security attributes.

The level formed by the root with elementary files (EFs) directly beneath it is called the *global level.* The level formed by dedicated files (DFs) with EFs beneath them is called the *local level.*

Files are created as objects in the non-volatile memory of the chip. Files which comprise the body and its attributes are created by instantiating a class in the non-volatile memory.

The following figure shows the structure of GemSafe v2 file hierarchy:

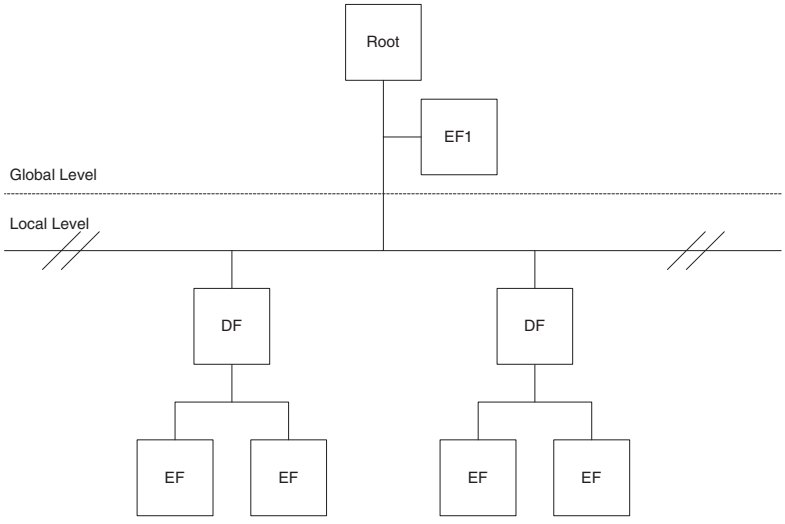


Figure 1 - Hierarchy of GemSafe v2 Files

Root

The root of the file structure in the GemSafe v2 applet has a unique identifier 3F00h.

Dedicated Files

Dedicated files (DFs) store sets of elementary files. Typically in GemSafe v2 applets, each DF stores the set of elementary files that form an application. A DF is the equivalent of a directory. Nested DFs are not supported; that is, a DF under another DF is not allowed.

The characteristics of a DF are recorded in its file control parameters (FCP) that are specified at the time of its creation in the **Create File** command. The FCP stores the information needed by GemSafe v2 to manage the file.

In the application phase, the creation of DFs is allowed only if specified in the access mode byte of the root. For more details, refer to “Access Mode Byte” on page 33.

File Control Parameters

When you create a file, the **Create File** command generates the file control parameters (FCP) template. This template is in tag, length, value (TLV) format and its own tag is 62h”. The template contains the data elements that are contained in a DF, and it is coded as follows:

Tag	Length	Tag	Length	Value	Description
62h	0Ah–19h				Tag and length of FCP template
		82h	01h	38h	TLV of FDB
		83h	02h	var.	TLV of File ID
		8Ch	01h–05h	var.	TLV of Security attributes (access mode & security condition bytes)
		84h	01-10h	var.	TLV of DF name

Table 2 - FCP for DFs

The DF name is optional and if used must be the final parameter. The FDB and file ID are mandatory.

The following table describes the structure of the FCP of a dedicated file:

Field	Length	Description
FDB	1 byte	The file descriptor byte (FDB) is set to 38h when a DF is created.
File ID	2 bytes	This is allocated when the file is created.
Security Attribute	1–5 bytes	The security attributes are made up of one Access Mode byte which indicates the commands to be “controlled”, followed by 0–4 security condition bytes which indicate the conditions for each command indicated in the Access Mode byte. For a more detailed description, see “Security Attributes” on page 33.
DF name	1–16 bytes	This contains the name of the DF up to a maximum of 16 bytes. It can be used to reference a file in a command.

For information on how to reference DFs, see “File Referencing Methods” on page 8.

Elementary Files

Elementary files (EFs) are the main component of the GemSafe v2 file structure. They are stored under either the root or DFs and contain data.

The characteristics of an EF are recorded in its file control parameters (FCP) that are specified at the time of its creation in the **Create File** command. The FCP stores the information needed by GemSafe v2 to manage the file.

In the application phase, the creation of EFs is allowed only if specified in the access mode byte of the parent file. For more details, refer to “Access Mode Byte” on page 33.

File Control Parameters

When you create a file, the **Create File** command generates the FCP template, which like for DFs, is in tag, length, value (TLV) format and has the tag 62h. The template contains the data elements that are contained in an EF, and it is coded as follows:

Tag	Length	Tag	Length	Value	Description
62h	11h–15h				Tag and length of FCP template
		81h	02h	var.	TLV of File size (in bytes)
		82h	01h	01h	TLV of FDB
		83h	02h	var.	TLV of File ID
		8Ah	01h	var.	TLV of Life cycle status
		8Ch	01h–05h	var.	TLV of Security attributes (access mode & security condition bytes)

Table 3 - FCP for EFs

All these elements are mandatory.

The following table describes the structure of the FCP of an elementary file:

Field	Length	Description
File Size	2 bytes	This specifies the size of the file.
FDB	1 byte	The file descriptor byte (FDB) is set to 01h, meaning transparent file, when an EF is created.
Identifier (FID)	2 bytes	This is allocated when the file is created. The short file identifier corresponds to the 5 least significant bits of the file identifier. It is used to reference a file in a command.

Field	Length	Description
Life cycle status	1 byte	The life cycle status shows the status of the EF. It is used by security environments to control the access to the file. The LCS byte is coded as follows:

b8...b5	b4	b3	b2	b1	State
0..0	0	0	0	1	CREATED
0..0	0	0	1	1	INITIALIZED
0..0	0	1	–	1	OPERATIONAL (ACTIVATED)
0..0	0	1	–	0	OPERATIONAL (DEACTIVATED)

Table 4 - Life Cycle Status Byte Format

Security Attribute	1–5 bytes	The security attributes are made up of one Access Mode byte which indicates the commands to be “controlled”, followed by 0–4 security condition bytes which indicate the conditions for each command indicated in the Access Mode byte. For a more detailed description, see “Security Attributes” on page 33.

EF Types

All EFs managed by GemSafe v2 are transparent EFs. These have an FDB value of 01h. A transparent file consists of an unstructured sequence of bytes that can be accessed by specifying an offset relative to the start of the EF. The offset size is given in bytes. The first byte of a transparent EF has the relative address 00h.

File Referencing Methods

Referencing by File Identifier

Any file can be referenced by its two–byte file identifier (tag 83h). Some commands such as **Read Binary** can reference EFs by the 5 least significant (rightmost) bits of the file identifier, known as the short file identifier (SFI).

In order to avoid ambiguity when selecting files by their identifier, follow these rules:

- Choose the file identifier when creating the file.
- Ensure that no two files under the same parent file have the same identifier.
- Ensure that no file has the same identifier as its parent file.

Note:

1. The value 3F 00h is reserved for the root.
2. The value FF FFh is reserved for future use (RFU).
3. The value 3F FFh is reserved (see “referencing by path” in “ISO/IEC 7816-4”).

Referencing by DF Name

Any DF can be selected by its name (1–16 bytes). In order to avoid ambiguity when selecting files by their DF name, make sure that no two DFs share the same name. When referencing a file by its name, the whole name must be specified (partial names are not supported).

Referencing by Path

A file's path is the concatenation of two–byte file identifiers starting with the root or current DF and ending with the file itself. It includes the file identifier of the parent DF. In this way, any file can be selected unambiguously from the current DF.

As the path does not include the current DF or the root in GemSafe v2, it is reduced to the two–byte file ID for DFs and EFs under the root. For EFs under a DF, the path becomes the concatenation of the DF file ID and the EF file ID, a total of four bytes.

Data Objects

The formats of these data objects are defined in Java Card 2.1.1 Specification. There are six different types:

- Security environments (SEs)
- PINs
- Secret keys
- Private keys
- Public keys
- Diffie–Hellman key exchange parameters

Data objects are independent of the rest of the file structure. They are created using the **Put Data** command. The **Put Data** command can also be used to update or initialize data objects during personalization. After personalization it can still be used to update secret, public and private keys and Diffie–Hellman key exchange parameters, but no other data objects. To update a PIN after personalization, use the **Change Reference Data** or **Reset Retry Counter** commands.

Security Environments

Security environments (SEs) have three purposes as follows:

- To control access to a file or data object (PIN protection)
- To indicate the keys and algorithms to use with the **Perform Security Operation (PSO)** commands (current SE only)
- To indicate the keys and algorithms to use with the **Internal Authenticate** command (current SE only)

In the first case, security attributes determine whether user authentication (PIN) is necessary to perform a command. The security attributes can specify an SE which contains the references of the PIN(s) to be used. For more detailed information about security attributes, see "Security Attributes" on page 33.

In all three cases, the SE defines in which life cycle states the keys and PINs can be used. Security environments are discussed in more detail in "Chapter 6 - Security Features".

PINs

PINs are used to identify the cardholder and to protect data. PINs are discussed in detail in “Chapter 6 - Security Features”.

Secret Keys

Secret keys are 3DES keys used by GemSafe v2 for symmetric key mutual authentication. Both the GemSafe v2 applet and the terminal possess the two secret keys, K_{ENC} and K_{MAC} which can be diversified, for example with the card serial number. Mutual authentication consists of each entity proving that it possesses the two keys to the other entity.

Private Keys

Keys are used for public key cryptographic operations such as digital signatures and decryption. Their function is described in more detail in “RSA Key Pairs” on page 13.

GemSafe v2 stores RSA private keys as five CRT elements. The Chinese remainder theorem (CRT) is a method of computing signatures by using five elements. The size of each element is equal to half the length of the key’s modulus. Although this takes up a little more memory, it has the advantage that the computation is significantly faster than using the full key.

The RSA private key must include the following key CRT elements:

Tag	Length	Value
92h	n/2	p
93h	n/2	q
94h	n/2	iq
95h	n/2	dp
96h	n/2	dq

Table 5 - Private Key CRT Elements

Private keys are always stored as data objects and must be created during the personalization phase. They can be initialized either during the personalization phase or during the application phase.

The five key elements can be initialized in any order. You can use the **Generate Public Key Pair** command to automatically update the private key value securely. Alternatively, you can generate the keys outside the card then update the key elements in GemSafe v2 by using the **Put Data** command.

Note: One **Put Data** command updates one CRT element.

For more details, see Table 66, “Example Data Field Structure for Put Data–Private Key Creation”, on page 110.

Public Keys

RSA public keys associated with RSA private keys in a key pair. These could be used for authentication or to verify a signature.

The following RSA public keys are always stored as data objects:

- CA public keys used in asymmetric key mutual authentication
- RSA public keys used by the application.

It is possible for more than one CA to store its public key PK.CA.AUT in GemSafe v2.

RSA public keys always contain a modulus, N, and a public exponent, e.

The tags for these keys are shown in the following table:

Tag	Length	Value
81h	N	N, modulus
82h	1–8 bytes	e, public exponent

Table 6 - Tags in RSA Public Key Data Objects

The two key elements can be initialized in any order. As with private keys, you can use the **Generate Public Key Pair** command to automatically update the public key values or you can generate the keys outside the card then update the key elements in GemSafe v2 by using the **Put Data** command.

Other public keys can exist within the application but these must be stored in transparent EFs.

Diffie–Hellman Key Exchange Parameters

The Diffie–Hellman key exchange parameters used in asymmetric key mutual authentication are also stored as a public key data object

The tags for Diffie–Hellman parameters are shown in the following table:

Tag	Length	Value
86h	80h	DH: p element
87h	14h	DH: q element
88h	80h	DH: g element

Table 7 - Tags in DH Key Exchange Parameters Data Objects

When created, a DH key exchange parameters data object is specified as such by the choice of control reference template (CRT).

3

Public Key Cryptographic Concepts

RSA Key Pairs

RSA keys operate as key pairs, one private key and one public key. Information encrypted with one key can be decrypted only by the other key in the key pair. The main principle is as follows:

- Public keys can be used to verify a signature or a certificate. They can also be used to encrypt data as encrypted data can be decrypted only by the owner of the corresponding private key.
- Private keys are used to generate signatures or decrypt data.

GemSafe v2 can store RSA key pairs. There are two main uses as follows:

- Mutual authentication
- For use in an application, either to sign or decipher data.

Data signed by a private key can be verified only by the public key in the same key pair and vice–versa.

All private and public keys in GemSafe v2 are created as data objects using the **Put Data** command during the personalization phase. The key's parameters are initialized (or entered) using one **Put Data** command for each key element.

RSA Key Identification

Apart from PK.IFD.AUT, all RSA keys are identified by a one-byte identifier (KID). They are never identified by a file identifier. PK.IFD.AUT uses the certificate holder reference (CHR) as a key name. This name is taken from the IFD certificate, C_CV.IFD.AUT, used in asymmetric key mutual authentication.

Note: All keys loaded during the personalization phase are identified by a KID that is set during this phase. The KID belongs to the key pair, that is, the private key and public key share the same KID.

Digital Signatures

A digital signature is simply the encryption of the hash of a message using a private key. This proves the sender of the message because the private key belongs only to its owner. However, since by definition the public key belongs to everybody, anybody can decrypt the signature. This is known as verifying the signature.

Performing a Digital Signature

The steps are as follows:

1. The sender hashes the original message using a hash algorithm.
2. The sender pads the hash, if necessary, to the required length.
3. The hashed message is ciphered with the sender's private key, using the **PSO–Compute Digital Signature** command. The result is known as a signature.
4. The signature is appended to the original message and sent.

The following figure illustrates this process:

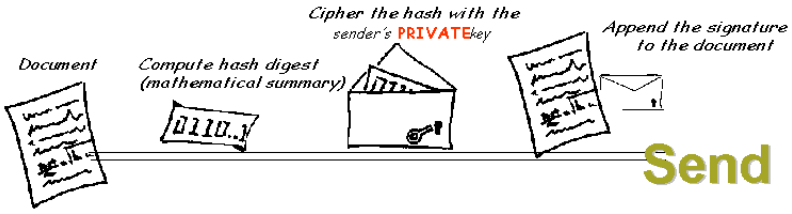


Figure 2 - Performing a Digital Signature

The private key is securely stored in the card. As the signature has been made using the sender's private key, then the message cannot have been sent by anybody else.

Hashing the Original Message

This hash can be performed in one of three ways:

- entirely by the card
- entirely externally
- partly by the card and partly externally (in this case the data is split)

GemSafe v2 uses the secure hash algorithm, SHA–1. If the hash is performed entirely externally, the algorithm can be different.

Padding the Hash

Digital signatures require input data (known as digital signature input (DSI)) whose length is the same as the length of the key modulus. The hash will be shorter than this length and needs to be padded accordingly. The DSI must also conform to a particular format, so the hash cannot simply be padded by adding a padding character. For this reason, GemSafe v2 performs the necessary padding.

The padding depends on the algorithm ID in the SE. This is either the current SE or an SE set by an **MSE–Set** command before computing the digital signature. The following table shows the algorithm ID values and their meanings:

Algo ID Value	Meaning
0xh	No hash indicated. Either the hash function is defined implicitly or is not applicable.
1xh	SHA–1
x1h	RSA with padding according to ISO 9796–2
x2h	RSA with padding according to PKCS#1 v2.x
x3h	RSA with padding according to RFC 2409
xxh	All other values are RFU

Table 8 - Algorithm ID Values

The most significant nibble indicates the hash algorithm to use. The least significant nibble indicates the cryptographic algorithm to use to compute the digital signature.

To perform the hash inside the card, or partially inside the card, GemSafe v2 provides a specific command. For information about this command see “PSO–Hash” on page 168.

Note: Even if all the hashing is performed externally, you must still perform the **PSO–Hash** command in order to set the hashed message for the **PSO–Compute Digital Signature** command.

Algo ID = 01h—Hash Performed Externally—ISO 9796–2 Padding

With ISO padding, the DSI includes padding bytes of 00h (ending with a 01h byte) and an eight–byte random number generated by the card as shown in the following table:

Value	Description	Length (in bytes)
60h	Padding indicator	1
00h, 00h, ..., 01h	ISO padding string	Key modulus length - (10 + Input data length)
XXh XXh ...XXh	Card random number	8
Input data	Data to be signed	1–36
BCh	ISO padding byte	1

Table 9 - Algo ID = 01h: Hash Performed Externally; ISO 9796–2 Padding

Algo ID = 02h—Hash Performed Externally—PKCS #1 Padding

With PKCS padding, the DSI includes padding bytes of FFh as shown in the following table:

Value	Description	Length (in bytes)
00h	Start byte	1
01h	Block type (01 for signature)	1
FFh, FFh...FFh	Padding string	Key modulus length - (3 + Input data length)
00h	Separator	1
Input data *	Data to be signed	1–36

Table 10 - Algo ID = 02h: Hash Performed Externally; PKCS #1 Padding

* If your hash computation involves a header, (as is often the case with the SHA-1 header for example), remember to include this header in the data to be signed.

Algo ID = 03h—Hash Performed Externally—RFC 2409 Padding

With PKCS padding, the DSI includes padding bytes of 00h (ending with a 01h byte) and an eight–byte random number generated by the card as shown in the following table:

Value	Description	Length (in bytes)
00h	Start byte	1
02h	Block type	1
XXh XXh ...XXh	Card random number inserted by the card (with no “00h” bytes)	Key modulus length - (3 + Input data length)
00h	Separator	1
Input data	Typically a hash result	16–36

Table 11 - Algo ID = 03h: Hash Performed Externally; RFC 2409 Padding

Algo ID = 11h—SHA–1 Hash Performed by Card—ISO 9796–2 Padding

Whether GemSafe v2 performs the hash partially or entirely, the result must always be a length of 20 bytes. With ISO 9796 padding, the DSI includes padding bytes of 00h (ending with a 01h byte) and an eight–byte random number generated by the card as shown in the following table:

Value	Description	Length (in bytes)
60h	Padding indicator	1
00h, 00h, ..., 01h	ISO 9796–2 Padding string	Key modulus length - 30
XXh, XXh, ...XXh	Card random number	8
Hash value	Hash result	20
BCh	ISO Padding byte	1

Table 12 - Algo ID = 11h: Hash Performed by Card; ISO 9796–2 Padding

Algo ID = 12h—SHA–1 Hash Performed by Card—PKCS#1 Padding

Whether GemSafe v2 performs the hash partially or entirely, the result must always be a length of 20 bytes. With PKCS#1 padding, the DSI includes padding bytes of FFh and the SHA–1 header as shown in the following table:

Value	Description	Length (in bytes)
00h	Start byte	1
01h	Block type (01 for signature)	1
FFh, FFh.....FFh	Padding string	Key modulus length - 38
00h	separator	1
30h, 21h, 30h, 09h, 06h, 05h, 2Bh, 0Eh, 03h, 02h, 1Ah, 05h, 00h, 04h, 14h	SHA–1 header	15
Hash value	Hash result	20

Table 13 - Algo ID = 11h: Hash Performed by Card; PKCS#1 Padding

Creating the Signature

GemSafe v2 uses the DSI to compute the digital signature. The current SE tells it which algorithm to use (always RSA but can be with either ISO 9796–2, PKCS#1 or RFC 2409 padding) and gives the key references.

To compute the signature, perform the **PSO–Compute Digital Signature** command. For details on this command, see “PSO–Compute Digital Signature” on page 162.

Verifying a Digital Signature

As the signature is created using the sender’s private key, it can be verified only by the sender’s corresponding public key. Since this does not require a high level of security, it is typically performed without a smart card so GemSafe v2 is not involved in this operation. The principle of signature verification is shown for information only.

1. The receiver uses the sender’s public key to decrypt the signature. If the hashed message was indeed signed by the sender’s private key, then the decrypted message should be the hashed message.
2. The receiver hashes the original message and compares it with the result obtained in step 1. If the two match, then the sender is authentic.

The following figure illustrates this process:

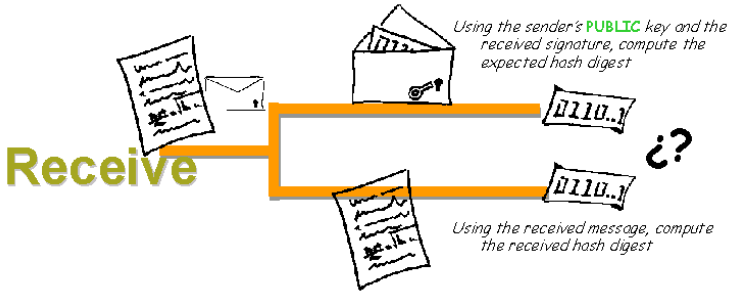


Figure 3 - Verifying a Digital Signature

Encryption and Decryption

Public key pairs can be used to encrypt and decrypt data. Typically this data is a session key, such as a 3DES key, that has been used to encrypt a message. The public key of the receiver (in this case the card) is used to encrypt the 3DES session key. The receiver's private key is used to decrypt the 3DES session key. Since by definition a public key is available to anybody, the encryption operation does not have to be done by a smart card. However the encryption must be done in a format that can be decrypted by GemSafe v2.

GemSafe v2 provides a decryption function, **PSO–Decipher** because this is a sensitive operation performed with a private key.

Encrypting a Message

The steps are as follows:

1. The sender encrypts a document with a one-time symmetric key. Typically this is a 3DES session key.
2. The sender encrypts the session key with the receiver's RSA public key with padding according to PKCS#1.
3. The sender sends the encrypted key and encrypted message to the receiver.

The following figure illustrates this process:

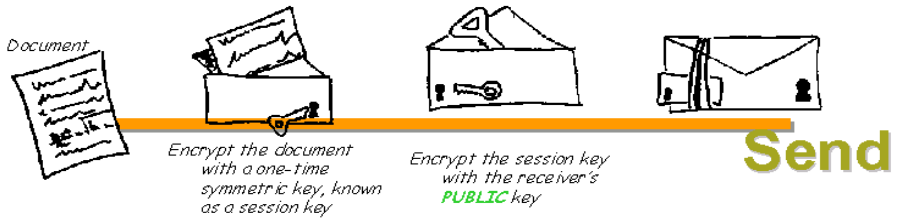


Figure 4 - Encrypting a Message

Decrypting a Message

The steps are as follows:

1. The receiver uses GemSafe v2 to decrypt the session key by using the **PSO–Decipher** command with the receiver's private key.
2. The receiver decrypts the message with the session key. This step is typically performed by application software.

The following figure illustrates this process:



Figure 5 - Decrypting a Message

On Board Key Generation

The GemSafe v2 applet provides a special command, **Generate Public Key Pair** that enables you to update the value of a public key pair during the application phase. This feature has several advantages:

- GemSafe v2 performs the computation of the key values for you.
- As the update takes place within the applet, GemSafe v2 handles the security of the operation instead of the application.
- The **Generate Public Key Pair** command must satisfy the security attributes for the private key data object.
- The new value of the private key never leaves the card.
- It enables you to manage the life span of a key pair easily, within an application.

The **Generate Public Key Pair** command writes the new value of the private key and public key in the card and returns the new value of the public key in its response. For more detailed information, refer to “Generate Public Key Pair” on page 144.

Mutual Authentication

The GemSafe v2 applet provides two types of mutual authentication mechanism where the card and the terminal must authenticate each other before they can exchange sensitive information:

- Asymmetric key mutual authentication based on the card and terminal verifying the existence of a certified key pair in the other entity and deriving a common session key using the Diffie–Hellman key exchange mechanism.
- Symmetric key mutual authentication based on the card and terminal verifying the existence of two 3DES secret keys in the other entity and deriving a common session key.

For more details, refer to “Chapter 8 - Mutual Authentication”.

4

Installing the GemSafe v2 Applet

The GemSafe v2 applet is present in the ROM of the following cards:

- GemXpresso Pro R3.2 E64 PK CC
- GemXpresso Pro R3.2 E36 PK CC

The applet is multi–instantiable, that is, you can install more than one instance of the applet.

The installation is performed in three stages:

1. The Host selects the Card Manager in the card.
2. The Host opens a secure channel by sending an **Initialize Update** command followed by an **External Authenticate (Perso)** command.

   Note: The **Initialize Update** and **External Authenticate (Perso)** commands are performed by the Card Manager and are reproduced in this document for information only. For descriptions of these commands, see “Initialize Update” on page 97 and “External Authenticate (Perso)” on page 87.

3. The Host sends an **Install** command in **Install** mode, to tell the Card Manager to install the GemSafe v2 applet from the load file, commonly referred to as “Install for Install”.

These steps are shown in “Figure 6” on page 22.

Note: Gemplus can provide software tools such as RAD III and JCard Manager 4.0, that manages the installation of applets. For details, please ask your Gemplus technical consultant.

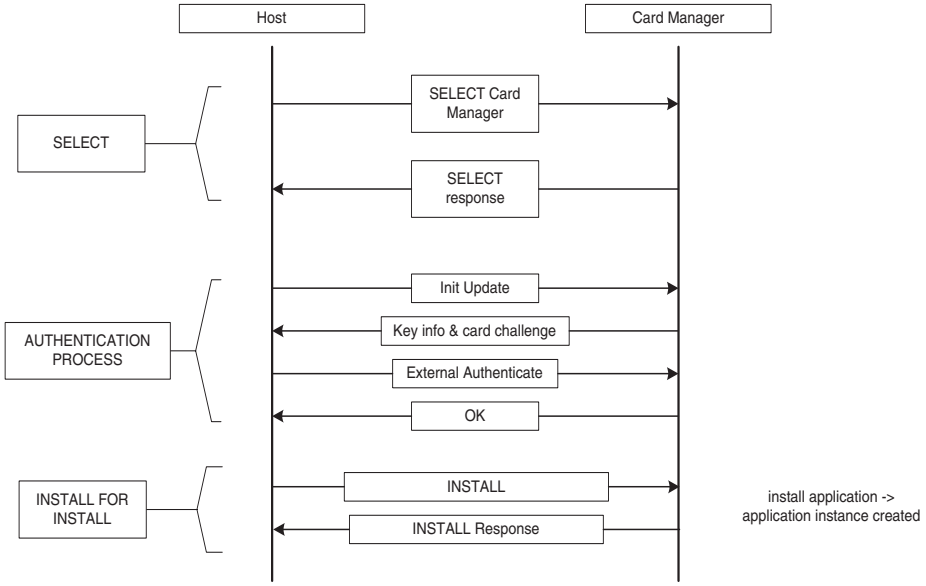


Figure 6 - Command Sequence to Install the GemSafe v2 Applet

For information about the **Install** command, see the *GemXpresso Pro R3.x Reference Manual*.

Install Command Parameters

The data to use for the **Install** command is described in *GlobalPlatform Card Specification 2.1.1*. For GemSafe v2, the data is as shown in the following table:

Presence	Length (in bytes) (decimal)	Name
Mandatory	1	Length of load file AID
Mandatory	5-16	Load file AID (often called “Package AID”)
Mandatory	1	Length of AID within load file
Mandatory	5-16	AID within load file (often called “Applet AID”)
Mandatory	1	Length of application instance AID
Mandatory	5-16	Application instance AID (often called “Instance AID”)
Mandatory	1	Length of application privileges
Mandatory	1	Application privileges[1]
Mandatory	1	Length of **Install** parameter field
Mandatory	2-n	**Install** parameter field [2] (see Table 15 on page 24)
Mandatory	1	Length of **Install** token
Conditional	0	**Install** token [3]

Table 14 - Parameters for Install Command in Install Mode

[1] The GemSafe v2 applet does not require any application privileges, but you can specify that an instance of the applet is selected as the default applet following an answer to reset (ATR). For this, the application privileges byte must be coded to 04h. For more details, refer to the *GemXpresso Pro R3.x Reference Manual*.

[2] The only parameters used in the **Install** parameter field are the GemSafe v2–specific parameters (see Table 15 on page 24)

[3] This parameter is mandatory for delegated management. As GemSafe v2 does not use delegated management, this value is absent (and the length parameter takes the value 00h).

GemSafe v2–Specific Parameters

The parameters in the **Install** parameter field are as follows:

Tag	Length (in bytes)	Name
C9h	var.	GemSafe v2–specific parameters (see Table 16 on page 24)

Table 15 - Install Parameter Field

The GemSafe v2–specific parameters are as shown in the following table:

Tag	Length (in bytes) (decimal)	Name
DF0Ah	6	Authentication key option byte (1 byte) || Option parameters (5 bytes) (see “Option Parameters” on page 25)
DF0Bh	1–3	Root security attributes (to determine the creation of DFs and EFs under the root). See “Access Mode Byte” on page 33.

Table 16 - GemSafe v2–Specific Parameters

Authentication Key Option Byte

This byte precedes the option parameters and defines the type of key to be used in authentication. It is coded as follows:

	b8	b7	b6	b5	b4	b3	b2	b1
Not checked	X	X	X	X	X			
Asymmetric authentication key						0	0	1
Symmetric authentication key (K_{ENC} & K_{MAC})						0	1	0

Table 17 - Authentication Key Option Byte Coding

Note: One and only one type of key must be specified. Any values of b3 b2 b1 other than those shown in “Table 17” return an error (6Ah 80h).

Option Parameters

The option parameters are as shown in the following table:

Length	Name
1 byte	Authentication ratification counter (initial value of 03h to 0Fh - A5h means no limit). This is used in both symmetric and asymmetric key authentication.
1 byte	Maximum number of RSA AUT public keys, M, used only in asymmetric key authentication (Cannot exceed 5).
1 byte	Maximum number of symmetric secret key pairs (K_{ENC} and K_{MAC}), used only in symmetric key authentication (Cannot exceed 15)
1 byte	Maximum number of RSA key pairs, N, (not including SK_ICC_AUT) (Cannot exceed 255)
1 byte	Application options (see Table 19 on page 25)

Table 18 - Option Parameters

Note: For CC certified applications, the authentication ratification counter must not be set to a value greater than 0Fh.

The Application options byte is coded as follows:

	b8	b7	b6	b5	b4	b3	b2	b1
Not checked	x	x	x	x	x	x	x	
Change PIN before first use activated								1
Change PIN before first use deactivated								0

Table 19 - Application Options Byte Coding

The “Change PIN before first use” option is used only by the **PSO–Compute Digital Signature** command. When activated, this means that if the signature key used by the **PSO–Compute Digital Signature** command is protected by one or more PINs, then these PINs must have been changed at least once since personalization for the signature key to be usable.

For a high-security certified signature application (Common Criteria certification) the “Change Pin before first use” option, is mandatory. For such a high-security product, the end-user must verify that the card received from the card-issuer has never been used to generate a signature before.

In order to proceed to this verification, the end-user should perform the following steps:

1. Perform a PIN verification using the PIN received by mail from the card-issuer.
2. Sign a message. The signing operation fails.
3. Change the PIN as required.
4. Verify the new PIN.
5. Sign a message. The signing operation is successful.

The card-issuer must provide the appropriate instructions to the developer of the signature creation application (SCA) and to the end-user to perform this operation.

5

Personalizing the GemSafe v2 Applet

Personalization According to Authentication Mechanism

Some files and data objects in the GemSafe v2 applet differ according to whether the application uses asymmetric or symmetric key mutual authentication. This chapter shows an example of how to personalize the applet for each type of authentication.

Personalization Example for an Asymmetric Key Mutual Authentication Application

In this example, the following files and objects are to be created:

- Two PIN objects (one signature PIN that is used to protect a signature private key and one security officer (SO) PIN that is used to unblock the signature PIN)
- A signature key object (containing an RSA private key, used for signatures); SK.ICC.DS
- A certification authority (CA) public key object (containing an RSA public key); PK.CA.AUT
- A card authentication key object (containing an RSA private key); SK.ICC.AUT
- A Diffie–Hellman key exchange parameters object
- Two security environment (SE) objects
- A C.ICC.AUT file that stores the card's certificate used in asymmetric key mutual authentication
- A Display Message EF used to store a message that displays on successful completion of an asymmetric key mutual authentication

Note: The Display Message EF can be used to indicate the a successful device authentication in order to inform the cardholder of the presence of a secure channel, as described in *CWA 14890 2004.* Its presence is optional, but if used there must be one such file only.

The stages to personalize the applet and the corresponding command APDUs to use are shown in the following table:

Step	Description	APDU
1.	Select the GemSafe v2 applet in the card.	**Select File**
2.	Authenticate the card to the personalization device.	**Initialize Update**
3.	Open secure channel between the personalization device and the card, and authenticate the personalization device to the card.	**External Authenticate (Perso)**
4.	Create data objects in the card, that is, the PIN objects, key objects and SE objects. The order of creating the objects does not matter, except that you must personalize the SO PIN before the signature PIN because the SO PIN is needed to protect the signature PIN. One **Put Data** command is used for each PIN object. Two or six **Put Data** commands are used for each private key object (in this example the signature key and the authentication key). The first **Put Data** command creates the data object, the five key elements: p, q, dp, dq, and iq can be initialized as five separate commands or combined in a single command. Two or three **Put Data** commands are used for each public key data object (in this example the CA public key). The first **Put Data** command creates the data object, the two key elements, modulus n and exponent e, can be initialized as two separate commands or combined in a single command. Two or four **Put Data** commands are used for the DH key exchange parameters data object. The first **Put Data** command creates the data object, the three DH elements p, q and g can be initialized as three separate commands or combined in a single command. One **Put Data** command is used for each SE object.	**Put Data**
5.	Create the Display Message, and C.ICC.AUT EFs as follows	
a.	Select the DF	**Select File**
b.	Create the EF	**Create File**
c.	Select the EF that you have just created	**Select File**
d.	Write the data to the EF.	**Update Binary**
6.	Create the DFs and EFs for your application in the same way (create the DF first and select it to create the EFs). Use the **Update Binary** command to write the data to the EFs. You can also create EFs directly under the root.	**Select File**, **Create File**, **Update Binary**
7.	Complete the personalization.	**End Personalization**

Table 20 - Command Sequence to Personalize the GemSafe v2 Applet

The file architecture after this personalization example looks like this:

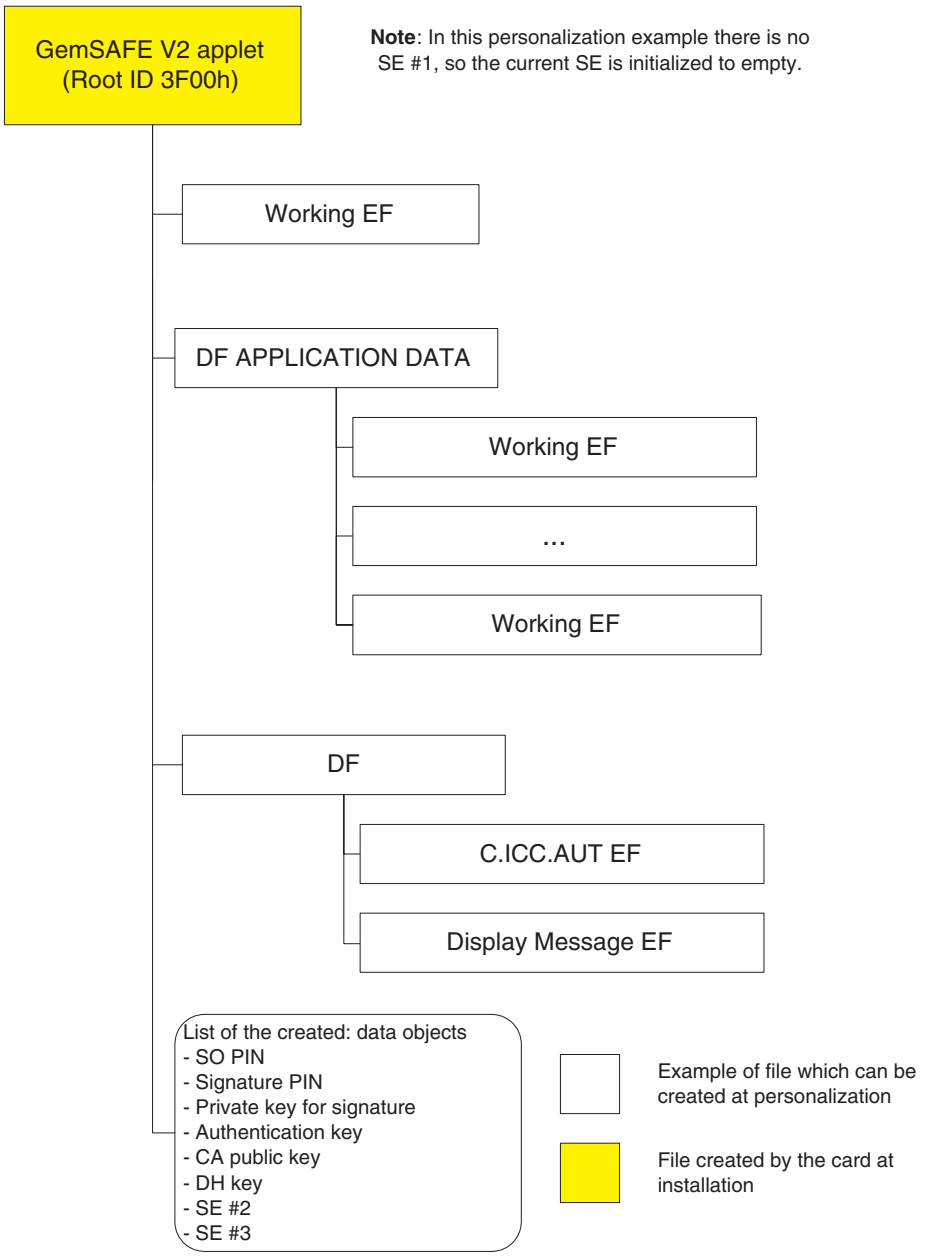


Figure 7 - Example File Architecture After Personalization (1)

The EFs C.ICC.AUT and Display Message are shown under the same DF for illustration purposes only.

Personalization Example for a Symmetric Key Mutual Authentication Application

In this example, the following files and objects are to be created:

- Two PIN objects (one signature PIN that is used to protect a signature private key and one security officer (SO) PIN that is used to unblock the signature PIN)
- A signature key object (containing an RSA private key, used for signatures)
- The two 3DES secret keys, K_{ENC} and K_{MAC}, necessary for mutual authentication
- A utility key data object
- Two security environment (SE) objects
- A Display Message EF used to store a message that displays on successful completion of a symmetric key mutual authentication

Note: The Display Message EF can be used to indicate the a successful device authentication in order to inform the cardholder of the presence of a secure channel, as described in *CWA 14890 2004*. Its presence is optional, but if used there must be one such file only.

The stages to personalize the applet and the corresponding command APDUs to use are shown in the following table:

Step	Description	APDU
1.	Select the GemSafe v2 applet in the card.	**Select File**
2.	Authenticate the card to the personalization device.	**Initialize Update**
3.	Open secure channel between the personalization device and the card, and authenticate the personalization device to the card.	**External Authenticate (Perso)**
4.	Create data objects in the card, that is, the PIN objects, key objects and SE objects. The order of creating the objects does not matter, except that you must personalize the SO PIN before the signature PIN because the SO PIN is needed to protect the signature PIN. One **Put Data** command is used for each PIN object. Two or six **Put Data** commands are used for each private key object (in this example the signature key and the utility key). The first **Put Data** command creates the data object, the five key elements: p, q, dp, dq, and iq can be initialized as five separate commands or combined in a single command. One or Two **Put Data** commands are used for the secret keys (one if the keys are initialized at the time of creation, two if they are initialized after creation). The **Put Data** command creates/ initializes the K_{ENC} and K_{MAC} keys simultaneously as they are stored in the same secret key object. One **Put Data** command is used for each SE object.	**Put Data**

Table 21 - Command Sequence to Personalize the GemSafe v2 Applet

Step	Description	APDU
5.	Create the Display Message EF as follows	
a.	Select the DF	**Select File**
b.	Create the EF	**Create File**
c.	Select the EF that you have just created	**Select File**
d.	Write the data (the display message) to the EF.	**Update Binary**
6.	Create the DFs and EFs for your application in the same way (create the DF first and select it to create the EFs). Use the **Update Binary** command to write the data to the EFs. You can also create EFs directly under the root.	**Select File**, **Create File**, **Update Binary**
7.	Complete the personalization.	**End Personalization**

Table 21 - Command Sequence to Personalize the GemSafe v2 Applet (continued)

The file architecture after this personalization example looks like this:

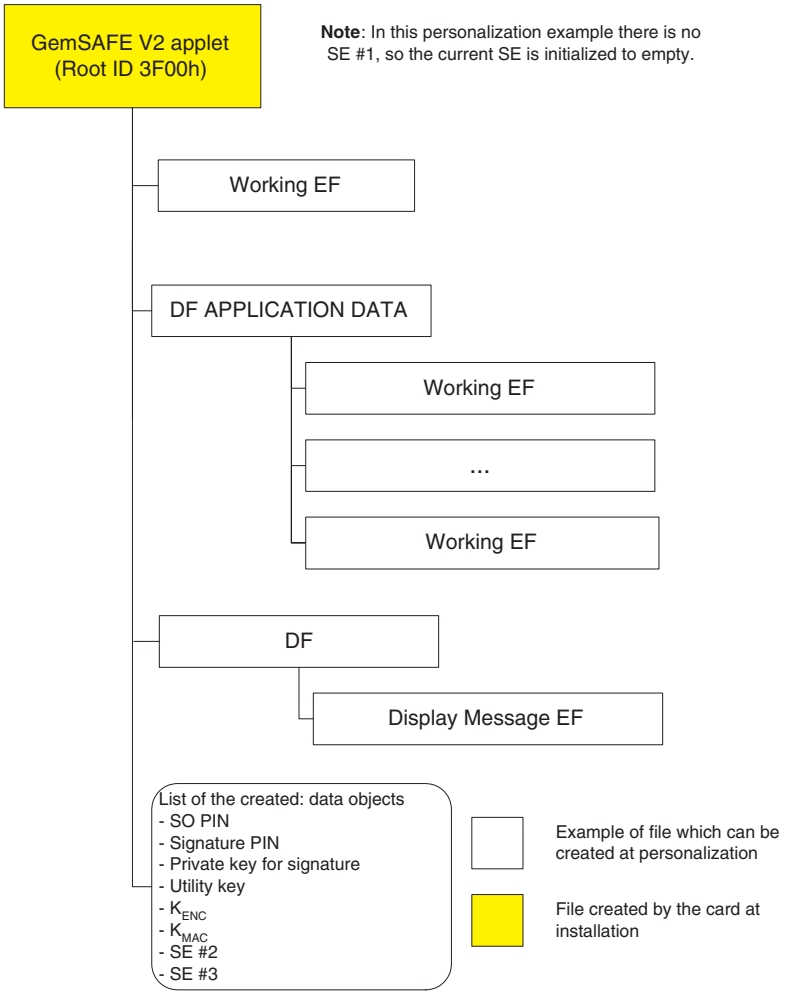


Figure 8 - Example File Architecture After Personalization (2)

PKCS#15 Personalization Profiles

If your application is to use PKCS#15 profiles, you need to create particular files. For more information about these files and PKCS#15 profiles, refer to *ISO/IEC 7816-15: Cryptographic Information Application.*

6

Security Features

Security Attributes

Files and data objects are protected by security attributes, that must be fulfilled before access is granted to the file or data object. These security attributes are defined at the time of file creation. For more details, see “File Control Parameters” on page 7. Each file or data object has one security attribute that is made up of:

- One access mode (AM) byte, that defines the command(s) to be protected against
- One security condition (SC) byte for each bit set to 1 in the AM byte

Each SC byte defines the conditions that must be fulfilled in order to allow the corresponding command to be performed.

Note:

1. Security attributes are checked by commands only after personalization.
2. Security Environments do not have security attributes because they cannot be modified.

The security attributes are coded in TLV format. For an example, see “Security Attributes Example” on page 36:

Access Mode Byte

The access mode byte determines the commands that are to be controlled. Its coding has a different meaning for files and data objects since the commands that are used with them are different.

Caution: In both cases, if a bit in the AM byte is set to zero, that is there is no corresponding SC byte, then the security attribute for the corresponding command is NEVER.

The AM byte is coded as follows:

For DFs

Meaning	b8	b7	b6	b5	b4	b3	b2	b1
	0							
Create DF (applies only to the root)						1		
Create EF							1	

Table 22 - Access Mode Byte Coding – For DFs

As DFs can be created under the root only, the Create DF condition is used only by the root.

For EFs

Meaning	b8	b7	b6	b5	b4	b3	b2	b1
	0							
Activate File				1				
Deactivate File					1			
Update Binary, **Erase Binary**							1	
Read Binary								1

Table 23 - Access Mode Byte Coding – For EFs

Bit 8 must be zero for EFs. Bits b7 to b1 are independent of each other, that is, more than one bit can be set to 1.

For Data Objects

Meaning	Type of Data Object	b8	b7	b6	b5	b4	b3	b2	b1
	—	1							
Reset Retry Counter	PIN		1						
Change Reference Data	PIN			1					
Verify	PIN				1				
PSO Compute Digital Signature, **PSO: Decipher**	Private Key					1			
Put Data, **Generate PK Pair**	Secret/Private/ Public Key Private Key DH Key Parameters							1	
Get Data	DH Key Parameters Public Key								1

Table 24 - Access Mode Byte Coding – For Data Objects

Bit 8 must be 1 for data objects, that is for secret keys, private and public keys and PINs. Bits b4 and b2 are independent of each other, that is, both can be set to 1.

Security Condition Byte

There is one security condition (SC) byte for each bit set to one in the AM byte. Each SC byte is coded as follows:

Meaning	b8	b7	b6	b5	b4	b3	b2	b1
No condition	0	0	0	0	0	0	0	0
Never	1	1	1	1	1	1	1	1
Conditions	x	x	x	x				
At least one condition (OR) (b7 to b5)	0							
All conditions (AND) (b7 to b5)	1							
Secure Messaging (command & response)		1						
Mutual Authentication			1					
User Authentication (PIN)				1				
Security Environment Reference					x	x	x	x
No SE referenced					0	0	0	0
SE# 0001–1110					—	—	—	—
RFU					1	1	1	1

Bits 8–5 indicate the conditions that must be fulfilled to access the file or data object.

Bit 7 set to 1 indicates that the command and the response must be sent with secure messaging. You cannot specify the level of secure messaging, that is “MAC” or “MAC and encryption”, as this is governed by GemSafe v2’s built–in security policy. For more details, refer to “Built–in Security Policy” on page 37.

Bit 6 set to 1 indicates that mutual authentication must be successfully executed before access is granted. If the application uses asymmetric key mutual authentication, then this means the extraction of the role ID from the CHA in the terminal certificate C_CV.IFD.AUT during processing of the **External Authenticate** command. For access to be granted, the extracted role ID must match that indicated in the security environment. The value of the role ID has no significance for GemSafe v2 and can be defined at the application level.

If the application uses symmetric key mutual authentication, bit 6 set to 1 means the **Mutual Authentication** command must be successfully executed before granting access.

Bit 5 set to 1 (user authentication) indicates that the PIN indicated by the security environment must be correctly presented to allow access to the file or data object. If no SE is referenced, the default SE #1 is used. GemSafe v2 returns the error 6982h if the referenced SE does not exist or if the PIN referenced by the SE does not exist.

Bits 4–1 indicate the security environment to use. They are used only if at least one bit in bits 7–5 are set to one.

Security Attributes Example

The AM byte and SC bytes are encapsulated together in TLV format under the tag “8Ch”. The length depends on the number of SM bytes specified in the AM byte. The following table shows an example of how a security attribute could be coded:

Tag	Length (bytes)	Value	Meaning
8Ch	05h		Tag and length of the security attribute (AM byte + 4 SC bytes)
		1Bh	AM byte. All four SC bytes for files are present.
		14h	SC byte for **Activate File** command. PIN protection defined in SE#4.
		35h	SC byte for **Deactivate File** command. **Mutual Authentication** OR PIN protection (using the PIN referenced in SE#5).
		B3h	SC byte for **Update Binary** and **Erase Binary** commands. **Mutual Authentication** AND PIN protection (using the PIN referenced in SE#3).
		00h	SC byte for **Read Binary** command. No condition.

Table 25 - Security Attributes Coding Example

Caution: Care should be taken when coding security attributes, particularly when they involve security environments and linking command usage to life cycle status. To illustrate the potential problems, here is an example.

A security attribute is coded so that in SE#4, the **Activate File** command can be performed only when the life cycle status of the file is INITIALIZED. If the file is later deactivated by a **Deactivate File** command, you will never be able to reactivate it in SE#4 because the status cannot revert to INITIALIZED.

Built–in Security Policy

The previous section described how to code security attributes. However, as an extra safety precaution, GemSafe v2 imposes a minimum level of secure messaging for certain commands, regardless of how the security attributes are coded during personalization.

The following table shows these commands and their corresponding level of require secure messaging.

Command	Minimum SM
Generate Public Key Pair	MA + MAC
Put Data–Secret Keys	MA + MAC + ENC
Put Data–Private Keys	MA + MAC + ENC
Put Data–Public Keys	MA + MAC
Put Data–DH Key Exchange Parameters	MA + MAC

Table 26 - Built-in Minimum Levels of Secure Messaging

Where:

MA means mutual authentication
MAC means the command must be sent with a MAC
ENC means the command must be encrypted

For further details, see “Secure Messaging During the Application Phase” on page 43.

PIN Management

PINs are used to identify the cardholder and to control access to data. It is possible for example to create two PINs, one for security officers and another user PIN to protect signatures, that is, a PIN which must be presented in order to perform the **Compute Digital Signature** command.

PINs are created using the **Put Data** command. The **Put Data** command can also be used to update a PIN up until the end of the applet personalization, that is, until the **End Personalization** command is performed. After personalization, PIN updates can be made using the **Change Reference Data** or **Reset Retry Counter** commands. PINs are presented using the **Verify** command

When using the **Put Data** command, the format of the PIN is in tag, length, value (TLV) format as follows:

Tag	Length	Value
FFh 20h	var	See “PIN Contents” on page 38.

Note: The PIN also contains a Try Counter and a length field whose values are not included in the **Put Data** command because they do not need to be initialized.

PIN Contents

A GemSafe v2 PIN object is composed of the following elements:

Tag	Length	Proprietary PIN template		
FFh 20h	var	**Tag**	**Length**	**Value**
		83h	01h	PIN Reference
		8Ch	var	Security attributes
		DFh 21h	04h	PIN attributes
		DFh 22h	L_{PIN}	PIN value

Table 27 - Structure of a PIN Data Object

Where:

PIN Reference = 8x (x is the PIN ID from 1 to Fh)

Security Attributes = AM Byte

SC Byte for **Reset Retry Counter** command

SC Byte for **Change Reference Data** command

SC Byte for **Verify** command

PIN attributes = Try Limit (1 byte)

Usage Counter (1 byte)

Unblocking Counter (1 byte)

Unblocking PIN reference (1 byte)

L_{PIN} = Length of PIN (8–16 bytes). This cannot be changed by any command.

PIN value = Value of PIN (L_{PIN} bytes). An application can specify a length shorter than L_{PIN}, in which case it must pad the PIN with any value to make it a length of L_{PIN}. At the GemSafe v2 level, the PIN is always considered as length L_{PIN}.

Note:

1. The presence of SC bytes in the security attributes depends on the coding of the AM byte. See Table 24, “Access Mode Byte Coding – For Data Objects”, on page 34 for more details.
2. For CC certified applications, the PIN must contain at least 6 digits.

The following table describes the Try Counter and the PIN attributes:

Element	Description
Try Counter	The number of remaining attempts that can be made to present the correct PIN before it is blocked. The initial value of this is equal to the value of Try Limit. It is decremented for each incorrect PIN verification. When this value reaches zero, the PIN is blocked
Try Limit	The maximum number of consecutive incorrect PIN presentations that can be made before the PIN is blocked (the maximum value is 0Fh).
Unblocking Counter	The number of times that the PIN can be unblocked. This counter can be set to maximum of 0Fh and is decremented each time that the PIN is unblocked. Once this counter reaches zero, the PIN can no longer be unblocked. Setting this counter to A5h means that the PIN can always be unblocked.
Usage Counter	The number of times that the PIN can be verified. This counter can be set to maximum of 0Fh and is decremented each time that the PIN is verified. Once this counter reaches zero, the PIN can no longer be verified. Setting this counter to FFh means that the PIN can always be verified. This feature is useful to limit the use of an unblock PIN for example.
Unblocking PIN reference	The reference of the PIN that must be used to unblock the reference PIN.

Table 28 - PIN Attributes

Note: For applications that are CC certified, the Try Limit must not exceed 03h.

Verifying a PIN

The PIN is verified using the **Verify** command. This command compares the value entered by the cardholder with the reference PIN, that is the PIN that is stored in the card. For details about the **Verify** command, see “Verify” on page 194.

PIN Status after Card Reset or Reselection of Applet

Following a card reset or the selection of the GemSafe V2 applet, the PIN validation flag for all local PINs are set to false. In other words, the local PINs need to be reverified.

Unblocking a PIN

The Try Counter counts the number of remaining attempts to verify a PIN. Initially it is set to the value of Try Limit and is decremented with each unsuccessful PIN verification. If the Try Counter reaches zero, the PIN is blocked.

To unblock a PIN:

Perform the **Reset Retry Counter** command to verify the PIN specified in the Unblocking PIN reference of the reference PIN and optionally change the value of the reference PIN. This action decrements the reference PINs Unblocking Counter and resets the reference PIN’s Try Counter to PIN Try Limit. For details about the **Reset Retry Counter** command, see “Reset Retry Counter” on page 186.

Note: If the Unblocking PIN reference is 00h, the PIN cannot be unblocked.

The number of times that you can unblock a PIN is limited by the PIN's unblocking counter. Once this value reaches zero, the PIN can no longer be unblocked.

Changing a PIN Value

To change the value of a PIN in the card, use the **Put Data** command before the end of personalization and either the **Change Reference Data** or **Reset Retry Counter** command after the end of personalization. For details about these commands, see "Put Data–PINs" on page 102, "Change Reference Data" on page 132, and "Reset Retry Counter" on page 186.

Secure Messaging

Secure messaging is the means by which communication between a card and the terminal is protected. This section describes the secure messaging processes during personalization of the applet, followed by using secure messaging as part of the GemSafe v2 application.

Secure Messaging During the Personalization Phase

Before personalization, the Card Manager and terminal must authenticate each other, known as Card Manager/terminal authentication. This is done by an **Initialize Update** command, followed by an **External Authenticate (Perso)** command.

Note: The **External Authenticate (Perso)** command is the GlobalPlatform **External Authenticate** command that is also described in the *GemXpresso Pro R3.x Reference Manual*.

The **External Authenticate (Perso)** command sets the secure channel mode that determines the level of security for the subsequent personalization commands.

Secure Channel Modes

The *GlobalPlatform Card Specification 2.1.1* defines four modes of secure channel:

- No secure messaging—Card Manager/terminal authentication is necessary before performing any command, but no secure messaging is applied to the command.
- MAC mode—Card Manager/terminal authentication is necessary before performing any command and the command must be sent with a MAC.
- Encryption mode—Card Manager/terminal authentication is necessary before performing any command and incoming data must be encrypted.
- MAC and Encryption mode—Card Manager/terminal authentication is necessary before performing any command; the incoming data must be encrypted and the command must be sent with a MAC.

When the CARD life cycle state is SECURED, the **External Authenticate (Perso)** must specify the "MAC and Encryption" mode. However, if the CARD life cycle state is INITIALIZED, it can specify "No security", "MAC" or "MAC and Encryption" as the mode. For details about the CARD life cycle states, refer to the *GemXpresso Pro R3.x Reference Manual*. GemSafe v2 never uses the "Encryption" mode.

Once the mode of secure channel is established, you can continue personalizing the applet by issuing the commands detailed in “Personalization According to Authentication Mechanism” on page 27 with secure messaging.

Keys Used in Secure Messaging

The secure messaging process involves creating session keys from a set of keys that are contained in the Card Manager. The key set is identified by its version ID. Each key within a key set is identified by its index. For information about the keys in the Card Manager, refer to the *GemXpresso Pro R3.x Reference Manual*.

Computing Session Keys

Session keys are computed using the 3DES algorithm. There are two types of session key, a MAC session key and AUTH/ENC session key. Each selects a different type of key from the Card Manager, (MAC key and AUTH/ENC key respectively). The Card Manager keys contain an algorithm ID that indicates the mode of 3DES to use for the computation as shown in the following table:

Case	Key Type	Algo Reference	3DES Mode
MAC session key	MAC	82	$3DES_{CBC}$
Auth/Enc session key	AUTH/ENC	81	$3DES_{ECB}$

Table 29 - Types of Session Key

The session key is computed as follows:

Session Key = $3DES_{XXX}$ [$CRnd_{4-1}$ || $TRnd_{8-5}$ || $CRnd_{8-5}$ || $TRnd_{4-1}$] <KeySet[Index]>

Where:

$3DES_{XXX}$ is the 3DES algorithm in either ECB or CBC mode.

KeySet[Index] is the key set in the Card Manager (index indicates key in the key set).

$CRnd_{XX}$ is a random number generated by the card (xx indicates the bit numbers).

$TRnd_{XX}$ is a random number generated by the terminal (xx indicates bit numbers).

Commands Sent in MAC Mode

To ensure the integrity of the command, it is sent with an 8–byte checksum called a message authentication code (MAC). The data is padded until the length of the command header (that is, the bytes CLA, INS, P1, P2) + Lc + data is a multiple of eight bytes, then the MAC is computed using the session key. The procedure is as follows:

To send a command in MAC mode:

1. Compute the MAC session key, using a MAC key from a key set in the card and using 3DES in CBC mode, as described in “Computing Session Keys” on page 41.
2. Add one byte of “80h” to the data.

3. Pad the data with as many bytes of “00h” as necessary until the command header + Lc + data is a multiple of eight bytes, for example if the data was five bytes, five bytes of “00h” would be needed to make a total of 16 bytes.

1	1	1	1	1	5	1	1	3	1
CLA	INS	P1	P2	Lc	Data	80h	00h	...	00h

4. Compute the MAC as follows:

   $MAC_n = 3DES_{CBC}[MAC_{n-1} \| CLA \| INS \| P1 \| P2 \| Lc \| Data \| Padding]<MAC session key>$

   Where MAC_{n-1} is the MAC from the previous command transmitted to the card. If there is no previous MAC because this is the first MAC computation, MAC_{n-1} takes the value zero.

5. Send the command with the MAC as follows:

CLA	INS	P1	P2	Lc	Data	MAC

Note: Lc is the length of the data + MAC. As the MAC is eight bytes, if the data is empty (zero bytes) Lc is 8 bytes.

Commands Sent in Encryption Mode

Although this mode is not used in GemSafe v2, it is described here to help understanding of the “MAC and Encryption” mode. To ensure the confidentiality of the command, it is encrypted using $3DES_{CBC}$. The data is padded and then enciphered. The procedure is as follows:

1. Add an extra Lc byte, Lc′ before the Lc byte that follows the command header.

CLA	INS	P1	P2	Lc′	Lc	Data

   Lc′ takes the value Lc + 1, that is, Lc + Data.

2. If Lc′ is not a multiple of eight, add one byte of 80h to the end of the data, so the value of Lc′ increases by one.

CLA	INS	P1	P2	Lc′	Lc	Data	80h

3. If Lc′ is still not a multiple of eight, add as many bytes of 00h as necessary until it is a multiple of eight.

CLA	INS	P1	P2	Lc′	Lc	Data	80h	00h	...	00h

4. The data is then encrypted eight blocks at a time:

   Encrypted $data_1$ = $3DES_{CBC}$ [Block 1 of Clear Data] <Auth/ENC Session Key>
   ...
   Encrypted $data_n$ = $3DES_{CBC}$ [Block n of Clear Data] <Auth/ENC Session Key>

   Where:
   Block 1 of Clear Data = Lc || Data
   ...
   Block n of Clear Data = Data || Padding (if any)

5. The encrypted eight byte blocks are concatenated:

   Encrypted $data_1$ || Encrypted $data_2$ || ... || Encrypted $data_n$

6. The command is sent as follows:

CLA	INS	P1	P2	Lc	Encrypted $data_1$ || Encrypted $data_2$ || ... || Encrypted $data_n$

   Where:
   Lc is the total of the encrypted data fields (a multiple of eight).

Commands Sent in MAC and Encryption Mode

This combined operation is performed on data with incoming data. The procedure is:

1. Encrypt the data as described in “Commands Sent in Encryption Mode” on page 42.
2. Compute the MAC as described in “Commands Sent in MAC Mode” on page 41.

Secure Messaging During the Application Phase

After personalization, the GemSafe v2 applet enters the application phase.

During this phase, commands can use secure messaging only if mutual authentication between the card (the GemSafe v2 applet) and the terminal has taken place. This operation is performed by the **External Authenticate** command in the case of asymmetric key applications and the **Mutual Authenticate** command in the case of symmetric key applications. For details about the mutual authentication process, see “Chapter 8 - Mutual Authentication”. For a description of the **Mutual Authenticate** command, see “Mutual Authenticate” on page 160. For a description of the **External Authenticate** command, see “External Authenticate (Application)” on page 141.

The data objects concerned are listed in the following table:

Tag	Length	Data Object
T_{PV} = 81h	L_{PV}	PV = Plain value (to be protected by a MAC)
T_{CG} = 87h	L_{CG}	CG = PI || Cryptogram (to be protected by a MAC)
T_{MAC} = 8Eh	L_{MAC}	MAC
T_{Le} = 97h	L_{Le}	Le (to be protected by a MAC)
T_{SW} = 99h	L_{SW}	Status info (to be protected by a MAC)

Table 30 - Data Objects Used in Secure Messaging During Application Phase

Note:
1. PI is a padding indicator. In the GemSafe v2 applet, this is always 01h to indicate that padding is as defined in ISO/IEC 7816-4, that is one byte of 80h followed by as many bytes of 00h as necessary to make the data a multiple of eight bytes.

2. Tpv is the Tag for the plain value object, Lpv is the length of the plain value object. This same notation is used for Tcg, Lcg and so on.

Cryptograms are the result of encryption and are used to ensure the confidentiality of data. They are computed using the data value as input and using the $3DES_{CBC}$ algorithm with “Null” as the Initial Check Block and the $K_{SK}ENC$ session key. The last block of data is padded as defined in ISO/IEC 7816-4 to make it a multiple of eight bytes.

MACs are cryptographic checksums that guarantee the integrity of commands. They are computed on the entire command header, Lc and data. They are computed using the ANSI X9.19 “Retail MAC” algorithm as described “MAC Computation” on page 46.

For certain commands, the level of secure messaging (MAC or ENC and MAC) for each command is pre–defined by GemSafe v2’s built–in security policy. For more details refer to “Built–in Security Policy” on page 37.

If a security attribute specifies secure messaging, it must be used. However secure messaging can still be used even if not specified by the security attribute, as long as mutual authentication has taken place.

Command Formats

The cases referred to in this section are those defined in ISO 7816–4. Refer to “Command–Response Pairs” on page 69 for a reminder.

Table 30 on page 44 shows that Lc, Data, Le and SW are sent in TLV format and are protected by a MAC. If the data is encrypted, then a cryptogram (the result of the encryption) is sent instead of the plain data. **The class byte is always XCh**.

No Data. Cases 1 and 2 with MAC

For commands sending no data, the command format is:

XC	INS	P1-P2	Lc	Tle	01	Le	Tcc	Lcc	MAC

Plain Data. Cases 3 and 4 with MAC

For commands sending plain data, the command format is:

XC	INS	P1-P2	Lc	Tpv	Lpv	PV	Tcc	Lcc	MAC

Note: For compatibility with other GemSafe applets, the TLV of Le must not be used.

Encrypted Data. Cases 3 and 4 with MAC and Encryption

For commands sending data in encrypted format, the command format is:

XC	INS	P1-P2	Lc	Tcg	Lcg	PI,CG	Tcc	Lcc	MAC

When sending commands with secure messaging during the application phase, you must observe the following rules:

- The Class byte must always be XCh. X is zero for ISO commands and 8 for proprietary commands. The “C” tells GemSafe v2 to compute the MAC according to ISO 7816–4 with the command header included.
- Lc must be less than 248 in order to process the padding.

Input Data for MACs in Command APDUs

The data used in the MAC computation is as follows for the four ISO 7816–4 cases.

Case 1. Input data = CH || PB

Case 2. Input data = CH || PB || T_{LE} || L_{LE} || Le || PB

Case 3. Input data = CH || PB || T_{PV} || L_{PV} || PV || PB

Case 4. Input data = CH || PB || T_{PV} || L_{PV} || PV || PB

Note:

1. If the commands are being used with encryption, cases 1 and 2 are unchanged, but in cases 3 and 4, T_{PV} || L_{PV} || PV are replaced by T_{CG} || L_{CG} || PI,CG.
2. In all four cases, CH is the command header CLA || INS || P1 || P2.
3. The PB that follow the CH are 80h, 00h, 00h, 00h, in order to pad the CH to an eight–byte block. The PB at the end of the input data is according to ISO 7816-4 (80h, 00h,...00h until the input data becomes a multiple of eight bytes).

Response Formats

No Data. Cases 1 and 3 with MAC

For responses that do not contain data, the response format is:

Tsw	02h	SW1	SW2	Tcc	Lcc	MAC	SW1	SW2

Plain Data. Cases 2 and 4 with MAC

For responses returning plain data, the usual CLA, INS, P1, P2, Lc, Data, Le becomes:

Tpv	Lpv	PV	Tcc	Lcc	MAC	SW1	SW2

Input Data for MACs in Response APDUs

The data used in the MAC computation is as follows for the four ISO 7816–4 cases.

Cases 1 and 3 (and cases 2 and 4 if error). Input data = T_{SW} || L_{SW} || SW || PB

Cases 2 and 4. Input data = T_{PV} || L_{PV} || PV || PB

Note:

1. If the responses are being used with encryption, cases 1 and 3 are unchanged, but in cases 2 and 4, T_{PV} || L_{PV} || PV are replaced by T_{CG} || L_{CG} || PI,CG.
2. PB is the padding bytes according to ISO 7816-4 (80h, 00h,...00h until the input data becomes a multiple of eight bytes).

MAC Computation

Let X be the input data as described in “Input Data for MACs in Command APDUs” on page 45 or “Input Data for MACs in Response APDUs” on page 46.

The steps involved in computing a MAC are as follows:

1. Pad X by preceding it with the SSC and appending padding bytes according to ISO 7816-4 until the result is a multiple of eight bytes such that

   XX = SSC || X || 80h, 00h,...00h.
2. Compute YY = DES_{CBC} [XX] <Kl>
3. Compute Y = DES^{-1} [YY] <Kr>
4. Compute MAC = DES [Y] <Kl>

Where:

MAC — is the message authentication code.

SSC — is the Send Sequence Counter on eight bytes. The initial value of this is 4 LSB RND.ICC || 4 LSB RND.IFD.

Note: The SSC must be incremented before each new MAC computation, thus it could be incremented once for a command APDU and once for a response APDU.

Kl and Kr — are the left (8 MSB) and right (8 LSB) halves respectively of the session key K_{SK}MAC.

The following figure shows the MAC computation:

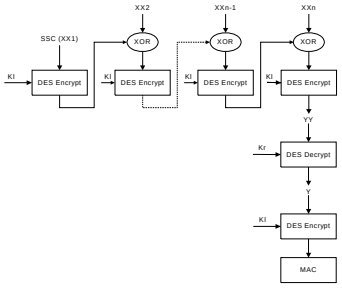


Figure 9 - Computing a MAC

Security Environments

A signature card may contain more than one utility key or signature key and more than one set of user reference data. GemSafe v2 manages the keys and data to use by referencing them in entities called security environments (SEs).

An SE defines the references to be used for secure mechanisms such as:

- Cryptographic algorithms
- PINs
- Keys (symmetric, public, private)
- Diffie–Hellman key exchange parameters
- Role IDs
- Modes of operation

There are two main situations where SEs are used:

- To control access to a file or data object. In this case the security attribute specifies the condition necessary to access the file (for example PIN) and the SE that contains the references to the secure mechanism (such as the reference to the PIN).
- To provide a context for signature creation, decryption and the verification of certificates. In this case the current SE specifies the keys and algorithms to use.

The GemSafe v2 applet can store up to a maximum of 14 SEs. There must be at least one SE, the default SE with SEID=1. After applet selection the current SE is initialized with the contents of the default SE. It can be empty, indicating no access restrictions.

The current SE is stored in RAM and is used only by the PSO and authenticate commands. To make a stored SE the current SE, use the **Manage Security Environment (MSE)–Restore** command. For more details, see “MSE–Restore” on page 157.

In GemSafe v2, SEs are stored as data objects in SE templates with a tag of 7Bh. To create an SE, use the **Put Data** command. For more details, see “Put Data–Security Environments” on page 101.

To modify the settings of the current SE, use the **MSE–Set** command. For more details, see“MSE–Set” on page 158.

Security Environments Structure

Security environments are made up of up to three components in GemSafe v2. They are coded in TLV format. The components of an SE are:

- The security environment identifier (SEID)
- An optional life cycle status (LCS) data object
- One or more control reference templates (CRT)

SEID

The SEID is a one–byte identifier that is unique for each SE. It is a data object with a tag value of 80h. The values for the SEID are shown in the following table:

Value	Meaning
00h	Empty SE
01h	Default SE
02h to 0Eh	SEs 2–14

Table 31 - SEID Values

LCS

The LCS data object has a tag value of 8Ah. It indicates the file life cycle status for which the SE is valid. This is useful to control access to files. GemSafe v2 allows access to the file if the LCS of the file matches the LCS of the SE. If there is no LCS data object present, the SE is valid for the OPERATIONAL (ACTIVATED) life cycle status. The LCS byte is coded as follows:

State	b8	b7	b6	b5	b4	b3	b2	b1
	0	0	0	0	X	X	X	X
CREATED					0	0	0	1
INITIALIZED					0	0	1	1
OPERATIONAL (ACTIVATED)					0	1	—	1
OPERATIONAL (DEACTIVATED)					0	1	—	0

Table 32 - LCS Values

CRT

GemSafe v2 supports five different types of control reference template (CRT), each defining the use of the SE and identified by its own tag. These templates and their corresponding purposes and tags are shown in the following table:

Tag	Template	Purpose
A4h	AT (authentication template)	Protect data objects by one of the following: Card holder verification (PIN reference) Symmetric key mutual authentication (key set) Asymmetric key mutual authentication (role ID). Verification of certificate used in asymmetric key mutual authentication
A6h	KAT (key agreement template)	Defines CA public key prior to certificate verification
B4h	CCT (cryptographic checksum template)	Protect data objects by specifying SM with MAC is necessary for the command, the response or both.
B6h	DST (digital signature template)	Defines the RSA private key and algorithm to be used when creating a digital signature
B8h	CT (confidentiality template)	Protect data objects by specifying SM with MAC and Encryption is necessary for the command, the response or both. Defines the RSA private key and algorithm to be used when decrypting data

Table 33 - CRT Values

It is possible to have more than one CRT with the same tag, for example there could be two authentication templates as in “Example 1” on page 205.

CRTs can contain six different kinds of CRDO each having its own tag. If more than one CRDO with the same tag exists in a CRT, then at least one of these objects must be fulfilled (OR condition). If you want for example to give PIN protection to a file, where either PINA or PINB can be presented, then PINA and PINB must be in different authentication templates. Such a case is shown in “Example 1” on page 205.

The following table shows the CRDOs, their tags and the templates in which they can be used:

Tag	CRDO	AT (A4h)	KAT (A6h)	CCT (B4h)	DST (B6h)	CT (B8h)
80h	Algorithm ID	—	—	—	X	—
83h	PIN reference (user authentication)	X	—	—	—	—
	Key reference for a symmetric key mutual authentication	X	—	—	—	—
	Diffie–Hellman key exchange parameters	—	X	—	—	—
	PK for Verify certificate	—	—	—	X	—
84h	Key reference for an RSA private key	X	—	—	X	—

Table 34 - Control Reference Data Objects

Tag	CRDO	AT (A4h)	KAT (A6h)	CCT (B4h)	DST (B6h)	CT (B8h)
91h	Random data object	—	X	—	—	—
95h	CRT Usage Qualifier	X	—	X	—	X
5F4C	Certificate holder authorization (CHA) containing the Role ID (asymmetric key mutual authentication)	X	—	—	—	—

Table 34 - Control Reference Data Objects

Where:

X indicates that the CRDO can be used in the corresponding template. All the CRDOs are optional.

In order to specify the type of use for a CRT, it can contain a special type of CRDO called a CRT usage qualifier (tag 95h). The possible values for this are shown in the following table:

Meaning	b8	b7	b6	b5	b4	b3	b2	b1
RFU		0				0	0	0
Mutual Authentication (AT)	1							
Secure Messaging in response (CCT, CT)			1					
Secure Messaging in command (CCT, CT)				1				
User Authentication (PIN) (AT)					1			

Table 35 - CRT Usage Qualifier Values

Thus for AT, the values are 80h for mutual authentication or 08h for user authentication. The values for CCT and CT are 20h (SM in response) or 10h (SM in command) or 30h (SM in command and response). Here SM refers to MAC for CCT and encryption for CT.

Authentication Templates (ATs) – Tag A4h

As “Table 34” shows, ATs have several uses. Each AT contains two CRDOs (one for the reference value itself (PIN, key or CHA) and a CRT usage qualifier to specify the type of reference).

The coding for an AT is therefore:

For a PIN:

Tag	Length	Value			Description
A4h	06h				Tag and length of AT
		Tag	**Length**	**Value**	Value of AT
		83h	01h	PIN Ref	TLV of PIN ref
		95h	01h	08h	CRT Usage qualifier (value 08h indicates user authentication)

Table 36 - Authentication Template Coding for a PIN

ATs contain only one PIN. A security attribute specifying two PINs requires two ATs, one for each PIN. For an example of how to code an SE containing an AT, see “Example 1” on page 205.

For a key reference used in symmetric key mutual authentication:

Tag	Length	Value			Description
A4h	06h				Tag and length of AT
		Tag	**Length**	**Value**	Value of AT
		83h	01h	Key Ref	TLV of Key ref
		95h	01h	80h	CRT Usage qualifier (value 80h indicates mutual authentication)

Table 37 - Authentication Template Coding for a Key Reference

For a more detailed example of how to code an SE with an AT containing a key reference, refer to “Example 2” on page 206.

For a CHA:

Tag	Length	Value			Description
A4h	0Dh				Tag and length of AT
		Tag	**Length**	**Value**	Value of AT
		5F4Ch	07h	CHA	TLV of CHA (role ID)
		95h	01h	80h	CRT Usage qualifier (value 80 indicates mutual authentication)

Table 38 - Authentication Template Coding for a CHA

Note: The CRT usage qualifiers in these examples are shown after the other CRDO. However the order of the two CRDOs is not important.

Public key mutual authentication procedures use card–verifiable certificates. CHAs are contained in such certificates. If a CHA is to be used as part of the security conditions to access data or functions, then the CHA must be present in the corresponding AT. The value of the CHA is AID || Role ID.

Where:

AID is the identifier of the application

Role ID is the role associated with the terminal.

The CHA in the certificate must match that specified by the SE. For each CHA CRDO, there must be a corresponding CRT Usage Qualifier to indicate mutual authentication (role ID), see Table 35, “CRT Usage Qualifier Values”, on page 50.

For a more detailed example of how to code an SE with an AT containing a CHA, refer to “Example 3” on page 207.

Key Agreement Templates (KATs) – Tag A6h

This template is used with asymmetric key mutual authentication. It is to send the card the ID of the Diffie–Hellman key exchange parameters it used to compute the K_{IFD} value and also to send the K_{IFD} value itself, thus enabling the card to calculate the value of the session key itself.

The coding of a KAT template is as follows:

Tag	Length	Value			Description
A6h	86h				Tag and length of KAT
		Tag	**Length**	**Value**	Value of AT
		83h	01h	Key ID	TLV of Key ID for DH key exchange parameters
		91h	81h80h	K_{IFD}	TLV of the K_{IFD} (considered as a random data object)

Table 39 - Key Agreement Template Coding

For a more detailed example of how to code an SE with a KAT, refer to “Example 4” on page 207.

Cryptographic Checksum Template (CCTs) – Tag B4h

This template is used to specify the secure messaging (MAC) for the commands used to access a data object. The template specifies whether secure messaging (MAC) is needed for the command or the response or both. The coding is as follows:

Tag	Length	Value			Description
B4h	06h				Tag and length of CCT
		Tag	**Length**	**Value**	Value of CCT
		83h	01h	00h	TLV of session key ref (it always has the value of 00h because there is only one possible session key generated during mutual authentication)
		95h	01h	Var	TLV of CRT Usage qualifier to indicate level of secure messaging (see Table 35, “CRT Usage Qualifier Values”, on page 50 for possible values)

Table 40 - Cryptographic Checksum Template Coding

Note: When using the **MSE–Set** command, the tag and length of the CCT are not included in the coding of the data field. For an example refer to “Coding Example” on page 159.

For a more detailed example of how to code an SE with a CCT, refer to “Example 6” on page 209.

Digital Signature Templates (DSTs) – Tag B6h

DSTs are used for one of the following two purposes:

- Specify the algorithm and RSA private key to use in a digital signature computation
- Specify the CA public key prior to a certificate verification.

In the first case, the coding for the DST is as follows:

Tag	Length	Value			Description
B6h	06h				Tag and length of DST
		Tag	**Length**	**Value**	Value of DST
		84h	01h	Key Ref	TLV of RSA private key ref
		80h	01h	Algo ID	TLV of Algo ID (for values see Table 43 on page 53)

Table 41 - Digital Signature Template Coding–For Signatures

Note: The algorithm ID and key reference can be coded in any order.

For an example of how to code an SE with a DST for signatures, see “Example 7” on page 209.

In the second case, the coding for the DST is as follows:

Tag	Length	Value			Description
B6h	03h				Tag and length of DST
		Tag	**Length**	**Value**	Value of DST
		83h	01h	Key ID	TLV of PK_CA_AUT

Table 42 - Digital Signature Template Coding–For Certificate Verification

For an example of how to code an SE with a DST for signatures see “Example 8” on page 210.

The values for the algorithm ID are shown in the following table:

Value	Meaning
01h	No hash; RSA padding according to ISO 9796–2
02h	No hash; RSA padding according to PKCS #1 v2.x
03h	No hash; RSA padding according to RFC 2409
11h	SHA–1 hash; RSA padding according to ISO 9796–2
12h	SHA–1 hash; RSA padding according to PKCS #1 v2.x
Other	RFU

Table 43 - Coding for an Algorithm ID

The most significant nibble indicates whether or not the DSI is hashed using the SHA–1 algorithm or not at all. The least significant nibble indicates the format of the RSA padding.

Note:

1. When the hash is performed outside the card, the most significant nibble is not interpreted by GemSafe v2.
2. When using the **MSE–Set** command, the tag and length of the DST are not included in the coding of the data field. For an example refer to "Coding Example" on page 159.

Confidentiality Template (CTs) – Tag B8h

These are used for one of the following two purposes:

- Specify the secure messaging (MAC and encryption) for the commands used to access a data object.
- Specify the algorithm and keys to use in a decipher operation.

In the first case, the template specifies whether secure messaging (MAC and encryption) is needed for the command or the response or both. The coding is as follows:

Tag	Length	Value			Description
B8h	03h				Tag and length of CT
		Tag	**Length**	**Value**	Value of CT
		95h	01h	Var	TLV of CRT Usage qualifier to indicate level of secure messaging (see Table 35, "CRT Usage Qualifier Values", on page 50 for possible values)

Table 44 - Confidentiality Template Coding

For a more detailed example of an SE with a CT used to define secure messaging, see "Example 10" on page 211.

In the second case, the coding is as follows:

Tag	Length	Value			Description
B8h	09h				Tag and length of CT
		Tag	**Length**	**Value**	Value of CT
		84h	01h	Key ID	TLV of RSA private key ref (always a utility key)
		80h	01h	1Ah	TLV of Algo ID for **PSO–Decipher** command (this value is always 1Ah indicating RSA with Padding Indicator not 00h)

Table 45 - Confidentiality Template Coding

For a more detailed example of an SE with a CT used for the **PSO–Decipher** command, see "Example 9" on page 210

Note: When using the **MSE–Set** command, the tag and length of the CT are not included in the coding of the data field. For an example of the coding to use for the **MSE–Set** command refer to "Coding Example" on page 159.

7

Applet Life Cycle States

This chapter describes the possible life cycle states of the GemSafe v2 applet from the time that it is installed and loaded onto the card. The life cycle states before this stage are considered as outside the scope of this document. For more information about these “off card” states, refer to the documentation for the card, such as the *GemXpresso Pro R3.x Reference Manual*.

Once the applet has been successfully installed on the card, the three applet life cycle states are:

- SELECTABLE: the initial state immediately after installation. For information about how to install the GemSafe v2 applet, see “Chapter 4 - Installing the GemSafe v2 Applet”. As the name implies, the applet is ready to be selected for execution.
- PERSONALIZED: This is the state of the applet after a successful execution of the **End Personalization** command. In this state, the applet contains all the necessary personalization data and keys to perform all its functions.
- BLOCKED: The applet can arrive at this state following an integrity problem or an authentication mechanism problem, such as if a ratification counter reaches the value 0 after too many incorrect authentication attempts.

All the applet life cycle states are fully described in section 5.3.1 “Application Life Cycle States” in the *GlobalPlatform Card Specification, version 2.1.1*.

8

Mutual Authentication

The GemSafe v2 applet makes the smart card a signature card. It generates digital signatures, enabling systems supporting GemSafe v2 to authenticate beyond doubt that the cardholder is who he or she claims to be.

Before exchanging data, the card and the terminal must authenticate each other. There are two different mutual authentication mechanisms in GemSafe v2:

- Asymmetric key mutual authentication (using RSA keys with the Diffie–Hellman mechanism for privacy)
- Symmetric key mutual authentication.

Each instance of GemSafe v2 uses one or the other and the choice is made during installation of the applet by setting a bit in the authentication key option byte. For more details refer to “Authentication Key Option Byte” on page 24.

This chapter describes the two mechanisms in detail.

Asymmetric Key Mutual Authentication (with Privacy)

The security of a GemSafe v2 system is based on three main entities:

- The card containing the GemSafe v2 applet
- The system that interacts with the card by means of a terminal
- The certification authority (CA) which guarantees the authenticity of cards and terminals by issuing them with certificates

Note: Although this chapter talks about the card, all the communication is with the GemSafe v2 applet that resides on the card.

Once the applet is in the life cycle state PERSONALIZED, the applet and terminal perform a mutual authentication before exchanging data from one to the other. This mutual authentication is based on two main principles:

- The card and terminal verify the existence of a certified key pair in the other device. The card must authenticate the terminal before the terminal authenticates the card.
- The two devices establish a common session key for use in subsequent operations. This process is described in “Appendix C - Generating Session Keys with the Diffie–Hellman Algorithm”.

The Asymmetric Key Mutual Authentication Mechanism

For asymmetric key mutual authentication to succeed between the card and the terminal, the following data must be present.

The card must contain:

- SK.ICC.AUT — the private key used for mutual authentication
- C.ICC.AUT — the certificate issued for the card (which contains PK.ICC.AUT the card's public key used for asymmetric key mutual authentication)
- p, q and g — the Diffie–Hellman key exchange parameters
- PK.CA.AUT — the CA's public key
- SN.ICC — the chip serial number

The terminal must contain:

- SK.IFD.AUT — the private key used for authentication
- C_CV.IFD.AUT — the certificate issued for the terminal by the CA (includes the CHA which in turn includes the role ID). This is used in asymmetric key mutual authentication.
- PK.CA.AUT — the CA's public key
- SN.IFD — the terminal serial number

Where:

ICC is the integrated circuit card

IFD is the interface device

The format of the certificate C.ICC.AUT is not specified, it can for example be in X.509 format or in the same format as C_CV.IFD.AUT (known as card verifiable format as specified in *CWA 14890 2004*). The format chosen must be able to be interpreted by the terminal and must contain at least the following data:

- SN.ICC — the chip serial number
- PK.ICC.AUT — the card's public key used for authentication

The certificate C_CV.IFD.AUT includes the terminal's public key and a hash. The exact contents of this certificate are given in "Appendix B - Format of Authentication Certificates".

The main steps of mutual authentication are shown in the following figure:

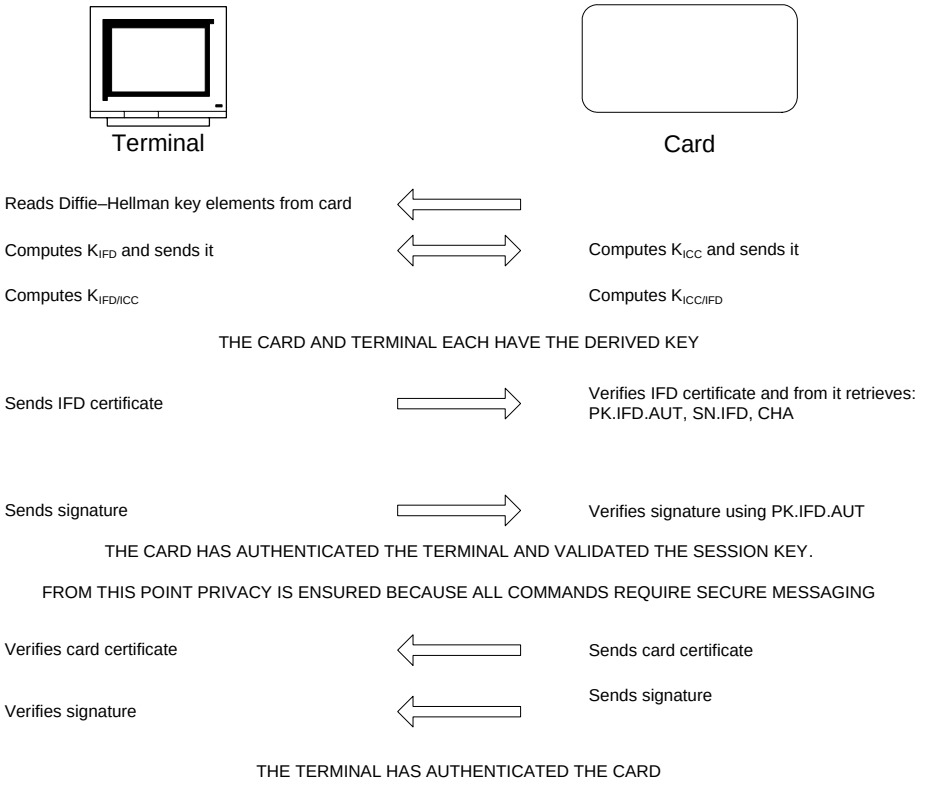


Figure 10 - Main Steps of Asymmetric Key Mutual Authentication

The appropriate command APDUs to use, and the data exchanged between the terminal and the card are shown in “Typical Command Sequence for Asymmetric Mutual Authentication” on page 59. For details about the APDUs, see each individual command description in “Chapter 11 - Application Phase Commands”.

For a detailed description of the key exchange, see “Appendix C - Generating Session Keys with the Diffie–Hellman Algorithm”.

Typical Command Sequence for Asymmetric Mutual Authentication

In the following command sequences, the names of the command APDUs are shown in bold.

Step	IFD (terminal)		ICC (Card)
1.	Insert card in terminal	<===	ATR
2.	**Select**s the GemSafe v2 applet instance using its AID	===> <===	 OK
3.	Reads PKCS#15 files if they exist (**Read Binary**).	<===	OK
4.	Reads the Diffie–Hellman key exchange parameters (**Get Data**).	<===	DH elements p, q, g

Step 3 is recommended for PKCS#15 compliant applications but not mandatory. It is up to the application to decide whether or not to perform this step.

Step	IFD (terminal)		ICC (Card)
5.	Computes $K_{IFD} = g^x \bmod p$		
6.	The **MSE: Set** command sends the Key ID for the DH key exchange parameters and K_{IFD} to the card (Key Agreement template). KID || K_{IFD}	===>	
7.	The **Get Data** command retrieves K_{ICC} Computes $K_{IFD/ICC} = K_{ICC}^x \bmod p$	===> <===	Computes $K_{ICC} = g^y \bmod p$ Computes $K_{ICC/IFD} = K_{IFD}^y \bmod p$ Returns K_{ICC}
8.	The **MSE: Set** command sets the security environment (SE) that will use the **PSO: Verify Certificate** command. PK.CA.AUT must be referenced in a DST (B6h) template.	===> <===	Selects PK.CA.AUT OK
9.	Sends the terminal's certificate C_CV.IFD.AUT used for asymmetric key mutual authentication (**PSO: Verify Certificate**).	===> <===	Verifies C_CV.IFD.AUT Returns OK and retrieves PK.IFD.AUT, SN.IFD, and CHA (including role ID) from C_CV.IFD.AUT

Where:

K_{ICC}	is the data element generated by the card using the Diffie–Hellman key exchange parameters. For more details about this algorithm, see "Appendix C - Generating Session Keys with the Diffie–Hellman Algorithm".
K_{IFD}	is the data element generated by the terminal using the Diffie–Hellman key exchange parameters.
$K_{ICC/IFD}$	is the shared secret data derived from K_{ICC} and K_{IFD}.
KID	is the ID for the Diffie–Hellman key exchange parameters data object.
SN.IFD	is the terminal serial number
PK.CA.AUT	is the CA's public key
PK.IFD.AUT	is the terminal public key
C_CV.IFD.AUT	is the certificate issued for the terminal by the CA
CHA	is the certificate holder authorization including the role ID)

In step 5, x is a random number chosen by the terminal such that 1 <= x <= q-1.

In step 6 the terminal sends the key ID of the DH key exchange parameters data object it used to compute K_{IFD}.

In step 7, y is a random number chosen by the card such that 1 <= y <= q-1. Using K_{IFD} the card computes the shared secret data $K_{ICC/IFD}$. The terminal retrieves K_{ICC} from which it computes the shared secret data $K_{IFD/ICC}$. Since $K_{ICC/IFD} = K_{IFD/ICC}$ (see "Appendix C - Generating Session Keys with the Diffie–Hellman Algorithm" for proof), both the card and the terminal know the shared secret data and from it, derive the two

session keys $K_{SK}ENC$ (used for encryption) and $K_{SK}MAC$ (used for MAC computations). For details on how to derive these keys see “Deriving the Session Keys” on page 202.

In step 8, the terminal initializes (or sets) the security environment that will be used by the succeeding **PSO: Verify Certificate** command.

During step 9, the terminal sends its certificate used for asymmetric key mutual authentication to the card for verification. This includes the value of the terminal public key, PK.IFD.AUT, the CHA including the role ID, and the terminal serial number, SN.IFD. However, although the card knows that the certificate originated from the CA, it does not yet know if the terminal possesses the correct private key.

Step	IFD (terminal)		ICC (Card)
10.	The **MSE: Set** command sets the security environment (SE) that will use the **External Authenticate** command. Referenced key must be AT (A4h) template.	===> <===	Selects PK.IFD.AUT OK
11.	Issues a **Get Challenge** command.	===> <===	 Returns CRnd (8 bytes)
12.	Issues an **External Authenticate (Application)** command. This sends SN.IFD || Sig to the card. Sig = Sig (6Ah || PRND2 || C || BCh)$_{SK.IFD.AUT}$	===> <===	Verifies the signature using PK.IFD.AUT. OK

Where:

PK.IFD.AUT	is the terminal public key
C	is the result of the hash computation on SHA–1 [PRND2 || K_{IFD} || SN.IFD || CRnd || K_{ICC} || DH.P]
SN.IFD	is the terminal serial number
CRnd	is the random number generated by the card. The four rightmost bytes form part of the initial value of the send sequence counter (SSC)
PRND2	are the padding bytes generated by the terminal such that the term beginning with 6Ah and ending in BCh is the same length as the modulus in the **External Authenticate** command.
SK.IFD.AUT	is the terminal’s private key used for authentication

In step 10, the terminal initializes (or sets) the security environment that will be used by the succeeding **External Authenticate (Application)** command. The **MSE: Set** command references the key taken from the terminal’s certificate, C_CV.IFD.AUT, used in asymmetric key mutual authentication.

In step 11, the card returns a random number CRnd. The terminal needs this value for two reasons:

- To initialize the value SSC
- To verify the value of the hash, C, returned by the card during step 11.

In step 12, the terminal sends a signature made by the terminal using its private key. The card can verify this because it received the terminal’s public key, PK.IFD.AUT in step 9.

AT THIS POINT, THE CARD HAS AUTHENTICATED THE TERMINAL.

From now until the end of the mutual authentication process, all commands must be sent with MAC and ENC secure messaging. This ensures **privacy** for the remaining exchanges between the card and the terminal.

Step	IFD (terminal)		ICC (Card)
13.	Reads the C.ICC.AUT EF (**Read Binary**).	<===	C.ICC.AUT

The terminal reads the card's certificate, from which it extracts the card's public key and the SN.ICC. At this stage, the terminal recognizes that the certificate was issued by the CA, but this is not enough to prove that the card possesses the correct private key

Note: The **Read Binary** command must be sent in "MAC and encryption" mode.

Step	IFD (terminal)		ICC (Card)
14.	The **MSE: Set** command sets the security environment (SE) that will use the **Internal Authenticate** command. Referenced key must be AT (A4h) template.	===> <===	Selects SK.ICC.AUT OK
15.	Issues an **Internal Authenticate** command.	<===	Sends SN.ICC || Sig $(S)_{SK.ICC.AUT}$ || S = 6Ah || PRND1 || C || BCh C= SHA–1 [PRND1 || K_{ICC} || SN.ICC || TRnd || K_{IFD} || DH.P]

In step 14, the terminal initializes (or sets) the security environment that will be used by the succeeding **Internal Authenticate** command. The **MSE: Set** command references the card's private key, SK.ICC.AUT, used in asymmetric key mutual authentication.

Note: The **MSE: Set** and **Internal Authenticate** commands must be sent with secure messaging for confidentiality.

In step 15, the terminal asks the card to authenticate itself. To do this, the card generates a signature and returns it with the chip serial number. The terminal verifies the signature.

AT THIS POINT, THE TERMINAL HAS AUTHENTICATED THE CARD.

As a final step, the terminal can read the contents of the Display Message EF from the card and displays them. This file is optional and if it exists, it contains an "authentication successful" message, which is used to provide visual confirmation. As the message can be decrypted only by the correct session key, this proves that the session key was generated correctly.

Step	IFD (terminal)		ICC (Card)
16.	Reads the Display Message (**Read Binary**).	<===	Display message

Symmetric Key Mutual Authentication

The security of a GemSafe v2 system is based on two main entities:

- The card containing the GemSafe v2 applet
- The system that interacts with the card by means of a terminal

Note: Although this chapter talks about the card, all the communication is with the GemSafe v2 applet that resides on the card.

In the application phase, that is once the applet is in the life cycle state PERSONALIZED, it may be necessary to authenticate the terminal before allowing access to certain protected data objects in the card. For this purpose, GemSafe v2 has a particular command, **Mutual Authenticate**, that simultaneously authenticates the terminal to the card as well as the card to the terminal. This mutual authentication is based on the card and terminal sharing two secret 3DES keys, K_{ENC} and K_{MAC}, and each proving that the other possesses the same keys.

The Mutual Authentication Mechanism

The main steps of mutual authentication are shown in the following figure:

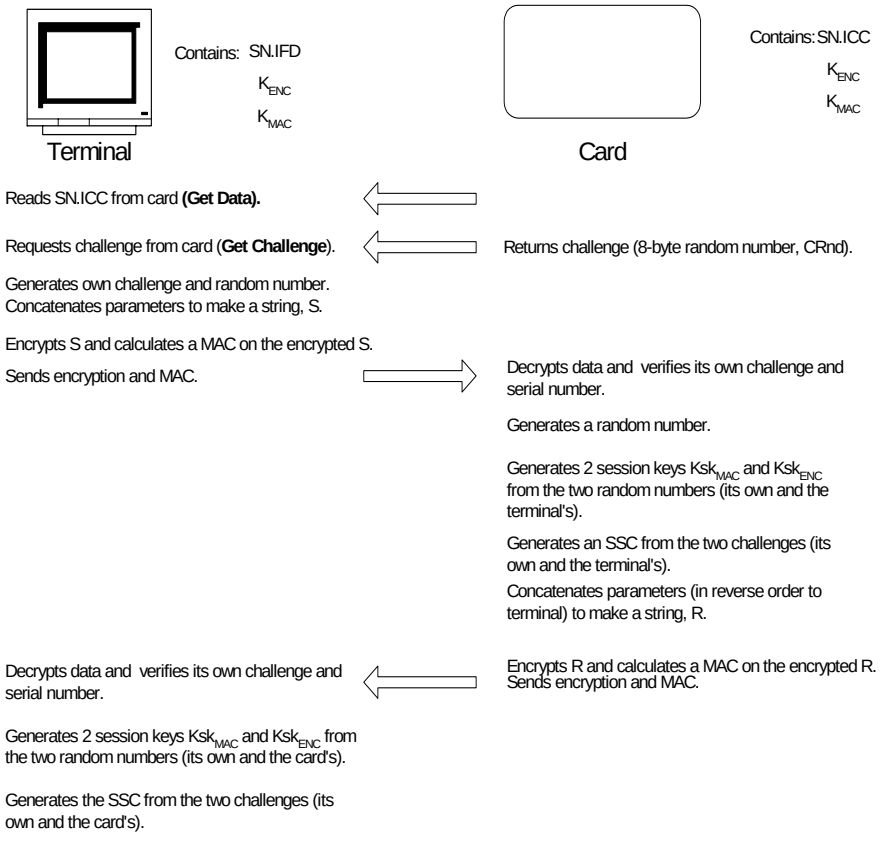


Figure 11 - Main Steps of Mutual Authentication

The command APDUs to use, and the data exchanged between the terminal and the card are shown in “Command Sequence for Mutual Authentication” on page 64. For details about the APDUs, see each individual command description in “Chapter 11 - Application Phase Commands”.

Note: For added security, the keys K_{ENC} and K_{MAC} stored in the card can be values that have already been diversified from a mother key (that must be stored in the terminals).

Command Sequence for Mutual Authentication

In the following command sequences, the names of the command APDUs are shown in bold.

Step		IFD (terminal)		ICC (Card)
1.		Insert card in terminal.	<===	ATR
2.		**Select**s the GemSafe v2 applet using its AID.	===> <===	 OK
3.		Reads the chip serial number (**Get Data**).	<===	SN.ICC
4		Issues **MSE Set** command for K_{ENC} and K_{MAC}	===>	
5.		Issues a **Get Challenge** command.	===> <===	 Returns CRnd (8 bytes)
6.		Issues a **Mutual Authenticate** command. This command performs the following actions:		
	6a.	Generates a 32–byte random number, K_{IFD}, and an 8–byte challenge TRnd.		
	6b.	Concatenates a string, S = (TRnd || SN.IFD || CRnd || SN.ICC || K_{IFD}). *Where* SN.IFD is the terminal serial number.		
	6c	Computes S′ = $3DES_{CBC}$ [S]<K_{ENC}> and MAC = RetailMAC [S′]<K_{MAC}>.		
	6d	Sends S′ || MAC.	===>	
	6e			Verifies MAC using its own K_{MAC} Decrypts S′ with K_{ENC} and verifies CRnd and SN.ICC
	6f			Generates a 32–byte random number, K_{ICC}
	6g			Computes session keys K_{SK}ENC and K_{SK}MAC from TRnd and CRnd. (See “Computation of Session Keys K_{SK}ENC & K_{SK}MAC” on page 65).
	6h			Concatenates a string SS = (RND.ICC || SN.ICC || RND.IFD || SN.IFD || K_{ICC}).

Step		IFD (terminal)		ICC (Card)
	6i			Computes SS′ = $3DES_{CBC}$ [SS]<K_{ENC}> and MAC = RetailMAC [SS′]<K_{MAC}>.
	6j			Generates SSC using the two challenges CRnd and TRnd.
	6k		<===	Returns SS′ || MAC.
	6l	Verifies the response (TRnd and SN.IFD), computes the session keys K_{SK}ENC and K_{SK}MAC and generates the SSC.		

Note:

1. Retail MAC is the name given to the MAC computation algorithm used by GemSafe v2. The computation is described in “MAC Computation” on page 46, but for this computation the SSC is null.
2. If K_{ENC} and K_{MAC} are stored as diversified values in the card, the terminal must recompute K_{ENC} and K_{MAC}. These diversified values are used in step 6c.

Computation of Session Keys K_{SK}ENC & K_{SK}MAC

The two session keys K_{SK}ENC and K_{SK}MAC that are used in secure messaging are computed from the derived key $K_{IFD/ICC}$. The computation is the same for both asymmetric and symmetric key mutual authentication, although the key $K_{IFD/ICC}$ is derived differently according to the type of mutual authentication as follows:

Asymmetric key mutual authentication

When computed by the terminal, $K_{IFD/ICC} = K_{ICC}^{x} \bmod p$.

When computed by the card, $K_{IFD/ICC} = K_{IFD}^{y} \bmod p$.

Where:

x is the terminal random number, TRnd

y is the card random number, CRnd

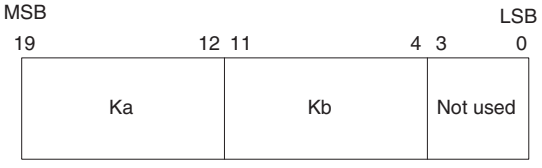
Symmetric key mutual authentication

$K_{IFD/ICC} = K_{IFD}$ XOR K_{ICC}

K_{SK}ENC

1. Compute a hash as follows:

   Hash = SHA–1 (($K_{IFD/ICC}$) || 00h 00h 00h 01h)



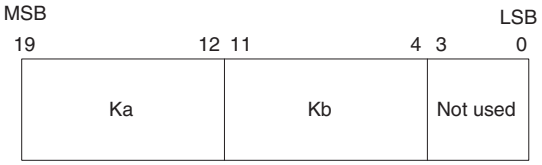
2. From the hash, extract Ka and Kb, so

   K_{SK}ENC = Ka || Kb

$K_{SK}MAC$

1. Compute a hash as follows:

   Hash = SHA–1 (($K_{IFD/ICC}$) || 00h 00h 00h 02h)



2. From the hash, extract Ka and Kb, so

   $K_{SK}ENC$ = Ka || Kb

The Send Sequence Counter (SSC)

In the case of asymmetric key mutual authentication, the terminal initializes the SSC as 1 following the **External Authenticate** command. At the end of the authentication, that is following the **Internal Authenticate** command, it is set to 4 LSB RND.ICC || 4 LSB RND.IFD.

In the case of symmetric key mutual authentication, the terminal initializes the SSC as 0. At the end of the authentication, that is following the **Mutual Authenticate** command, it is set to 4 LSB RND.ICC || 4 LSB RND.IFD.

The counter increments by one before each MAC computation.

9

APDU Command Formats

The GemSafe v2 applet accepts commands and responses in compliance with the application protocol data unit (APDU) format defined by the ISO 7816-4 standard.

However, you should bear in mind that the transmission layer protocol depends on the Java card—not the GemSafe v2 applet. The card might use T=0 or T=1 or both, which consequently affects the way in which the command and response APDUs are transmitted.

This reference manual describes the APDUs for each command.

T=0 Protocol

If the card is using the T=0 protocol, this converts APDUs into transmission protocol data units (TPDUs). The reader sends TPDU commands to the card and the card returns TPDU responses to the reader, as described in Annex A of ISO 7816-4. Refer to this standard if you need more information about converting APDUs into TPDUs.

T=1 Protocol

If the card is using the T=1 protocol, the command and response are mapped directly onto the information field of one I-block or the concatenation of the information fields of successive I–blocks that are chained, as described in Annex B of ISO 7816-4. Refer to this standard if you need more information about the T=1 protocol.

Command Format

Command APDUs are structured as follows:

Header				**Body**		
CLA	INS	P1	P2	Lc	Parameters/Data	Le

The fields are described in the following sections.

Header Fields

The header fields, which are mandatory, are as follows:

Field Name	Length	Description	
CLA	1 byte	Instruction class, FFh is not allowed. This can be any of the following values:	
		x0	No secure messaging
		x4h	Secure messaging in personalization phase.
		xCh	Secure messaging in application phase
		Bit 5 = 1 indicates chaining mode. The command will be repeated until all the data has been sent. The final command in the chain is sent with bit 5 = 0	
INS	1 byte	Instruction code. This is given with the command descriptions.	
P1	1 byte	Parameter 1	
P2	1 byte	Parameter 2	

Body Fields

The command body is present or absent depending on the type of command. It includes the following fields:

Field Name	Length	Description
Lc	1 byte	Data length
Data	n bytes	Command parameters or data
Le	1 byte	Expected length of data to be returned

Response Format

The GemSafe v2 applet returns responses to commands in the following format:

Body	Trailer
Data	SW1, SW2

The **Body** field is present or absent depending on the type of command and holds the data returned by the card.

The **Trailer** field includes the following two mandatory bytes:

- SW1: Status byte 1 that returns the command processing status
- SW2: Status byte 2 that returns the command processing qualification

See “Appendix A - Status Bytes” for a list of the values that the GemSafe v2 applet can return in the status bytes.

Command–Response Pairs

As the command APDU and the response APDU may or may not contain data, there are four cases of command–response pairs as described in ISO 7816–4. These pairs can be summarized as follows:

Case 1: No Input/No Output

APDU Command

CLA	INS	P1	P2	---	-----	---

APDU Response

-----	SW1	SW2

Case 2s: No Input/Output of Expected Length

APDU Command

CLA	INS	P1	P2	---	-----	Le

APDU Response:

Data Field	SW1	SW2

Case 3s: Input/No Output

APDU Command

CLA	INS	P1	P2	Lc	Data Field	---

APDU Response

-----	SW1	SW2

Case 4s: Input/Output of Expected Length

APDU Command

CLA	INS	P1	P2	Lc	Data Field	Le

APDU Response

Data Field	SW1	SW2

For Input/Output of expected length, when a command with response parameters/data is used, the terminal must send a **Get Response** command to retrieve the response message.

T=0 Protocol

When using the T=0 protocol, the GemSafe v2 returns status bytes of SW1 = 61h, SW2 = Licc (the total number of bytes in the response) and the terminal sends the **Get Response** command automatically.

Retrieval Sequence for Large Outgoing Data

If Licc >= 256 bytes, GemSafe v2 returns 61h 00h and opens a retrieval sequence as follows:

1. The initial command (status 61h 00h).
2. One or more **Get Response** commands.

The following table illustrates the possible responses to a **Get Response** command:

Le: Licc	Data Returned	Status Bytes Returned	Sequence Open/Closed
Le > Licc	None	6Ch Licc	Open
Le = Licc	“Licc” bytes	90h 00h	Closed
Le < Licc	“Le” bytes	61h Licc–Le	Open

Table 46 - Get Response when Licc > 256 Bytes

T=1 Protocol

When using the T=1 protocol, GemSafe v2 returns one of the following to the initial case 4 command:

Licc	Data Returned	Status Bytes Returned	Sequence Open/Closed
Licc < = 256	256 bytes	90h 00h	Closed
Licc > 256	256 bytes	61h XXh Licc–256	Open

Table 47 - Response to Case 4 Command

Where: XXh is Licc – 256.

Retrieval Sequence for Large Outgoing Data

If Licc >= 256 bytes, then one or more **Get Response** commands must be sent:

The following table illustrates the possible responses to a **Get Response** command:

XXh	Data Returned	Status Bytes Returned	Sequence Open/Closed
XXh < = 256	XX bytes	90h 00h	Closed
XXh > 256	256 bytes	61h XXh–256	Open

Table 48 - Get Response when Licc > 256 Bytes

When using the T=1 protocol, the terminal does not send the **Get Response** automatically. You need to issue the command to retrieve the response.

10

Personalization Phase Commands

This chapter describes the GemSafe v2 applet commands that are available in the personalization phase. In order to illustrate each command clearly, each description shows the case where the command is sent without secure messaging. For details on the command format to use with secure messaging, refer to “Secure Messaging During the Personalization Phase” on page 40.

An example of the commands needed to personalize the GemSafe v2 applet is given in “Chapter 5 - Personalizing the GemSafe v2 Applet”.

The following table lists the commands that are available in the personalization phase:

Command	Description	Page
Activate File*	Changes the life cycle status of a file to OPERATIONAL (ACTIVATED).	73
Create File*	Creates an EF under the root or the currently selected DF or creates a DF under the root.	76
Deactivate File*	Changes the state of a file to OPERATIONAL (DEACTIVATED).	80
End Personalization	Changes the applet life cycle state to PERSONALIZED.	82
Erase Binary*	Erases part of a binary file.	84
External Authenticate (Perso)	Authenticates the external terminal to the card. Sets the secure channel mode.	87
Generate Public Key Pair**	Generates an RSA key pair and stores both keys in the card. It returns the modulus as part of its response.	144
Get Data*	Retrieves one of the following: • CPLC data • Applet version • Public key elements • K_{ICC} • The contents of a specified SE	89
Get Response	Retrieves the response from a previous command.	96
Initialize Update	First step to open a secure channel. The personalization device authenticates the card.	97

Table 49 - Personalization Commands

Command	Description	Page
Manage Security Environment–SET**	Sets or replaces one component of the current SE.	158
PSO–Compute Digital Signature**	Computes a digital signature.	162
PSO–Decipher**	Deciphers an encrypted message using a utility key stored in the card.	165
PSO–Hash**	Entirely or partially hashes data prior to a **PSO–Compute Digital Signature** command or prepares the data if hashed externally	168
Put Data*	Creates or updates a data object such as a security environment, PIN, secret key, public key, private key or DH key exchange parameters	100
Select File*	Selects a DF or an EF by its file ID, path or name (in the case of DFs).	121
Update Binary*	Updates part of a binary file.	125

Table 49 - Personalization Commands (continued)

* This command is also available during the application phase.

** This command is also available during the application phase and as its operation is identical, it is described once only in “Chapter 11 - Application Phase Commands”.

For each APDU command, GemSafe v2 verifies the following conditions, and returns the corresponding status codes if the check fails:

1. Check that the class byte is allowed. => **6E 00h**
2. Check the instruction code.
   a) Check that instruction code is present and valid. => **6D 00h**
   b) Check instruction code is compatible with class byte. => **68 00h**
   c) Check instruction is allowed for the current card life phase. => **6D 00h**
   d) Check instruction is allowed in current DF or application. => **69 85h**

A comprehensive description of each command is given in the following sections.

ACTIVATE FILE

The **Activate File** command changes the state of an EF to OPERATIONAL (ACTIVATED). The state of a file is stored in the life cycle status byte in the EF's file descriptor. For more details about the file life cycle status refer to Table 4 on page 8.

The file can be referenced by its file identifier or by its path. For more details about these methods, see "Referencing by File Identifier" on page 8 and "Referencing by Path" on page 9. The command can be used to activate the current EF by specifying P1 and P2 as 00h. In this case, the data field is empty.

Conditions of Use and Security

Usage

The command can be used in either the personalization phase or the application phase. For details about its use in the application phase, see "Activate File" on page 129.

The command can be used on EFs only. It is not valid for DFs or for data objects.

The EF can be in any state prior to issuing the **Activate File** command. If the file is already in the state OPERATIONAL (ACTIVATED), the **Activate File** command has no effect, but does not fail.

If P1 and P2 are both 00h, the command applies to the currently selected EF. In this case Lc is zero and the data field is empty.

Security

Any EF in the life cycle status CREATED or INITIALIZED can be freely activated.

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of secure messaging for the **Activate File** command is set by the **External Authenticate (Perso)** command. For details about how to send the command with secure messaging, see "Secure Messaging During the Personalization Phase" on page 40.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	44h	P1	P2	Lc	Data

Where:

CLA 00h transmit normally

04h transmit with secure messaging

P1 indicates the file referencing method and is coded as follows:

P1	b8	b7	b6	b5	b4	b3	b2	b1
RFU	0	0	0	0				
Reference EF by path					1	0	0	0
Reference EF by File ID					0	0	X	X
Currently selected EF							0	0
Search for EF under current DF							1	0

P2 00h indicates file is the currently selected EF. This value is valid only if P1 = 00h

0Ch indicates no response data

Lc indicates the length of the data field as follows:

00h indicates no data (when P1 and P2 are 00h)

02h if selecting EF by file ID

xxh where “xxh” is the length of the path name

Data is either the 2–byte file ID or the path name. The path is the concatenation of the file IDs from the root down to the file ID of the EF to be activated. The file ID of the root is not included in the path.

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
64h	00h	Execution error (file descriptor checksum error)
67h	00h	Incorrect length, Lc
69h	82h	Security conditions not satisfied (security attribute not fulfilled or error during secure messaging)
6Ah	80h	Incorrect data, such as referenced file not found
6Ah	81h	Function not supported (P1 and P2 = 00 but no current EF, command not valid for DFs)
6Ah	86h	Incorrect P1, P2. This includes the case when P1 and P2 are 00h but a data field exists.

CREATE FILE

The **Create File** command creates an elementary file (EF) under the current DF or the root or a dedicated file (DF) under the root.

You can create as many DFs and EFs as the GemSafe v2 applet's memory space permits.

The file data is sent in TLV format in the FCP template (tag 62h).

Note: As the **Erase Binary**, **Read Binary** and **Update Binary** commands specify an offset on 15 bits, Gemplus recommends that you do not create files of 32KB or larger.

Conditions of Use and Security

Usage

The command can be used in either the personalization phase or the application phase. For details about its use in the application phase, see "Create File" on page 134.

A secure channel must have been established using the **External Authenticate (Perso)** command.

DFs cannot be created under other DFs.

If the data field contains more than one object with the same tag, the command creates the first one only.

The body size is limited to 7FE0h bytes.

Security

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of security is determined by the secure channel, set by the **External Authenticate (Perso)** command. For details on the command format to use with secure messaging, refer to "Secure Messaging During the Personalization Phase" on page 40.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	E0h	00h	00h	Lc	FCP Template

Where:

CLA Specifies transmission mode (normal or secure messaging):

- 00h transmit normally
- 04h transmit with secure messaging

Lc Specifies the data field length (maximum 20h = 32 bytes)

Data The data is stored in the FCP template (tag 62h). The elements and their tags are as shown in the following table:

Tag Level 1	Tag Level 2	Description	EF	DF	Mandatory?
62h		FCP Template	X	X	M
	81h	File Size (in bytes)	X		M
	82h	FDB	X	X	M
	83h	File ID	X	X	M
	8Ah	Life Cycle Status	X		
	8Ch	Security Attributes (Access mode and security condition bytes)	X	X	M
	84h	DF Name		X	

Table 50 - Data Elements for Create File Command

Where:

FDB = 38h for DFs
01h for EFs

File ID can take any value within the restrictions imposed by ISO 7816–4. The short file identifier (SFI) corresponds to the 5 least significant bits of the file identifier. Certain commands can use the SFI to reference an EF (but not a DF).

Life Cycle Status is coded on one byte. For information about its possible values, see Table 4 on page 8.

Security attributes are made up of an access mode byte, followed by up to four security condition bytes, one for each bit set to 1 in the access mode byte. If no bits are set to 1, there are no security condition bytes. For information on how to code security attributes, see “Security Attributes” on page 33.

DF name is optional and is 1–16 bytes long.

Example of DF Creation

In this example, the APDU to send for the **Create File** command is:

CLA	INS	P1	P2	Lc	Data
80h	E0h	00h	00h	19h	FCP Template

where the FCP template is coded as follows:

Tag	Length	Value			Description
62h	17h				Tag and length of FCP template
		Tag	**Length**	**Value**	
		82h	01h	38h	TLV of FDB
		83h	02h	4Fh 00h	TLV of File ID
		8Ch	03h	06h 00h 00h	TLV of security conditions (free)
		84h	09h	44h 46h 20h 69h 73h 73h 75h 65h 72h	TLV of DF name

Table 51 - Example of DF Creation

Example of EF Creation

In this example, the APDU to send for the **Create File** command is:

CLA	INS	P1	P2	Lc	Data
80h	E0h	00h	00h	15h	FCP Template

where the FCP template is coded as follows:

Tag	Length	Value			Description
62h	13h				TLV of FCP template
		Tag	**Length**	**Value**	
		81h	02h	00h 20h	TLV of file size (in bytes). 20h = 32 bytes
		82h	01h	01h	TLV of FDB (01=elementary file)
		83h	02h	4Fh 01h	TLV of file ID (ID = 4Fh 01h)
		8Ah	01h	05	TLV of life cycle status (05=OPERATIONAL (ACTIVATED)
		8Ch	03h	03h 13h 00h	TLV of security attribute's access mode byte is 03h, (access defined for the **Update Binary, Erase Binary,** and **Read Binary** commands). 13h indicates PIN protection defined by SE#3 for the **Update Binary** and **Erase Binary** commands. 00h indicates free access for the **Read Binary** command.

Table 52 - Example of EF Creation

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
67h	00h	Incorrect length, Lc. Maximum value is 20h (32 bytes)
69h	82h	Security status not satisfied
69h	85h	Conditions of use not satisfied, for example incorrect tag or length
6Ah	84h	Insufficient memory
6Ah	86h	Incorrect P1, P2

DEACTIVATE FILE

The **Deactivate File** command changes the state of an EF to OPERATIONAL (DEACTIVATED). The state of a file is stored in the life cycle status byte in the EF's file descriptor. For more details about the file life cycle status refer to Table 4 on page 8.

This change is reversible; a file can reactivated by issuing the **Activate File** command. Once a file is deactivated, the only command that is allowed for that file is the **Activate File** command.

The file can be referenced by its file identifier or by its path. For more details about these methods, see "Referencing by File Identifier" on page 8 and "Referencing by Path" on page 9. The command can be used to activate the current EF by specifying P1 and P2 as 00h. In this case, the data field is empty.

Conditions of Use and Security

Usage

The command can be used in either the personalization phase or the application phase. For details about its use in the application phase, see "Deactivate File" on page 135.

The command can be used on EFs only and only in the state OPERATIONAL (ACTIVATED). It is not valid for DFs or for data objects.

If P1 and P2 are both 00h, the command applies to the currently selected EF. In this case Lc is zero and the data field is empty.

Security

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of secure messaging for the **Deactivate File** command is set by the **External Authenticate (Perso)** command. For details about how to send the command with secure messaging, see "Secure Messaging During the Personalization Phase" on page 40.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	04h	P1	P2	Lc	Data

Where:

CLA 00h transmit normally

04h transmit with secure messaging

P1 indicates the file referencing method and is coded as follows:

P1	b8	b7	b6	b5	b4	b3	b2	b1
RFU	0	0	0	0				
Reference EF by path					1	0	0	0
Reference EF by File ID					0	0	X	X
Currently selected EF							0	0
Search for EF under current DF							1	0

P2 00h indicates file is the currently selected EF. This value is valid only if P1 = 00h.

0Ch indicates no response data

Lc indicates the length of the data field as follows:

00h indicates no data (when P1 and P2 are 00h)

02h if selecting EF by file ID

xxh where “xxh” is the length of the path name

Data is either the 2–byte file ID or the path name. The path is the concatenation of the file IDs from the root down to the file ID of the EF to be activated. The file ID of the root is not included in the path.

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
64h	00h	Execution error (file descriptor checksum error)
67h	00h	Incorrect length, Lc
69h	82h	Security conditions not satisfied (security attribute not fulfilled or error during secure messaging)
6Ah	80h	Incorrect data, such as referenced file not found
6Ah	81h	Function not supported (P1 and P2 = 00 but no current EF, command not valid for DFs)
6Ah	86h	Incorrect P1, P2

END PERSONALIZATION

As the name implies, this command is used at the end of the personalization process. It sets the applet life cycle state to PERSONALIZED.

Caution: Do not use the Card Manager command **Set Status** to change the applet's life cycle state to PERSONALIZED. Although the state changes, any further GemSafe v2 commands fail with the status bytes 69h 85h. GemSafe v2 is designed this way to force you to use the **End Personalization** command at the end of the personalization phase. This is for your protection because your application will not work correctly if one or more mandatory files are missing, and once GemSafe v2 is in the application phase it cannot return to the personalization phase.

Conditions of Use and Security

Usage

The command can be used in the personalization phase only, that is the GemSafe v2 applet must be in the life cycle state SELECTABLE.

Security

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of security is determined by the secure channel, set by the **External Authenticate (Perso)** command. For details on the command format to use with secure messaging, refer to "Secure Messaging During the Personalization Phase" on page 40.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	E4h	00h	00h	0Ah	Personalization device data

Where:

CLA Specifies transmission mode (normal or secure messaging):

- 80h transmit normally
- 84h transmit with secure messaging

Data The personalization device data is stored in the following format:

- 09h indicating the length of the data
- DTHR the date and time supplied by the personalization device (5 bytes). The format of this data is not imposed by GemSafe v2.
- TermID identifier of the personalization device (4 bytes). The format of this data is not imposed by GemSafe v2.

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
69h	85h	Conditions of use not satisfied
6Ah	82h	File or data object not found
6Ah	86h	Incorrect P1, P2

ERASE BINARY

This command erases all or part of an EF.

There are two ways of referencing the EF:

- Implicit selection. In this case, the EF is under the current DF, and is specified by its short file identifier (SFI). P1 specifies the SFI and P2 specifies the start address for the erase (called the offset). The command cannot be used for files with an SFI of 1Fh.
- Direct selection. In this case the EF must have been previously selected using the **Select File** command. The data is erased directly from the current EF. In this case the offset is specified over P1 and P2.

The command erases data from the offset until the end of the file is reached or until the end address in the data field, if one exists.

Note: The end address is in fact the address of the first byte NOT to be erased.

Conditions of Use and Security

Usage

The **Erase Binary** command is available during the personalization phase and the application phase. For details on its use in the application phase, see “Erase Binary” on page 138.

The command can be used on any activated EF. It is not valid for deactivated EFs, for DFs or for data objects.

In the case of implicit selection, the EF becomes the current EF after execution of the command.

Security

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of secure messaging for the **Erase Binary** command is set by the **External Authenticate (Perso)** command. For details about how to send the command with secure messaging, see “Secure Messaging During the Personalization Phase” on page 40.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	0Eh	P1	P2	Lc	Data

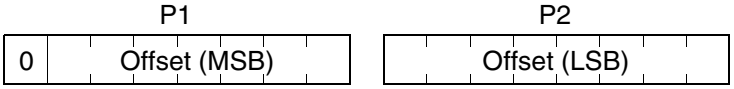
Where:

CLA 00h transmit normally

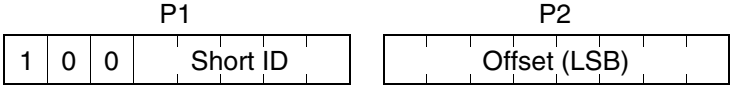
04h transmit with secure messaging

P1, P2 are the parameter bytes.

To erase from the currently selected EF, P1 and P2 specify the offset, in bytes, from the beginning of the file to the first byte to be erased. P1 and P2 have the following format:



To select and erase from an EF, P1 specifies the SFI of the file containing the data to be erased. P2 specifies the offset, in bytes, from the beginning of the file to the first byte to be erased. P1 and P2 have the following format:



Lc indicates the length of the data field as follows:

empty if no data field

02h if end address specified in data field

Data indicates the end point for the erase as follows:

empty indicates the command erases data until the end of the file

XXh XXh if present indicates the offset in bytes of the first byte NOT to be erased

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
62h	82h	End of file reached (address specified in data field is higher than the end of the file)
64h	00h	Execution error (file descriptor checksum error)
67h	00h	• Incorrect length, Lc • Offset given in data field is not higher than offset of start address in P1
69h	82h	Security status not satisfied, for example: • The security attributes are not satisfied • Error during secure messaging
69h	86h	Command not allowed (no current EF)
6Ah	82h	File not found (b8 of P1 = 1 indicating implicit selection, but no SFI given)
6Ah	86h	Incorrect P1, P2 (check that the offset is inside the EF and that the SFI is not 1Fh)
6Bh	00h	Incorrect P1, P2 (offset outside EF)

EXTERNAL AUTHENTICATE (PERSO)

This is a Card Manager command, not a GemSafe v2 command. It is reproduced here for information.

The **External Authenticate (Perso)** command authenticates the external terminal to the GemSafe v2 applet during personalization. It sets the secure channel mode. The following modes are available:

- No security
- MAC
- MAC and encryption

Conditions of Use and Security

Usage

The command can be used in the personalization phase only, that is the GemSafe v2 applet must be in the life cycle state SELECTABLE.

An **Initialize Update** command must have been successfully performed before the **External Authenticate (Perso)** command.

Security

The **External Authenticate (Perso)** command must be performed with MAC secure messaging. For details about how to send the command in this mode, see “Commands Sent in MAC Mode” on page 41.

Note: For clarity, the command is described here with the data in plain text.

If the **External Authenticate (Perso)** command fails, the secure channel is closed. A successful **Initialize Update** command must be performed before a further **External Authenticate (Perso)** command can be executed.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
84h	82h	Security Control	00h	10h	personalization device cryptogram + MAC

Where:

P1 is the security control parameter. The bit settings are as follows:

P1	b8	b7	b6	b5	b4	b3	b2	b1
RFU	0	0	0	0	0	0		
MAC and encryption							1	1
MAC							0	1
No security							0	0

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
67h	00h	Incorrect length
69h	82h	Security conditions not satisfied
69h	85h	Command conditions not satisfied
6Ah	86h	Incorrect P1, P2

GET DATA

This command retrieves one of the five following sets of data:

- The card production life cycle (CPLC) data, without selecting the Card Manager
- The applet version
- Public key elements (a BER-TLV object)
- The value of K_{ICC}, a data element generated by the card during mutual authentication (a BER-TLV object)
- The contents of a specified SE (a BER-TLV object)

It cannot be used to retrieve any other data. The data in the response is in plain text for CPLC and the applet version and in TLV format for public key elements and SEs.

Conditions of Use and Security

Usage

The **Get Data** command is available during the personalization phase and the application phase. In the application phase, it can be used to retrieve the applet personalization data set during the **End Personalization** command. for more details, see “Get Data” on page 147.

Security

During the personalization phase, the command cannot use secure messaging. The command can be used in either of these cases:

- Before a Card Manager/terminal mutual authentication is performed
- After a Card Manager/terminal mutual authentication has been performed to open a secure channel, but only if the secure channel is level 0 (set by the **External Authenticate (Perso)** command).

Format

The command format is as follows:

CLA	INS	P1	P2	Lc		Le
00h	INS	P1	P2	Lc	Data	Le

The values depend on the type of object being retrieved. The following table shows the possible values:

Data object	INS	P1	P2	Lc	Data	Le
CPLC Data	CAh	9Fh	7Fh	Empty	Empty	2Ah
Applet Version	CAh	DFh	30h	Empty	Empty	08h
BER–TLV	CBh	00h	FFh	Lc	TLV structure	Le

Table 53 - Get Data Values (Perso Phase)

Note: The format of this command has been designed for backward compatibility.

Examples of Coding the Data Field for BER–TLV Objects

To retrieve a public key element (the modulus in this example):

INS = CBh

P1/P2 = 00h FFh

Lc = 0Ah

Data is as shown in the following table

Tag	Length	Value			Description	No. of Bytes
B6h	03h				Tag and length of DST Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	02h				Tag and length of public key template	3
		Tag	**Length**			
		81h	00h		Tag and length of modulus (value coded on 128 bytes)	2
					TOTAL BYTES	**10**

Table 54 - Get Data Coding Structure – Public Key Element

Note: The length of 00h means that all the element is recovered.

To retrieve an entire public key, that is, the modulus and exponent elements:

INS = CBh

P1/P2 = 00h FFh

Lc = 0Ah

Data is as shown in the following table

Tag	Length	Value			Description	No. of Bytes
B6h	03h				Tag and length of DST Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	02h				Tag and length of public key template	3
		Tag	**Length**			
		80h	00h		Tag and length of entire public key	2
					TOTAL BYTES	**10**

Table 55 - Get Data Coding Structure – Entire Public Key

Note: The length of 00h means that all the element is recovered.

To retrieve a K_{ICC}

INS = CBh

P1/P2 = 00h FFh

Lc = 04h

Data is as shown in the following table

Tag	Length				Description	No. of Bytes
A6h	02h				Tag and length of KAT Template	2
		Tag	**Length**			
		91h	00h		Tag and length of K_{ICC} (a random data object)	2
					TOTAL BYTES	**4**

Table 56 - Get Data Coding Structure – K_{ICC}

Note: The length of 00h means that all the element is recovered.

To retrieve a Diffie–Hellman element ("p" in this example)

INS = CBh

P1/P2 = 00h FFh

Lc = 0Ah

Data is as shown in the following table

Tag	Length	Value			Description	No. of Bytes
A6h	03h				Tag and length of KAT Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	02h				Tag and length of public key template	3
		Tag	**Length**			
		86h	00h		Tag and length of DH p element	2
					TOTAL BYTES	**10**

Table 57 - Get Data Coding Structure –Diffie–Hellman Element

Note: The length of 00h means that all the element is recovered.

To retrieve all the contents of a specified SE:

INS = CBh

P1/P2 = 00h FFh

Lc = 05h

Data =

T	L	T	L	V
7Bh	03h	80h	01h	SEID

Where:

7Bh is the tag for an SE

80h indicates that the entire SE is recovered.

Response

Response	SW1	SW2

Response is coded as follows:

CPLC Data

Data Element	Length (in bytes)
IC Fabricator	2
IC Type	2
Operating System Identifier	2
Operating System Release Date	2
Operating System Release Level	2
* IC Fabrication Date	2
* IC Serial Number	4
* IC Batch Identifier	2
IC Module Fabricator	2
IC Module Packaging Date	2
ICC Manufacturer	2
IC Embedding Date	2
IC Pre–Personalizer	2
IC Pre–Personalization Date	2
IC Pre–Personalization Equipment Identifier	4
IC Personalizer	2
IC Personalization Date	2
IC Personalization Equipment Identifier	4

Table 58 - CPLC Data

* The chip serial number, SN.ICC is IC Fabrication Date || IC Serial Number || IC Batch Identifier.

Applet Version

The command returns the applet version in TLV format as follows:

Tag	Length	Value
DFh 30h	05h	Applet version

The applet version is returned as five ASCII characters to display “vX.XX”. For example an applet version of v1.01 is returned as follows:

ASCII Value	76h	31h	2Eh	30h	31h
Display value	v	1	.	0	1

Public Key Element (the Modulus in this Example)

The response is as shown in the following table:

Tag	Length	Value			Description	No. of Bytes
B6h	03h				Tag and length of DST Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	L				Tag and length of public key template	3
		Tag	**Length**	**Value**		
		81h	L_{Mod}	Val_{Mod}	TLV of modulus (value coded on 128 bytes)	130
					TOTAL BYTES	**138**

Table 59 - Get Data Response Structure – Public Key Element

Caution: There is one case where the format of the response differs according to the version of the GemSafe V2 applet. This case is when retrieving the modulus for a 2048 key using secure messaging. For versions of the applet before 2.03, the first seven bytes of the response must be ignored.

Entire Public Key

The response is as shown in the following table:

Tag	Length	Value			Description	No. of Bytes
B6h	03h				Tag and length of DST Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	L				Tag and length of public key template	3
		Tag	**Length**	**Value**		
		81h	L_{Mod}	Val_{Mod}	TLV of modulus (value coded on 128 bytes)	130
		82h	L_{EXP}	Val_{EXP}	TLV of exponent (value coded on 8 bytes)	**8**
					TOTAL BYTES	**146**

Table 60 - Get Data Response Structure – Entire Public Key

K_{ICC}

The response is as shown in the following table:

Tag	Length	Value			Description	No. of Bytes
A6h	L				Tag and length of KAT Template	2
		Tag	**Length**	**Value**		
		91h	81h 80h	K_{ICC}	Tag and length of K_{ICC}	131
					TOTAL BYTES	**133**

Table 61 - Get Data Response Structure – K_{ICC}

The Entire Contents of an SE (AT Containing a PIN in this Example)

The response is the entire SE encapsulated in a template with tag 7B, as shown in the following table:

Tag	Length	Value					
7B	L_{SE}						Tag and Length of current SE
		Tag	**Length**	**Value**			**Description**
		A4h	06h				Tag and length of AT
				Tag	**Length**	**Value**	Value of AT
				83h	01h	PIN Ref	TLV of PIN ref
				95h	01h	08h	CRT Usage qualifier (value 08h indicates user authentication)

Table 62 - Get Data Response Structure for an SE

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
6Ah	86h	Incorrect P1, P2

GET RESPONSE

The **Get Response** command is an administrative and application TPDU mechanism, that is used to recover the response for case 4 APDU commands. It is sent automatically by the APDU commands and is only used in these circumstances, that is it is not sent on its own.

Conditions of Use and Security

Usage

The **Get Response** command must be used only after command case 4. It can be used in the personalization phase and the application phase.

Security

The **Get Response** command can be accessed freely.

Format

CLA	INS	P1	P2	Le
00h	C0h	00h	00h	Le

Where:

Le Expected length of data.

Response

Response	SW1	SW2

Where:

Response Holds the card response.

SW1 and SW2 Hold the status byte values returned by the card, as follows:

SW1	SW2	Description
61h	XXh	Le < Licc, XXh is the remaining number of available data bytes in the card
62h	81h	Part of returned data may be corrupted
6Ah	86h	Incorrect P1 P2
6Ch	Licc	if Le > Licc
6Fh	00h	No available data (Licc = 0)
90h	00h	Successful execution of the command (Le = Licc)

INITIALIZE UPDATE

This is a Card Manager command, not a GemSafe v2 command. It is reproduced here for information.

Use the **Initialize Update** command to open a secure channel for communication between the GemSafe v2 applet in the card and the client terminal. The secure channel protocol provides a mechanism for the secure (authenticated, authorized and confidential) communication exchange between the applet (on-card) and the client terminal (off-card). For more information about secure channels, refer to “Secure Channel Modes” on page 40.

Conditions of Use and Security

Usage

The command can be used in the personalization phase only, that is the GemSafe v2 applet must be in the life cycle state SELECTABLE.

Security

The **Initialize Update** command is performed with no security.

Format

The format of this command is as follows:

CLA	INS	P1	P2	Lc	Data	Le
80h	50h	Key Set Version	Key	08h	Personalization device session data	1Ch

Where:

P1 is the version number in the range 00h–FFh of the Secure Channel Key Set in the Card Manager used for secure channel initiation. 00h indicates that the Key Set Version is implicitly known by the card. The implicit key set is the first found which, as the keys are managed as last in first out (LIFO), is the last created key set.

If the current key set is FFh, the P1 parameter must take the value 00h.

P2 is the specific key in the Secure Channel Key Set, as indicated by the Key Index.

The three key indexes in a key set are as follows:

Index = 01	Key [1]	AUTH/ENC
	Key [2]	MAC
	Key [3]	KEK
Index = 02	Key [1]	KEK
	Key [2]	AUTH/ENC
	Key [3]	MAC
Index = 03	Key [1]	MAC
	Key [2]	KEK
	Key [3]	AUTH/ENC

Where:

AUTH/ENC is the authentication key, used for all data encryption and decryption, including the decryption of encrypted command data sent in a secure channel.

MAC is the MAC key used for all authentication cryptogram computations and verifications including the verification of the MACs used in a secure channel.

KEK is the key encryption key used to decrypt the **Put Key** command data. The **Put Key** command is used to create and update the key sets in the card.

Data is eight bytes of data that are generated by the personalization device and are unique to the current secure channel session.

Response

Data	SW1	SW2

The data returned by the card is as follows:

Data	Length (in bytes)
Key diversification data as follows: – Two rightmost bytes of Card Manager AID (2 bytes) – IC Fabrication Data * (2 bytes) – IC Serial Number * (4 bytes) – IC Batch Identifier * (2 bytes)	10
Key set version (01h — 7Fh or FFh)	1
Key Index (01h)	1
Card Random	8
Card Cryptogram	8

* These fields are taken from the CPLC data.

For further details see *GlobalPlatform Card Specification 2.1.1*. This data is valid for the session.

The data is followed by the SW1, SW2 status codes. Possible values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
67h	00h	Incorrect length, Lc
69h	85h	Command conditions not satisfied
6Ah	86h	Incorrect P1, P2

PUT DATA

This command creates the following data objects in the personalization phase:

- Security environments
- PINs
- DES secret keys
- RSA private keys
- RSA public keys
- Diffie–Hellman key exchange parameters

The conditions of use and data are different for each type of data object. For clarity, each case is described individually.

Note: The format of this command has been designed for backward compatibility.

PUT DATA–SECURITY ENVIRONMENTS

This command is used to create SEs. It can be used in chaining mode, that is, the data can be split over two **Put Data** commands.

When chaining data and using secure messaging, the MAC is computed on the whole of the command header and data field as if it were one command APDU.

Conditions of Use and Security

Usage

The command can be used in the personalization phase only, that is the GemSafe v2 applet must be in the life cycle state SELECTABLE. SEs cannot be modified after their creation.

Security

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of secure messaging for the **Put Data** command is set by the **External Authenticate (Perso)** command. For details on the command format to use with secure messaging, refer to “Secure Messaging During the Personalization Phase” on page 40.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	DBh	00h	FFh	Lc	Data

Where:

CLA 00h transmit final or only portion of data normally and execute the command

04h transmit final or only portion of data with SM and execute the command

10h transmit first portion of data in chaining mode normally and wait for final portion

14h transmit first portion of data in chaining mode with SM and wait for final portion

Lc indicates the length of the data field

Data as described in “Appendix E - Security Environment Coding Examples”.

Note: The data for the SE must be encapsulated in within an SE template (tag 7Bh)

Response

The responses for the six **Put Data** commands are grouped together. See “Response” on page 117.

PUT DATA–PINS

This command is used to create PINs. It can be used in chaining mode, that is, the data can be split over two **Put Data** commands.

When chaining data and using secure messaging, the MAC is computed on the whole of the command header and data field as if it were one command APDU.

Conditions of Use and Security

Usage

The command can be used in the personalization phase only, that is the GemSafe v2 applet must be in the life cycle state SELECTABLE. To update a PIN when the state is PERSONALIZED, that is, in the application phase, use the command **Change Reference Data** or **Reset Retry Counter**.

Security

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of secure messaging for the **Put Data** command is set by the **External Authenticate (Perso)** command. For details on the command format to use with secure messaging, refer to “Secure Messaging During the Personalization Phase” on page 40.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	DBh	00h	FFh	var	Data

Where:

CLA 00h transmit final or only portion of data normally and execute the command

04h transmit final or only portion of data with SM and execute the command

10h transmit first portion of data in chaining mode normally and wait for final portion

14h transmit first portion of data in chaining mode with SM and wait for final portion

Lc indicates the length of the data field

Data as described in “Data for PINs” on page 103

The PIN structure is as follows:

Tag	Length	Proprietary PIN template		
FFh 20h	var	**Tag**	**Length**	**Value**
		83h	01h	PIN Reference
		8Ch	var	Security attributes
		DFh 21h	04h	PIN attributes
		DFh 22h	L_{PIN}	PIN value

Table 63 - Structure of a PIN Data Object

Data for PINs

The data field is coded in TLV format as described in “PIN Contents” on page 38. The **Put Data** command can create one PIN only at a time.

Example

Suppose you want to create two PINs, PIN 1 and PIN 4 where PIN 4 has to be presented to change and reset PIN 1.

PIN#1 has the following values:

PIN reference	= 81h (PIN#1)
Try Limit	= 03h
Unblocking Counter	= A5h (no limit)
Unblocking PIN reference	= 86h (PIN#6 must be presented to unblock PIN#1)
Usage Counter	= FFh (no limit)
PIN#1 value	= 30h 31h 32h 33h 34h 35h 36h 37h
Security Attributes:	**Reset Retry Counter** = SM defined in SE#2
	Change Reference Data = PIN#4 and SM defined in SE#2
	Verify = FREE

PIN#4 has the following values:

PIN reference	= 84h (PIN#4)
Try Limit	= 04h
Unblocking Counter	= 00h (cannot be reset)
Unblocking PIN reference	= FFh (not checked)
Usage Counter	= 05h (can be used 5 times only)
PIN value	= 39h 38h 37h 36h 35h 34h 33h 32h 31h 30h 30h 30h
Security Attributes:	**Reset Retry Counter** = NEVER
	Change Reference Data = NEVER
	Verify = FREE

The two PINs must be created by separate **Put Data** commands. The data field for each PIN is:

PIN#1:

Tag	Length	Proprietary PIN template		
FFh 20h	1Bh	**Tag**	**Length**	**Value**
		83h	01h	PIN Reference = 81h
		8Ch	04h	Security attributes = F0h 42h 62h 00h • F0h: AM Byte indicates there are SC bytes for **Reset Retry Counter, Change Reference Data**, & **Verify** • 42h: SC byte for **Reset Retry Counter** (SM defined in SE#2) • F2h: SC byte for **Change Reference Data** (SM & PIN defined in SE#2) • 00h: SC byte for **Verify** (FREE)
		DFh 21h	04h	PIN attributes = 03h FFh A5h 86h • 03h: Try Limit • FFh: Usage Counter (no limit) • A5h: Unblocking counter (no limit) • 86h: Unblocking PIN reference (PIN#6)
		DFh 22h	08h	PIN value = 30h 31h 32h 33h 34h 35h 36h 37h

For this example, the value of Lc is therefore 1Eh.

PIN#4:

Tag	Length	Proprietary PIN template		
FFh 20h	1Dh	**Tag**	**Length**	**Value**
		83h	01h	PIN Reference = 84h
		8Ch	02h	Security attributes = 90h 43h • 90h: AM Byte indicates there is an SC byte for **Verify** only • 43h: SC byte for **Verify** (SM defined in SE#3)
		DFh 21h	04h	PIN attributes = 04h 05h 00h FFh • 04h: Try Limit • 05h: Usage Counter (can be used 5 times) • 00h: Unblocking counter (cannot be reset) • FFh: Unblocking PIN reference (not checked)
		DFh 22h	0Ch	PIN value = 39h 38h 37h 36h 35h 34h 33h 32h 31h 30h 30h 30h

For this example, the value of Lc is therefore 20h.

Response

The responses for the six **Put Data** commands are grouped together. See “Response” on page 117.

PUT DATA–SECRET KEYS

The GemSafe v2 applet uses two 3DES secret keys, K_{ENC} and K_{MAC}. These are used in symmetric key mutual authentication (but not in asymmetric key mutual authentication), so must be known by the terminal.

The keys must be created as a pair and initialized during the personalization phase. They can be updated in both the personalization phase and the application phase, using the **Put Data** command.

This command is used to create these 3DES secret keys. It can be used in chaining mode, that is, the data can be split over two **Put Data** commands.

When chaining data and using secure messaging, the MAC is computed on the whole of the command header and data field as if it were one command APDU.

Conditions of Security

Security

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of secure messaging for the **Put Data** command is set by the **External Authenticate (Perso)** command. For details on the command format to use with secure messaging, refer to "Secure Messaging During the Personalization Phase" on page 40.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	DBh	00h	FFh	Lc	Data

Where:

CLA 00h transmit final or only portion of data normally and execute the command

04h transmit final or only portion of data with SM and execute the command

10h transmit first portion of data in chaining mode normally and wait for final portion

14h transmit first portion of data in chaining mode with SM and wait for final portion

Lc indicates the length of the data field

Data as described in "Data for Secret Key Creation" on page 107

Data for Secret Key Creation

The data field is coded in TLV format as follows:

Tag	Length	Value	Description	No. of Bytes
A4h	03h		Key Usage: Authentication Template	2
		83 01 xxh	*where*: xx = key id	3
8C	02h	82h[1] XXh[2]	TLV of key security attributes	4
DF23h	10h	K_{ENC}	Value of K_{ENC}	19
DF24h	10h	K_{MAC}	Value of K_{MAC}	19
			TOTAL BYTES	**47**

Table 64 - Data Field Structure for Put Data–Secret Key Creation

[1] 82h is the value of the access mode byte.

[2] XXh is the value of the security condition byte. However since the built–in security policy specifies a minimum of SM in MAC and ENC mode, the value of this byte has no effect. For more details, see “Built–in Security Policy” on page 37.

In this example, the data is 47 bytes, so the value of Lc in the **Put Data** command is 2Fh.

Data for Secret Key Update

The data field is coded in TLV format as follows:

Tag	Length	Value	Description	No. of Bytes
A4h	03h		Key Usage: Authentication Template	2
		83 01 xxh	*where*: xx = key id	3
DF23h	10h	K_{ENC}	Value of K_{ENC}	19
DF24h	10h	K_{MAC}	Value of K_{MAC}	19
			TOTAL BYTES	**43**

Table 65 - Data Field Structure for Put Data–Secret Key Update

In this example, the data is 43 bytes, so the value of Lc in the **Put Data** command is 2Bh.

Response

The responses for the six types of **Put Data** commands are grouped together. See “Response” on page 117.

PUT DATA–PRIVATE KEYS

A private key contains five CRT elements, p, q, iq, dp and dq. For details of these elements, see “Private Keys” on page 10. A private key is created during the personalization phase by six **Put Data** commands—one to create the key pair container, used to store the private and public key values— and five to initialize the data for each element. The key and each element are identified by their individual tags.

In the application phase, the **Put Data** command can be used to update the value of one or more elements. For more information see “Put Data–Private Keys” on page 177.

This command can be used in chaining mode, that is, the data can be split over two **Put Data** commands.

When chaining data and using secure messaging, the MAC is computed on the whole of the command header and data field as if it were one command APDU.

Conditions of Use and Security

Usage

No more than N private keys can be created in the GemSafe v2 applet, where N is a parameter specified at the time of applet installation by the **Install** command in Install mode, see Table 18, “Option Parameters”, on page 25.

Only one private key can be specified with authentication usage (SK.ICC.AUT).

In the personalization phase, the first **Put Data** command must be the one to create the key (see “Example Data Field for Key Pair Creation:” on page 110). The commands to initialize the elements (tag 7F48h followed by a tag from 92h to 96h depending on the element) can be in any order.

When creating an asymmetric key with private and public elements, the private elements must be created first.

Security

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of secure messaging for the **Put Data** command is set by the **External Authenticate (Perso)** command. For details on the command format to use with secure messaging, refer to “Secure Messaging During the Personalization Phase” on page 40.

Note: For clarity, the command is described here with the data in plain text.

Asymmetric keys are identified by their key ID. This means that two Asymmetric keys cannot share the same key ID even if one is for authentication and one is for digital signatures.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	DBh	00h	FFh	Lc	Data

Where:

CLA 00h transmit final or only portion of data normally and execute the command

04h transmit final or only portion of data with SM and execute the command

10h transmit first portion of data in chaining mode normally and wait for final portion

14h transmit first portion of data in chaining mode with SM and wait for final portion

Lc indicates the length of the data field

Data as described in “Data for Private Keys”

Data for Private Keys

The data field is coded in TLV format. Examples are given for each **Put Data** command.

Example Data Field for Key Pair Creation:

Tag	Length	Value	Description	Bytes
A4h or B6h	03h		AT (authentication key SK.ICC.AUT) DST (digital signature and/or utility key)	2
		84h 01h KeyID	TLV of private key	3
DFh 25h	02h	Key Maximum Length	TLV of Key Maximum Length 08h 00h indicates 2,048 bits 06h 00h indicates 1,536 bits 05h 00h indicates 1,280 bits 04h 80h indicates 1,152 bits 04h 00h indicates 1,024 bits 03h 00h indicates 768 bits 02h 00h indicates 512 bits	5
DFh 26h	01h	XXh	TLV of Private Key Usage. Bit 8 7 6 5 4 3 2 1 RFU RFU RFU RFU RFU RFU U DS b1 = 1 indicates digital signature (DS) key b2 = 1 indicates utility key A key can be both a utility key and a DS key	4
8Ch	03h[3]		Tag and length of security attribute (AM byte and SC bytes)	2
		8Ah	Access mode byte (8Ah means both SC bytes are present)	1
		00h	Security Condition byte for PSO. 00h means no condition specified.	1
		27h	Security Condition byte for **Put Data** and **Generate PK Pair** (27h means External Authentication required, SE#7)	1
			TOTAL BYTES	**19**

Table 66 - Example Data Field Structure for Put Data–Private Key Creation

Key Maximum Length: Authentication keys must be 1,024 bits. Digital signature keys can be any of the values indicated, however for CC compliancy they must be at least 1,024 bits.

Private Key Usage: For the A4 template (AT) this byte has no meaning but must be present. Utility keys are used for **PSO–Decipher** operations.

Note: For CC compliancy, an RSA key pair that is to be used for certified digital signatures must have the private key usage byte set to 01h, to prevent them being used for decipher operations.

Security Attributes: The length of the security attribute is 01h when there are no security condition bytes. The security attribute is mandatory and must contain at least the access mode byte. For CC compliancy, an RSA key pair that is to be used for certified digital signatures must include the following coding:

- SM must be specified for the **Get Data** command (bit 1 of the AM byte and bit 7 of its SC byte must be set to 1)
- SM + User Authentication must be specified for the **PSO–Compute Digital Signature** command (bit 4 of the AM byte and bits 8, 7 and 5 of its SC byte must be set to 1)

In this example, the data is 19 bytes, so the value of Lc in the **Put Data** command is 13h.

Example Data Field for Writing a Private Element (1,024–bit keys):

Tag	Length	Value			Description	No. of Bytes
A4h or B6h	03h				AT (authentication key SK.ICC.AUT) DST (digital signature)	2
		84h	01h	XXh	TLV of private key ID (XXh is key ID value)	3
7Fh 48h	XXh				Tag and length of private elements	3
		Tag	**Length**	**Value**		
		92h–96h			Tag of private key element (p, q, iq, dp, or dq) (for values see “Data for Private Keys” on page 109).	1
			40h		Length of element (all five elements are 64 bytes for 1,024–bit keys, 40h)	1
				Value	Value of element	64
					TOTAL BYTES	**74**

Table 67 - Example Data Field Structure for Put Data–Writing Private Key Elements

In this example, one element only is updated, making a total of 74 bytes, so the value of Lc in the **Put Data** command is 4Ah. Add 66 bytes for each additional element (one for the tag, one for the length, 64 for the value).

More than one element can be updated in the same **Put Data** command. If the total exceeds FFh bytes in length, the command must be used in chaining mode.

Response

The responses for the six **Put Data** commands are grouped together. See “Response” on page 117.

PUT DATA–PUBLIC KEYS

A public key data object contains two elements: a modulus, N, and a public exponent, e. It is created during the personalization phase in two stages:

- One **Put Data** command to create the key itself.
- One or two **Put Data** commands to initialize the data for each element (the elements can be initialized by separate **Put Data–Public Keys** commands or combined in one **Put Data–Public Keys** command).

If the public key is part of a key pair, use the **Put Data–Private Keys** command, which creates the private key and the public key at the same time. For details about this first command see “Put Data–Private Keys” on page 108 and in particular the section “Data for Private Keys” on page 109.

If the public key is not part of a key pair, use the **Put Data–Public Keys** command to create it.

The key and each element are identified by their individual tags.

In the application phase, the **Put Data** command can be used to update the value of one or more elements. For more information, see “Put Data–Public Keys” on page 179.

This command can be used in chaining mode, that is, the data can be split over two **Put Data** commands.

When chaining data and using secure messaging, the MAC is computed on the whole of the command header and data field as if it were one command APDU.

Conditions of Use and Security

Usage

No more than M public key data objects can be created in the GemSafe v2 applet, where M is a parameter specified at the time of applet installation by the **Install** command in Install mode, see Table 18, “Option Parameters”, on page 25.

In the personalization phase, the first **Put Data** command must be the one to create the key container.

The commands to initialize the modulus, N and the public exponent, e can be in any order or combined in a single command.

Security

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of secure messaging for the **Put Data** command is set by the **External Authenticate (Perso)** command. For details on the command format to use with secure messaging, refer to “Secure Messaging During the Personalization Phase” on page 40.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	DBh	00h	FFh	Lc	Data

Where:

CLA 00h transmit final or only portion of data normally and execute the command

04h transmit final or only portion of data with SM and execute the command

10h transmit first portion of data in chaining mode normally and wait for final portion

14h transmit first portion of data in chaining mode with SM and wait for final portion

Lc indicates the length of the data field as follows:

Data as described in “Data for DH Key Exchange Parameters”

Data for Public Keys

The data field is coded in TLV format. Examples are given for each **Put Data** command.

Example Data Field for Public Key Creation (When not part of a Key Pair):

Tag	Length	Value	Description	No. of Bytes
B6h	03h		Tag and length of DST Template	2
		83h 01h KeyID	TLV of key ID	3
DFh 25h	02h	Key Maximum Length	TLV of Key Maximum Length[1] 08h 00h indicates 2,048 bits 06h 00h indicates 1,536 bits 05h 00h indicates 1,280 bits 04h 80h indicates 1,152 bits 04h 00h indicates 1,024 bits 03h 00h indicates 768 bits 02h 00h indicates 512 bits	5
8Ch	02h[1]		Tag and length of security attribute (access mode byte and security condition bytes)	2
		82h	Access mode byte (Example value of 82h indicates SC byte for **Put Data** only is present)	1
		27h	Security Condition byte for **Put Data** (Value of 27h indicates External Authentication required, SE #7)	1
			TOTAL BYTES	**14**

Table 68 - Data Field Example for Put Data–Public Key Creation (not part of a Key Pair)

[1] This value is 01h when there is no security condition byte. The security attribute is mandatory and must contain at least the access mode byte.

In this example, the data is 14 bytes, so the value of Lc in the **Put Data** command is 0Eh.

Data Field Structure for Writing Exponent Element:

Tag	Length	Value			Description	No. of Bytes
A4h or B6h	03h				Tag and length of AT or DST Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	06h				Tag and length of public key template	3
		Tag	**Length**	**Value**		
		82h	04h	Prime	TLV of exponent (value coded on 4 bytes)	6
					TOTAL BYTES	**14**

Table 69 - Data Field Structure for Put Data–Writing Exponent Value

In this example, the data is 14 bytes, so the value of Lc in the **Put Data** command is 0Eh. The exponent can be any value but must be a prime number. The number cannot exceed 8 bytes in length.

Data Field Structure for Writing Modulus Element:

Tag	Length	Value			Description	No. of Bytes
A4h or B6h	03h				Tag and length of AT or DST Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	81h 83h				Tag and length of public key template	3
		Tag	**Length**	**Value**		
		81h	81h 80h	var	TLV of modulus (value coded on 128 bytes)	131
					TOTAL BYTES	**139**

Table 70 - Data Field Structure for Put Data–Writing Modulus Value

In this example for a 1,024–bit key, the data is 139 bytes, so the value of Lc in the **Put Data** command is 8Bh.

Note: If the two elements are sent in one **Put Data** command, the 81h and 82h tag are both contained in the public key template (tag 7Fh 49h)

Response

The responses for the six **Put Data** commands are grouped together. See “Response” on page 117.

PUT DATA–DH KEY EXCHANGE PARAMETERS

A Diffie–Hellman key exchange parameters data object is created during the personalization phase in two stages:

- One **Put Data–DH Key Exchange Parameters** command to create the object itself
- One or three **Put Data–DH Key Exchange Parameters** commands to initialize the data for each DH parameter p, q and g (the elements can be initialized by separate **Put Data–DH Key Exchange Parameters** commands or combined in one **Put Data–DH Key Exchange Parameters** command)

In the application phase, the **Put Data** command can be used to update the value of one or more elements. For more information, see “Put Data–DH Key Exchange Parameters” on page 181.

This command can be used in chaining mode, that is, the data can be split over two **Put Data** commands.

When chaining data and using secure messaging, the MAC is computed on the whole of the command header and data field as if it were one command APDU.

Conditions of Use and Security

Usage

GemSafe v2 can contain one Diffie–Hellman key exchange parameters data object only.

In the personalization phase, the first **Put Data** command must be the one to create the key. The exchange parameters can be initialized in any order.

Security

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of secure messaging for the **Put Data** command is set by the **External Authenticate (Perso)** command. For details on the command format to use with secure messaging, refer to “Secure Messaging During the Personalization Phase” on page 40.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	DBh	00h	FFh	Lc	Data

Where:

CLA 00h transmit final or only portion of data normally and execute the command

04h transmit final or only portion of data with SM and execute the command

10h transmit first portion of data in chaining mode normally and wait for final portion

14h transmit first portion of data in chaining mode with SM and wait for final portion

Lc indicates the length of the data field as follows:

Data as described in “Data for DH Key Exchange Parameters”

Data for DH Key Exchange Parameters

The data field is coded in TLV format. Examples are given for each **Put Data** command.

Example Data Field for DH Key Exchange Parameters Creation:

Tag	Length	Value			Description	No. of Bytes
A6h	03h				Tag and length of KAT Template	2
		83h	01h	KeyID	TLV of DH key exchange parameters	3
DFh 25h	02h	04h 00h			All DH key exchange parameter objects are 1,024 bits	5
8Ch	02h[1]				Tag and length of security attribute (access mode byte and security condition bytes)	2
		82h			Access mode byte (Example value of 82h indicates SC byte for **Put Data** only is present)	1
		27h			Security Condition byte for **Put Data** (Value of 27h indicates External Authentication required, SE #7)	1
					TOTAL BYTES	**14**

Table 71 - Data Field Example for Put Data–DH Key Exchange Parameters Creation

[1] This value is 01h when there is no security condition byte. The security attribute is mandatory and must contain at least the access mode byte.

In this example, the data is 14 bytes, so the value of Lc in the **Put Data** command is 0Eh.

Data Field Structure for Writing DH Key Exchange Parameters

Tag	Length	Value			Description	No. of Bytes
A6h	03h				Tag and length of KAT	2
		83h	01h	KeyID	TLV of Key ID for DH key exchange parameters data object	3
7Fh 49h	var				Tag and length of DH key exchange parameters template	3
		Tag	**Length**	**Value**		
		86h 87h 88h	81h 80h 20h 81h 80h	var	TLV of DH:p element TLV of DH:q element TLV of DH:g element	var
					TOTAL BYTES	**var**

Table 72 - Data Field Structure for Put Data–Writing DH Key Exchange Parameters Value

The three DH key exchange parameters can be combined in one command as shown or sent as three separate **Put Data** commands. If they are combined, and the total length of the parameters exceeds FFh, the command must be sent in chaining mode.

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
67h	00h	Incorrect length, Lc
69h	82h	Security status not satisfied, for example: • The security attributes are not satisfied • Error during secure messaging
69h	85h	Conditions of use not satisfied, for example: • Attempt to create a PIN with a PIN ID that already exists (**Put Data** command) • Secret or private key already exists (when trying to create the key) • Secret or private key does not exist (when trying to write the value of an element • PIN to be created is protected by a PIN that has not yet been created • Incorrect AM byte in data field (only the **Put Data** access mode bit is allowed for a public key)
6Ah	80h	Incorrect data, for example incorrect tag or tag length
6Ah	84h	Insufficient memory (allowed maximum number of private keys exceeded)
6Ah	86h	Incorrect P1, P2
6Ah	88h	Reference data not found (Tag specified in P1, P2)

READ BINARY

This command reads all or part of an EF.

There are two ways of referencing the EF:

- Implicit selection. In this case, the EF is under the current DF, and is specified by its short file identifier (SFI). P1 specifies the SFI and P2 specifies the start address for the read (called the offset). The command cannot be used for files with an SFI of 1Fh.
- Direct selection. In this case the EF must have been previously selected using the **Select File** command. The data is read directly from the current EF. In this case the offset is specified over P1 and P2.

The command reads the number of bytes specified in Le from the offset, up to a maximum of 255 bytes (including secure messaging data if used).

Conditions of Use and Security

Usage

The **Read Binary** command is available during the personalization phase and the application phase. For details on its use in the application phase, see “Read Binary” on page 183.

The command can be used on any activated EF. It is not valid for deactivated EFs, for DFs or for data objects.

In the case of implicit selection, the EF becomes the current EF after execution of the command.

Security

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of secure messaging for the **Read Binary** command is set by the **External Authenticate (Perso)** command. For details about how to send the command with secure messaging, see “Secure Messaging During the Personalization Phase” on page 40.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Le
CLA	B0h	P1	P2	Le

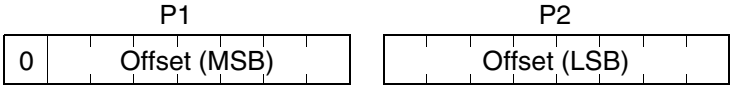
Where:

CLA 00h transmit without secure messaging

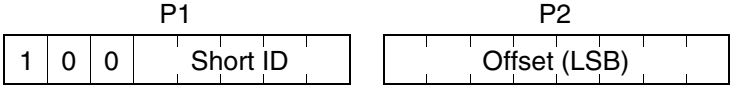
04h transmit with secure messaging

P1, P2 are the parameter bytes.

To read from the currently selected EF, P1 and P2 specify the offset, in bytes, from the beginning of the file to the first byte to read. P1 and P2 have the following format:



To select and read from an EF, P1 specifies the SFI of the file containing the data to be read. P2 specifies the offset, in bytes, from the beginning of the file to the first byte to read. P1 and P2 have the following format:



Le indicates the number of bytes to read. If empty or zero, the command reads until the end of the file, up to a maximum of 255 bytes (including data for secure messaging if SM specified).

Response

Data	SW1	SW2

Data is the data read from the card. It is followed by the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
62h	82h	End of file reached (offset + Le is higher than the end of the file)
64h	00h	Execution error (file descriptor checksum error)
67h	00h	Incorrect length, Lc (if using secure messaging)
69h	82h	Security status not satisfied, for example: • The security attributes are not satisfied • Error during secure messaging
69h	86h	Command not allowed (no current EF)
6Ah	80h	Incorrect Lpv, Lcg or Le when using secure messaging in application phase
6Ah	81h	Function not supported: SM in ENC/MAC mode specified for the response and command was issued with no SM.
6Ah	82h	File not found (b8 of P1 = 1 indicating implicit selection, but no SFI given)
6Ah	86h	Incorrect P1, P2 (check that the offset is inside the EF and that the SFI is not 1Fh)
6Bh	00h	Incorrect P1, P2 (offset outside EF)

SELECT FILE

The **Select File** command selects a DF or an EF.

If P1, P2 and Lc are all zero, the command selects the root but returns no data. If you want to select the root and return the file control information, FCI, you must select it by its file ID (3F00h).

Conditions of Use and Security

Usage

The root and DFs can be selected by their file identifier, by their path or by their name, but not by partial name. EFs can be selected by their file identifier or by their path.

The **Select File** command is available during the personalization phase and the application phase. For information on how to use the command in the application phase, see “Select File” on page 188.

The command cannot be used to select a deactivated file.

Security

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of secure messaging for the **Select File** command is set by the **External Authenticate (Perso)** command. For details on the command format to use with secure messaging, refer to “Secure Messaging During the Personalization Phase” on page 40.

Note: For clarity, the command is described here with the data in plain text.

Format

This command is formatted as follows:

CLA	INS	P1	P2	Lc	Data	Le
CLA	A4h	Ref Ctrl	Ref Ctrl	Lc	Data	Le

Where

CLA 00h transmit normally

04h transmit with secure messaging

P1 is a reference control parameter. The bit settings are as follows:

P1	b8	b7	b6	b5	b4	b3	b2	b1
RFU	0	0	0	0	x	x	x	x
Select by path from root					1	0	0	0
Select DF or root by name					0	1	0	0
Select EF by file ID under current DF					0	0	1	0
Select root, DF or EF by File ID					0	0	0	0

Note: When P1 is 00h, to indicate file selection by file ID, but not under the current DF, GemSafe v2 searches all the files in the GemSafe v2 applet in the order of creation. It selects the first file that matches the file ID specified in the data field, which might not be the file intended.

P2 is the reference control parameter that specifies the format of the response or that there is no response. The bit settings are as follows:

P2	b8	b7	b6	b5	b4	b3	b2	b1
RFU	0	0	0	0				
FCI option					x	x		
Return FCI template					0	0		
Return FCP template					0	1		
No response					1	1		
First or only file							0	0

Lc is the length of the data field as follows:

- 02h for file selection by ID
- 01h–10h for selection by DF or root name
- 02h or 04h for file selection by path

Data is the file reference and is one of the following:

- file ID identifier of the file to be selected
- DF name name of the DF to be selected
- path is the concatenation of the file IDs from the root or current DF to the file ID of the file to be selected, not including the file ID of the current DF or root. For more detailed information about paths, see “Referencing by Path” on page 9.

Le is the length of the data returned, which is determined by the P2 setting.

Response

Response	SW1	SW2

The card returns the data requested by the P2 reference control parameter, followed by the SW1, SW2 status codes. The response data is either in the FCI template, the FCP template or does not exist (P2 = 0Ch).

The FCP template contains the file control parameters that were specified when creating the file. For DFs, this data is as shown in Table 51, “Example of DF Creation”, on page 78 but does not include the FDB (TLV= 82h 01h 38h). For EFs the data is as shown in Table 52, “Example of EF Creation”, on page 78.

The FCI information returned by the command (P2 = 00h) is shown in “Table 73” and “Table 74”.

For DFs:

Offset	Data	Description
0	6Fh	Tag of FCI template
1	L	Length of FCI data
2	83h	Tag of File ID
3	02h	Length of File ID
4–5	File ID	Value of File ID
6	8Ch	Tag of security attributes
7	L	Length of security attributes
8	AM	Access mode byte
9–(8+X)	SC	Security condition bytes (X)
9+X	84h	Tag of DF Name
10+X	L	Length of DF Name
11+X –	DF name	Value of DF name (up to 16 bytes)

Table 73 - FCI for DFs

For EFs

Offset	Data	Description
0	6Fh	Tag of FCI template
1	L	Length of FCI data
2	81h	Tag of File Size
3	02h	Length of File Size
4–5	File Size	Value of File Size
6	82h	Tag of FDB
7	01h	Length of FDB
8	FDB	Value of FDB
9	83h	Tag of File ID
10	02h	Length of File ID
11–12	File ID	Value of File ID
13	8Ah	Tag of Life Cycle Status byte for file
14	01h	Length of Life Cycle Status byte for file
15	Var	Value of Life Cycle Status byte for file
16	8Ch	Tag of security attributes
17	L	Length of security attributes
18	AM	Access mode byte
19–(18+X)	SC	Security condition bytes (X)

Table 74 - FCI for EFs

Possible values for the SW1, SW2 status codes are as follows:

SW1	SW2	Description
90h	00h	Command processed without error. No FCI data returned
62h	83h	Deactivated file
64h	00h	Execution error (file descriptor checksum error)
67h	00h	Incorrect length, Lc
69h	82h	Security status not satisfied, error during secure messaging
6Ah	82h	File not found
6Ah	86h	Incorrect P1, P2 parameter

UPDATE BINARY

This command updates all or part of an EF.

There are two ways of referencing the EF:

- Implicit selection. In this case, the EF is under the current DF, and is specified by its short file identifier (SFI). P1 specifies the SFI and P2 specifies the start address for the update (called the offset). The command cannot be used for files with an SFI of 1Fh.
- Direct selection. In this case the EF must have been previously selected using the **Select File** command. The data is updated directly in the current EF. In this case the offset is specified over P1 and P2.

The command updates the number of bytes specified in Lc from this offset.

Conditions of Use and Security

Usage

The **Update Binary** command is available during the personalization phase and the application phase. For details on its use in the application phase, see “Update Binary” on page 192.

The command can be used on any activated EF. It is not valid for deactivated EFs, for DFs or for data objects.

In the case of implicit selection, the EF becomes the current EF after execution of the command.

Security

During the personalization phase, a Card Manager/terminal mutual authentication must be performed to open a secure channel at the start of the session. Once this channel is established, there is no limitation to the number of personalization commands that the terminal can send. The level of secure messaging for the **Update Binary** command is set by the **External Authenticate (Perso)** command. For details about how to send the command with secure messaging, see “Secure Messaging During the Personalization Phase” on page 40.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

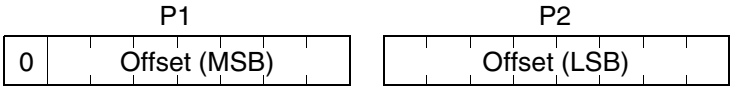
CLA	INS	P1	P2	Lc	Data
CLA	D6h	P1	P2	Lc	Data

Where:

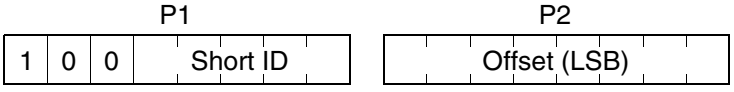
CLA
- 00h — transmit normally
- 04h — transmit with secure messaging

P1, P2 are the parameter bytes.

To update the currently selected EF, P1 and P2 specify the offset, in bytes, from the beginning of the file to the first bytes to be updated. P1 and P2 have the following format:



To select and update an EF, P1 specifies the SFI of the file containing the data to be updated. P2 specifies the offset, in bytes, from the beginning of the file to the first byte to be updated. P1 and P2 have the following format:



Lc indicates the length of the data field

Data contains the value of the data to be updated in the card

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
62h	82h	End of file reached (offset + Lc is higher than the end of the file)
64h	00h	Execution error (file descriptor checksum error)
67h	00h	Incorrect length, Lc
69h	82h	Security status not satisfied, for example: • The security attributes are not satisfied • Error during secure messaging
69h	86h	Command not allowed (no current EF)
6Ah	82h	File not found (b8 of P1 = 1 indicating implicit selection, but no SFI given)
6Ah	86h	Incorrect P1, P2 (check that offset is inside the EF and that SFI is not 1Fh)
6Bh	00h	Incorrect P1, P2 (offset outside EF)

11

Application Phase Commands

This chapter describes the GemSafe v2 applet commands that are available in the application phase. In order to illustrate each command clearly, each description shows the case where the command is sent without secure messaging. For details on the command format to use with secure messaging, refer to “Secure Messaging During the Application Phase” on page 43.

The following table lists the commands that are available in the application phase.

Command	Description	Page
Activate File*	Changes the state of a file to OPERATIONAL (ACTIVATED).	129
Change Reference Data	Changes the value of a PIN.	132
Create File	Creates an EF under the root or the currently selected DF or creates a DF under the root.	134
Deactivate File*	Changes the state of a file to OPERATIONAL (DEACTIVATED).	135
Erase Binary*	Erases part of a binary file.	138
External Authenticate (Application)	Authenticates the terminal to the card by checking for the presence of the certified private key SK.IFD.AUT	141
Generate Public Key Pair*	Generates an RSA key pair and stores both keys in the card. It returns the modulus as part of its response.	144
Get Challenge	Generates an eight–byte random number.	146
Get Data*	Retrieves one of the following: • CPLC data • Applet version • Applet personalized data • Public key elements • K_{ICC} • The contents of a specified SE	147
Get Response	Retrieves the response to a command APDU	154
Internal Authenticate	Authenticates the card to the terminal by checking for the presence of the certified private key SK.ICC.AUT	155

Table 75 - Application Commands

Command	Description	Page
Manage Security Environment*	Supports two functions, **Restore** and **Set**. **Restore** replaces the current SE by an SE stored in the card. **Set** sets or replaces one component of the current SE.	157 158
Mutual Authenticate	Authenticates the card and the terminal to each other by checking for the presence of the two secret keys K_{ENC} and K_{MAC} and generates the session keys used in secure messaging.	160
PSO–Compute Digital Signature*	Computes a digital signature.	162
PSO–Decipher*	Deciphers an encrypted message using a utility key stored in the card.	165
PSO–Hash*	Entirely or partially hashes data prior to a **PSO–Compute Digital Signature** command or prepares the data if hashed externally	168
PSO–Verify Certificate	Sends the IFD certificate C_CV.IFD.AUT used in asymmetric key mutual authentication to the card for verification	172
Put Data*	Creates or updates a secret key, RSA private key, RSA public key or DH key exchange parameters data object.	174
Read Binary	Reads part of a binary file.	183
Reset Retry Counter	Unblocks a PIN.	186
Select File*	Selects a DF or an EF by its file ID, path or name (in the case of DFs).	188
Update Binary*	Updates part of a binary file.	192
Verify	Authenticates a user by comparing the entered PIN with the reference PIN.	194

Table 75 - Application Commands (continued)

* This command is also available during the personalization phase.

For each APDU command, GemSafe v2 verifies the following conditions, and returns the corresponding status codes if the check fails:

1. Check that the class byte is allowed. => **6E 00h**
2. Check the instruction code.
   a) Check that instruction code is present and valid. => **6D 00h**
   b) Check instruction code is compatible with class byte. => **68 00h**
   c) Check instruction is allowed for the current card life phase. => **6D 00h**
   d) Check instruction is allowed in current DF or application. => **69 85h**

A comprehensive description of each command is given in the following sections.

ACTIVATE FILE

The **Activate File** command changes the state of an EF to OPERATIONAL (ACTIVATED). The state of a file is stored in the life cycle status byte in the EF's file descriptor. For more details about the file life cycle status refer to Table 4 on page 8.

The file can be referenced by its file identifier or by its path. For more details about these methods, see "Referencing by File Identifier" on page 8 and "Referencing by Path" on page 9. The command can be used to activate the current EF by specifying P1 and P2 as 00h. In this case, the data field is empty.

Conditions of Use and Security

Usage

The command can be used in either the personalization phase or the application phase. For details about its use in the personalization phase, see "Activate File" on page 73.

The command can be used on EFs only. It is not valid for DFs or for data objects.

The EF can be in any state prior to issuing the **Activate File** command. If the file is already in the state "operational (activated)", the **Activate File** command has no effect, but does not fail.

If P1 and P2 are both 00h, the command applies to the currently selected EF. In this case Lc is zero and the data field is empty.

Security

Any EF in the life cycle status CREATED or INITIALIZED can be freely activated. If the EF to be activated is in either of the OPERATIONAL life cycle statuses and is protected by a security attribute, then this security attribute must be fulfilled in order to perform the **Activate File** command. For more details see "Security Attributes" on page 33.

During the application phase, use of the command depends on the security attribute. The security attribute specifies if prior mutual authentication is necessary, and if so whether or not secure messaging is also necessary.

For details on the command format to use with secure messaging see "Secure Messaging During the Application Phase" on page 43.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	44h	P1	P2	Lc	Data

Where:

CLA 00h transmit normally

0Ch transmit with secure messaging

P1 indicates the file referencing method and is coded as follows:

P1	b8	b7	b6	b5	b4	b3	b2	b1
RFU	0	0	0	0				
Reference EF by path					1	0	0	0
Reference EF by File ID					0	0	X	X
Currently selected EF							0	0
Search for EF under current DF							1	0

P2 00h indicates file is the currently selected EF. This value is valid only if P1 = 00h.

0Ch indicates no response data

Lc indicates the length of the data field as follows:

00h indicates no data (when P1 and P2 are 00h)
02h if selecting EF by file ID
xxh where “xxh” is the length of the path name

Data is either the 2–byte file ID or the path name. The path is the concatenation of the file IDs from the root down to the file ID of the EF to be activated. The file ID of the root is not included in the path.

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
64h	00h	Execution error (file descriptor checksum error)
67h	00h	Incorrect length, Lc. Lc must be less than 248 bytes when using secure messaging in the application phase.
69h	82h	Security conditions not satisfied (security attribute not fulfilled or error during secure messaging)
6Ah	80h	Incorrect data, for example: • Referenced file not found • Incorrect Lpv, Lcg or Le when using secure messaging in application phase
6Ah	81h	Function not supported (P1 and P2 = 00 but no current EF, command not valid for DFs)
6Ah	86h	Incorrect P1, P2. This includes the case when P1 and P2 are 00h but a data field exists.

CHANGE REFERENCE DATA

The **Change Reference Data** command replaces the value of the reference PIN by a new value. The security attributes of a PIN can specify that certain access conditions must be satisfied before this command is allowed (see “PIN Contents” on page 38). If the security attributes do not contain an SC byte for Change Reference Data, then the reference PIN can never be modified.

The current value of the PIN must be presented as part of the **Change Reference Data** command.

If the **Change Reference Data** command is successful, the following actions take place:

- The reference PIN Try Counter is set to PIN Try Limit
- The reference PIN validated flag is set to false (because the new value has not yet been verified)
- The reference PIN value is updated with the value sent in the **Change Reference Data** command
- The reference PIN changed flag is set to “true”
- The reference PIN usage counter is unchanged.

If the **Change Reference Data** command fails, the following actions take place:

- The reference PIN Try Counter is decremented by 1
- The reference PIN validated flag is set to false

For more information about the structure of PINs and their security attributes, see “PIN Management” on page 37.

Conditions of Use and Security

Usage

- The command is available only in the application phase, that is when the GemSafe v2 applet is in the life cycle state PERSONALIZED.
- The new PIN value must be the same length as the previous one.

Security

The reference PIN can be changed only if the security attribute specified in the security attributes has been satisfied.

The PIN Try Counter and the Usage Counter for both the reference PIN and the change PIN must not be zero.

The use of secure messaging is determined by the security attribute of the PIN.

Note: For clarity, the command is described with the data in plain text. For information on how to send the command with secure messaging, refer to “Secure Messaging During the Application Phase” on page 43.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	24h	00h	P2	10h–20h	Change PIN || New PIN

Where:

CLA = 00h transmit normally

0Ch transmit with secure messaging

P2 is the PIN reference (81h–8Fh)

Data is as follows:

- Change PIN (8–16 bytes)
- New PIN value (8–16 bytes)

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
63h	Cxh	Reference PIN not verified.”x” attempts remaining for change PIN
67h	00h	Incorrect length for example: • Lc incorrect (Lc must be less than 248 bytes when using secure messaging). • New PIN value is not the same as existing value
69h	82h	Security conditions not satisfied (error during secure messaging or security attribute of reference PIN for **Change Reference Data** is never)
69h	83h	Authentication method blocked (Change PIN blocked)
69h	84h	Referenced data invalidated (PIN Try Counter or PIN Usage Counter for either the reference PIN or the change PIN has reached zero)
6Ah	80h	Incorrect Lpv, Lcg or Le when using secure messaging in application phase
6Ah	86h	Incorrect P1, P2
6Ah	88h	Referenced data not found

CREATE FILE

The **Create File** command creates an elementary file (EF) under the current DF or the root or a dedicated file (DF) under the root.

Note: As the **Erase Binary**, **Read Binary** and **Update Binary** commands specify an offset on 15 bits, Gemplus recommends that you do not create files of 32KB or larger.

Conditions of Use and Security

Usage

The command can be used in either the personalization phase or the application phase. For a description of the command, see “Create File” on page 76.

Note: To send the command with secure messaging in the application phase, the CLA byte is OCh not 04h.

Security

The **Create File** command is allowed only if specified in the security attributes of the parent file. For more details, see “Access Mode Byte” on page 33.

In the application phase, the use of secure messaging and whether or not the command must be preceded by mutual authentication, depends on the security attribute of the parent file.

DEACTIVATE FILE

The **Deactivate File** command changes the state of an EF to OPERATIONAL (DEACTIVATED). The state of a file is stored in the life cycle status byte in the EF's file descriptor. For more details about the file life cycle statuses refer to Table 4 on page 8.

This change is reversible; a file can reactivated by issuing the **Activate File** command. Once a file is deactivated, the only command that is allowed for that file is **Activate File**.

The file can be referenced by its file identifier or by its path. For more details about these methods, see "Referencing by File Identifier" on page 8 and "Referencing by Path" on page 9. The command can be used to activate the current EF by specifying P1 and P2 as 00h. In this case, the data field is empty.

Conditions of Use and Security

Usage

The command can be used in either the personalization phase or the application phase. For details about its use in the personalization phase, see "Deactivate File" on page 80.

The command can be used on EFs only. It is not valid for DFs or for data objects.

The command is valid only for EFs in the state OPERATIONAL (ACTIVATED).

Security

If the EF to be activated is protected by a security attribute, then this security attribute must be fulfilled in order to perform the **Deactivate File** command. For more details see "Security Attributes" on page 33.

During the application phase, use of the command depends on the security attribute. The security attribute specifies if prior mutual authentication is necessary, and if so whether or not secure messaging is also necessary.

For details on the command format to use with secure messaging see "Secure Messaging During the Application Phase" on page 43.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	04h	P1	P2	Lc	Data

Where:

CLA 00h transmit normally

0Ch transmit with secure messaging

P1 indicates the file referencing method and is coded as follows:

P1	b8	b7	b6	b5	b4	b3	b2	b1
RFU	0	0	0	0				
Reference EF by path					1	0	0	0
Reference EF by File ID					0	0	X	X
Currently selected EF							0	0
Search for EF under current DF							1	0

P2 00h indicates file is the currently selected EF. This value is valid only if P1 = 00h

0Ch indicates no response data

Lc indicates the length of the data field as follows:

00h indicates no data (when P1 and P2 are 00h)

02h if selecting EF by file ID

xxh where “xxh” is the length of the path name

Data is either the 2–byte file ID or the path name. The path is the concatenation of the file IDs from the root down to the file ID of the EF to be activated. The file ID of the root is not included in the path.

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
64h	00h	Execution error (file descriptor checksum error)
67h	00h	Incorrect length, Lc. Lc must be less than 248 bytes when using secure messaging in the application phase
69h	82h	Security conditions not satisfied (security attribute not fulfilled or error during secure messaging)
6Ah	80h	Incorrect data, for example: • Referenced file not found • Incorrect Lpv, Lcg or Le when using secure messaging in application phase
6Ah	81h	Function not supported (P1 and P2 = 00 but no current EF, command not valid for DFs)
6Ah	86h	Incorrect P1, P2

ERASE BINARY

This command erases all or part of an EF.

There are two ways of referencing the EF:

- Implicit selection. In this case, the EF is under the current DF, and is specified by its short file identifier (SFI). P1 specifies the SFI and P2 specifies the start address for the erase (called the offset). The command cannot be used for files with an SFI of 1Fh.
- Direct selection. In this case the EF must have been previously selected using the **Select File** command. The data is erased directly from the current EF. In this case the offset is specified over P1 and P2.

The command erases data from the offset until the end of the file is reached or until the end address in the data field, if one exists.

Note: The end address is in fact the address of the first byte NOT to be erased.

Conditions of Use and Security

Usage

The **Erase Binary** command is available during the personalization phase and the application phase. For details on its use in the personalization phase, see “Erase Binary” on page 84.

The command can be used on any activated EF. It is not valid for deactivated EFs, for DFs or for data objects.

In the case of implicit selection, the EF becomes the current EF after execution of the command.

Security

If the EF containing the data to be erased is protected by a security attribute, then this security attribute must be fulfilled in order to perform the **Erase Binary** command. For more details see “Security Attributes” on page 33.

During the application phase, use of the command depends on the security attribute. The security attribute specifies if prior mutual authentication is necessary, and if so whether or not secure messaging is also necessary.

For details on the command format to use with secure messaging see “Secure Messaging During the Application Phase” on page 43.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	0Eh	P1	P2	Lc	Data

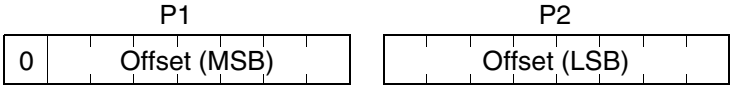
Where:

CLA 00h transmit normally

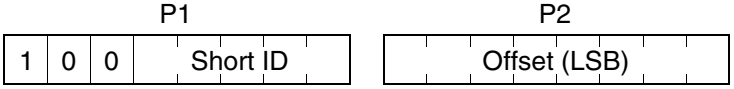
0Ch transmit with secure messaging

P1, P2 are the parameter bytes.

To erase from the currently selected EF, P1 and P2 specify the offset, in bytes, from the beginning of the file to the first byte to be erased. P1 and P2 have the following format:



To select and erase from an EF, P1 specifies the SFI of the file containing the data to be erased. P2 specifies the offset, in bytes, from the beginning of the file to the first byte to erase. P1 and P2 have the following format:



Lc indicates the length of the data field as follows:

empty if no data field

02h if end address specified in data field

Data indicates the end point for the erase as follows:

empty indicates the command erases data until the end of the file

XXh XXh if present indicates the offset in bytes of the first byte NOT to be erased

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
62h	82h	End of file reached (address specified in data field is higher than the end of the file)
64h	00h	Execution error (file descriptor checksum error)
67h	00h	• Incorrect length, Lc. Lc must be less than 248 bytes when using secure messaging in the application phase • Offset given in data field is not higher than offset of start address in P1
69h	82h	Security status not satisfied, for example: • The security attributes are not satisfied • Error during secure messaging
69h	86h	Command not allowed (no current EF)
6Ah	80h	Incorrect Lpv, Lcg or Le when using secure messaging in application phase
6Ah	82h	File not found (b8 of P1 = 1 indicating implicit selection, but no SFI given)
6Ah	86h	Incorrect P1, P2 (check that the offset is inside the EF and that the SFI is not 1Fh)
6Bh	00h	Incorrect P1, P2 (offset outside EF)

EXTERNAL AUTHENTICATE (APPLICATION)

The **External Authenticate (Application)** command authenticates a device such as an external terminal to the GemSafe v2 applet. It does this by signing a challenge from the card with the certified private key, SK.IFD.AUT, thus proving to the card that it possesses this key. The card needs the terminal's public authentication key PK.IFD.AUT to verify this signature.

For overall details about asymmetric key mutual authentication, refer to "Asymmetric Key Mutual Authentication (with Privacy)" on page 57. For details about the Diffie–Hellman key exchange parameters, see "Appendix C - Generating Session Keys with the Diffie–Hellman Algorithm".

Prior to the **External Authenticate (Application)** command:

1. The terminal has sent its authentication certificate C_CV.IFD.AUT to the card via a **PSO: Verify Certificate** command. From this, the card retrieved the terminal's public key, PK.IFD.AUT. This corresponds to step "Figure " on page 60 of the asymmetric key mutual authentication procedure. See page 172 for a description of **PSO: Verify Certificate**.
2. The terminal retrieved the card's data element, K_{ICC} using a **Get Data** command. This enables the terminal to calculate the shared secret data $K_{IFD/ICC}$. The data is hashed before sending, the result being C.

Command Processing

1. The terminal sends the following data to the card in the **External Authenticate (Application)** command:

•SN.IFD	Terminal Serial Number
•Signature	Generated using its private key SK.IFD.AUT. The signed data includes the hash (C). For more information about the contents of C, refer to Table 76, "Data Field for External Authenticate (Application) Command", on page 142.

2. The card verifies the signature using the terminal's public key PK.IFD.AUT. This reveals the hash computation, C and thus K_{IFD}. This also proves the signature was made by SK.IFD.AUT.
3. Now that the card has K_{IFD} it computes the shared secret data, $K_{IFD/ICC}$ and from this derives the session key K_{SK}ENC.

For more details about these steps and parameters, refer to "Asymmetric Key Mutual Authentication (with Privacy)" on page 57.

Conditions of Use and Security

Usage

The command is available only in the application phase, that is when the GemSafe v2 applet is in the life cycle state PERSONALIZED.

The command can be sent with Lc=00h and no data to check if authentication has already been made.

Security

The authentication ratification counter is decremented each time that the **External Authenticate (Application)** command fails. When the command is successful, the counter is reset to its initial value. The initial value of the authentication ratification counter is set with the **Install** command, see Table 18 on page 25.

The command does not use secure messaging.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
80h	82h	00h	00h	88h or 00h	Data

Where:

Lc is the length of the data. This is always 88h (136 bytes) or 00h

Data is as shown in Table 76 on page 142.

	Value	Description	No. of Bytes
	SN.IFD	Terminal Serial Number	8
Signature	6Ah	Padding byte as specified in ISO/IEC 9796	1
	XXh XXh.....XXh	PRND2 random padding bytes (not equal to 00h) generated by the terminal	106
	C = SHA-1 [PRND2 || K_{IFD} || SN.IFD || CRnd || K_{ICC} || DH.P]	Result of hash computation	20
	BCh	Padding byte as specified in ISO/IEC 9796	1
		Total Number of Bytes	136[1]

Table 76 - Data Field for External Authenticate (Application) Command

[1] 136 bytes = 88h = Lc

Where:

SN.IFD is the terminal serial number

PRND2 is the random padding bytes generated by the terminal

K_{IFD} is the data element generated by the terminal using the Diffie–Hellman key exchange parameters

CRnd is the random number generated by the card and returned by the **Get Challenge** command

K_{ICC} is the data element generated by the card using the Diffie–Hellman key exchange parameters

DH.P is the concatenation of the Diffie–Hellman key exchange parameters

Response

SW1	SW2

The card returns the SW1, SW2 status codes. Possible status code values are:

SW1	SW2	Description
90h	00h	Command processed without error
63h	00h	Authentication process blocked—too many failed attempts
63h	Cxh	Authentication failed—”x” attempts remaining
64h	00h	Data corrupted
67h	00h	Incorrect length
69h	85h	Command conditions not satisfied
6Ah	86h	Incorrect P1, P2
6Ah	88h	Data not found

GENERATE PUBLIC KEY PAIR

The **Generate Public Key Pair** command is GemSafe v2's on board public key generation function. It does not create the key pair data objects, but can be used to initialize or update an existing key pair. It stores the private key and public key in the GemSafe v2 applet and returns the value of the corresponding public key modulus in its response to the terminal.

Note: The command sets the value of the keys but does not change their access conditions.

The command is particularly useful for applications where the private key is not stored anywhere other than in the card, and has the advantage that GemSafe v2 controls the security of the operation.

Note: The **Put Data** command can be used after mutual authentication to update private key values. In this case the values are generated by the application outside GemSafe v2 and the application is responsible for the security of the private key value.

Conditions of Use and Security

Usage

The command is also available in the personalization phase.

Security

In the application phase, the security attributes for the private key data object must be satisfied to update the value of the private key. GemSafe v2's built–in security policy ensures that the command is allowed only after mutual authentication and must always use secure messaging in at least MAC mode during the application phase. For more information refer to "For Data Objects" on page 34, and Table 26, "Built-in Minimum Levels of Secure Messaging", on page 37.

In the personalization phase, GemSafe v2 does not check the security attributes.

For details on the command format to use with secure messaging see "Secure Messaging During the Application Phase" on page 43.

Note: For clarity, the command is described here with the data in plain text.

Format

This command is formatted as follows:

CLA	INS	P1	P2	Lc	Data	Le
CLA	46h	02h	P2	Lc	Data	Le

Where:

CLA 00h personalization phase without secure messaging

04h personalization phase with secure messaging

0Ch application phase (always with secure messaging)

P2 is the key ID of the private key to be updated

Lc is the length of the key exponent data object

Data is the key exponent data object in TLV format as follows:

Tag	Length	Tag	Length	Value
7F49h	Length of TLV for exponent	82h	01h–08h	Public Exponent

Le is the expected length of the response.

Response

Response	SW1	SW2

The response is the modulus of the public key in TLV format as follows:

Tag	Length	Tag	Length	Value
7F49h	$L_{Mod} + 2$	81h	L_{Mod}	Modulus

Note: If the response is greater than 256 bytes the card opens a retrieval sequence. The sequence differs according to the communication protocol in use (T=0 or T=1). For details see “Retrieval Sequence for Large Outgoing Data” on page 70.

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
67h	00h	Incorrect length, Lc. Lc must be less than 248 bytes when using secure messaging in application phase
69h	82h	Security status not satisfied: • The update security attribute for the private key has not been fulfilled • Error during secure messaging
6Ah	80h	Incorrect data, for example: • Incorrect tag or tag length (including TLVs used in secure messaging) • Incorrect data message length
6Ah	86h	Incorrect P1, P2

GET CHALLENGE

The **Get Challenge** generates a response that is used by the **External Authenticate (Application)** command (for asymmetric key mutual authentication) or the **Mutual Authenticate** command (for symmetric key mutual authentication). For details about mutual authentication, see “Chapter 8 - Mutual Authentication”.

Conditions of Use and Security

Usage

The random number generated by the **Get Challenge** command is valid for the next command only.

The command is available only in the application phase, that is when the GemSafe v2 applet is in the life cycle state PERSONALIZED.

If authentication is already in progress, issuing the **Get Challenge** command breaks the session. The **Get Challenge** command cannot be used if the **Mutual Authenticate** command has been successfully performed during the session.

Security

The **Get Challenge** command is performed without secure messaging.

Format

The command format is as follows:

CLA	INS	P1	P2	Le
80h	84h	00h	00h	08h

Response

Response	SW1	SW2

The response is the eight–byte challenge RND.ICC

Following the response, the card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
67h	00h	Incorrect length
69h	82h	Security conditions not satisfied
6Ah	81h	Function not supported: SM in ENC/MAC mode specified for the response and command was issued with no SM.
6Ah	86h	Incorrect P1, P2
6Dh	00h	Instruction not supported

GET DATA

This command retrieves one of the six following sets of data:

- The card production life cycle (CPLC) data, without selecting the Card Manager.
- The applet version.
- The applet personalization data set during the **End Personalization** command.
- Public key elements (a BER-TLV object)
- The value of K_{ICC}, a data element generated by the card during mutual authentication (a BER-TLV object)
- The contents of a specified SE (a BER-TLV object)

It cannot be used to retrieve any other data. The data in the response is in plain text for CPLC and the applet version and in TLV format for public key elements and SEs.

Conditions of Use and Security

Usage

The **Get Data** command is available during the personalization phase and the application phase. For details on how to use it during the personalization phase, see “Get Data” on page 89.

Security

For non BER–TLV objects, the command can be used before or after mutual authentication and secure messaging is optional. However for BER–TLV objects, the use of the **Get Data** command depends on the security attributes of the object whose value is to be retrieved. For details on the command format to use with secure messaging see “Secure Messaging During the Application Phase” on page 43.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc		Le
00h	INS	P1	P2	Lc	Data	Le

The values depend on the type of object being retrieved. The following table shows the possible values:

Data object	INS	P1	P2	Lc	Data	Le
CPLC Data	CAh	9Fh	7Fh	Empty	Empty	2Ah
Applet Version	CAh	DFh	30h	Empty	Empty	08h
Applet personalization data	CAh	DFh	31h	Empty	Empty	0Ch
BER–TLV	CBh	00h	FFh	Lc	TLV structure	Le

Table 77 - Get Data Values (Perso Phase)

Note: The format of this command has been designed for backward compatibility.

Examples of Coding the Data Field for BER–TLV Objects

To retrieve a public key element (the modulus in this example):

INS = CBh

P1/P2 = 00h FFh

Lc = 0Ah

Data is as shown in the following table

Tag	Length	Value			Description	No. of Bytes
B6h	03h				Tag and length of DST Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	02h				Tag and length of public key template	3
		Tag	**Length**			
		81h	00h		Tag and length of modulus (value coded on 128 bytes)	2
					TOTAL BYTES	**10**

Table 78 - Get Data Coding Structure – Public Key Element

Note: The length of 00h means that all the element is recovered.

To retrieve an entire public key, that is, the modulus and exponent elements:

INS = CBh

P1/P2 = 00h FFh

Lc = 0Ah

Data is as shown in the following table

Tag	Length	Value			Description	No. of Bytes
B6h	03h				Tag and length of DST Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	02h				Tag and length of public key template	3
		Tag	**Length**			
		80h	00h		Tag and length of entire public key	2
					TOTAL BYTES	**10**

Table 79 - Get Data Coding Structure – Entire Public Key

Note: The length of 00h means that all the element is recovered.

To retrieve a K_{ICC}

INS = CBh

P1/P2 = 00h FFh

Lc = 04h

Data is as shown in the following table

Tag	Length				Description	No. of Bytes
A6h	02h				Tag and length of KAT Template	2
		Tag	**Length**			
		91h	00h		Tag and length of K_{ICC} (a random data object)	2
					TOTAL BYTES	**4**

Table 80 - Get Data Coding Structure – K_{ICC}

Note: The length of 00h means that all the element is recovered.

To retrieve a Diffie–Hellman element ("p" in this example)

INS = CBh

P1/P2 = 00h FFh

Lc = 0Ah

Data is as shown in the following table

Tag	Length	Value			Description	No. of Bytes
A6h	03h				Tag and length of KAT Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	02h				Tag and length of public key template	3
		Tag	**Length**			
		86h	00h		Tag and length of DH p element	2
					TOTAL BYTES	**10**

Table 81 - Get Data Coding Structure – Diffie–Hellman Element

Note: The length of 00h means that all the element is recovered.

To retrieve all the contents of an SE:

INS = CBh

P1/P2 = 00h FFh

Lc = 05h

Data =

T	L	T	L	V
7Bh	03h	80h	01h	SEID

Where:

7Bh is the tag for an SE

80h indicates that the entire SE is recovered.

Response

Response	SW1	SW2

Response is coded as follows:

CPLC Data

Data Element	Length (in bytes)
IC Fabricator	2
IC Type	2
Operating System Identifier	2
Operating System Release Date	2
Operating System Release Level	2
* IC Fabrication Date	2
* IC Serial Number	4
* IC Batch Identifier	2
IC Module Fabricator	2
IC Module Packaging Date	2
ICC Manufacturer	2
IC Embedding Date	2
IC Pre–Personalizer	2
IC Pre–Personalization Date	2
IC Pre–Personalization Equipment Identifier	4
IC Personalizer	2
IC Personalization Date	2
IC Personalization Equipment Identifier	4

Table 82 - Get Data Response Structure – CPLC Data

* The chip serial number, SN.ICC is IC Fabrication Date || IC Serial Number || IC Batch Identifier.

Applet Version

The command returns the applet version in TLV format as follows:

Tag	Length	Value
DFh 30h	05h	Applet version

The applet version is returned as five ASCII characters to display “vX.XX”. For example an applet version of v1.25 is returned as follows:

ASCII Value	76h	31h	2Eh	32h	35h
Display value	v	1	.	2	5

Applet Personalization Data

This is the data that is set during the **End Personalization** command as follows:

Data Element	Length (in bytes)
Tag (DFh 31h)	2
Length (09h)	1
DTHR (date and time supplied by the personalization device). The format of this data is not imposed by GemSafe v2.	5
TermID (identifier of the personalization device)	4

Public Key Element (the Modulus in this Example)

The response is as shown in the following table:

Tag	Length	Value			Description	No. of Bytes
B6h	03h				Tag and length of DST Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	L				Tag and length of public key template	3
		Tag	**Length**	**Value**		
		81h	L_{Mod}	Val_{Mod}	TLV of modulus (value coded on 128 bytes)	130
					TOTAL BYTES	**138**

Table 83 - Get Data Response Structure – Public Key Element

Caution: There is one case where the format of the response differs according to the version of the GemSafe V2 applet. This case is when retrieving the modulus for a 2048 key using secure messaging. For versions of the applet before 2.03, the first seven bytes of the response must be ignored.

Entire Public Key

The response is as shown in the following table:

Tag	Length	Value			Description	No. of Bytes
B6h	03h				Tag and length of DST Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	L				Tag and length of public key template	3
		Tag	**Length**	**Value**		
		81h	L_{Mod}	Val_{Mod}	TLV of modulus (value coded on 128 bytes)	130
		82h	L_{EXP}	Val_{EXP}	TLV of exponent (value coded on 8 bytes)	**8**
					TOTAL BYTES	**146**

Table 84 - Get Data Response Structure – Entire Public Key

K_{ICC}

The response is as shown in the following table:

Tag	Length				Description	No. of Bytes
A6h	L				Tag and length of KAT Template	2
		Tag	**Length**	**Value**		
		91h	81h 80h	K_{ICC}	Tag and length of K_{ICC}	131
					TOTAL BYTES	**133**

Table 85 - Get Data Response Structure – K_{ICC}

The Entire Contents of an SE (AT Containing a PIN in this Example)

The response is as shown in the following table:

Tag	Length	Value				
7B	L_{SE}					Tag and Length of current SE
		Tag	**Length**	**Value**		**Description**
		A4h	06h			Tag and length of AT
				Tag	**Length**	**Value**	Value of AT
				83h	01h	PIN Ref	TLV of PIN ref
				95h	01h	08h	CRT Usage qualifier (value 08h indicates user authentication)

Table 86 - Get Data Response Structure for an SE

The SW1, SW2 possible status codes values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
69h	82h	Security status not satisfied, error during secure messaging
6Ah	80h	Incorrect Lpv, Lcg or Le when using secure messaging in application phase
6Ah	86h	Incorrect P1, P2.

GET RESPONSE

The **Get Response** command is an administrative and application TPDU mechanism, that is used to recover the response for case 4 APDU commands. When using the T=0 protocol, the **Get Response** command is sent automatically by the APDU commands. However when using the T=1 protocol it needs to be issued after the original command in order to retrieve the response.

Conditions of Use and Security

Usage

The **Get Response** command must be used only after command case 4. It can be used in the personalization phase and the application phase.

Security

The **Get Response** command can be accessed freely.

Format

CLA	INS	P1	P2	Le
00h	C0h	00h	00h	Le

Where:

Le Expected length of data.

Response

Response	SW1	SW2

Where:

Response Holds the card response.

SW1 and SW2 Hold the status byte values returned by the card, as follows:

SW1	SW2	Description
61h	XXh	Le < Licc, XXh is the remaining number of available data bytes in the card
62h	81h	Part of returned data may be corrupted
6Ah	86h	Incorrect P1 P2
6Ch	Licc	if Le > Licc
6Fh	00h	No available data (Licc = 0)
90h	00h	Successful execution of the command (Le = Licc)

INTERNAL AUTHENTICATE

The **Internal Authenticate** command authenticates the card to the terminal. It does this by checking the GemSafe v2 applet for the presence of the certified private key, SK.ICC.AUT.

Prior to the **Internal Authenticate** command the terminal issued a **Read Binary** command to read the card's certificate, C.ICC.AUT used in asymmetric key mutual authentication. From this it obtained the card's public key PK.ICC.AUT.

The basic principle behind the **Internal Authenticate** command is as follows:

1. The terminal sends the **Internal Authenticate** command to receive the following data from the card:
   - The chip serial number SN.ICC
   - Signature of the hash C. C is the SHA–1 hash of PRND1 || K_{ICC} || SN.ICC || TRnd || K_{IFD} || DH.P.
2. The terminal verifies the signature using the card's public key PK.ICC.AUT, which it received during step "13."of the asymmetric key mutual authentication procedure. See page 62. This reveals the hash computation, C. If C is correct, this proves that the signature was made by SK.ICC.AUT and therefore that the card is certified.

For more details about these steps and parameters, refer to "Asymmetric Key Mutual Authentication (with Privacy)" on page 57. For more details about the Diffie–Hellman algorithm and deriving the session keys, K_{SK}ENC and K_{SK}MAC, see "Appendix C - Generating Session Keys with the Diffie–Hellman Algorithm".

Conditions of Use and Security

Usage

The command is available only in the application phase, that is when the GemSafe v2 applet is in the life cycle state PERSONALIZED.

Security

The command always uses secure messaging in MAC + ENC mode.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data	Le
8Ch	88h	00h	00h	08h	TRnd	88h

Response

Response	SW1	SW2

Response is coded as follows:

	Value	Description	No. of Bytes
	SN.ICC	Chip serial Number	8
Signature	6Ah	Padding byte as specified in ISO/IEC 9796	1
	XXh XXh.....XXh	PRND1 random padding bytes (not equal to 00h) generated by the card	106
	C = SHA-1 [PRND1 || K_{ICC} || SN.ICC || TRnd || K_{IFD} || DH.P]	Result of hash computation	20
	BCh	Padding byte as specified in ISO/IEC 9796	1
		Total Number of Bytes	136[1]

Table 87 - Data Field for Internal Authenticate Command

[1] 136 bytes = 88h = Lc

Where:

SN.ICC is the chip serial number

PRND1 is the random padding bytes generated by the card

K_{ICC} is the data element generated by the card using the Diffie–Hellman key exchange parameters

TRnd is the random number generated by the terminal.

K_{IFD} is the data element generated by the terminal using the Diffie–Hellman key exchange parameters

DH.P is the concatenation of the Diffie–Hellman key exchange parameters

The possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
67h	00h	Incorrect length
69h	82h	Data not initialized. Security status not satisfied
69h	85h	Command conditions not satisfied
6Ah	86h	Incorrect P1, P2
6Ah	88h	Data not found

MSE–RESTORE

The **MSE–Restore** command replaces the current SE by one stored in the GemSafe v2 applet. The stored SE becomes the current SE.

Conditions of Use and Security

Usage

The command can be used only during the application phase, that is the GemSafe v2 applet must be in the life cycle state PERSONALIZED.

Security

In the application phase, the command can be used before or after mutual authentication. Secure messaging is optional. For details on the command format to use with secure messaging see “Secure Messaging During the Application Phase” on page 43.

Note: For clarity, the command is described here with the data in plain text.

Format

This command is formatted as follows:

CLA	INS	P1	P2
CLA	22h	73h	P2

Where

CLA 00h transmit normally

0Ch transmit with secure messaging

P2 is the SEID to be restored.

Note: The format of this command has been designed for backward compatibility.

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
69h	82h	Security status not satisfied, error during secure messaging
6Ah	80h	Incorrect Lpv, Lcg or Le when using secure messaging in application phase
6Ah	86h	Incorrect P1, P2

MSE–SET

The **MSE–Set** command updates a CRT in the current SE. The value of the CRT in the original SE remains unchanged.

Conditions of Use and Security

Usage

The command is also available in the personalization phase.

The command applies to the current SE only.

Security

In the application phase, the command can be used before or after mutual authentication. Secure messaging is optional. For details on the command format to use with secure messaging see “Secure Messaging During the Application Phase” on page 43.

Note: For clarity, the command is described here with the data in plain text.

Format

This command is formatted as follows:

CLA	INS	P1	P2	Lc	Data
CLA	22h	41h	P2	Lc	Data

Where

CLA 00h transmit normally

04h transmit with secure messaging (personalization phase)

0Ch transmit with secure messaging (application phase)

P2 is the tag of the CRT to be updated as follows:

A4h authentication template AT

A6h key agreement template KAT

B4h cryptographic checksum template CCT

B6h digital signature template, DST

B8h confidentiality template, CT

Lc is the length of the data field. This value cannot exceed 30 bytes.

Data is the concatenation of the control reference data objects (CRDOs) to be updated. This cannot exceed 30 bytes.

CRDOs are the items contained in the CRT. The data does not contain the tag and length of the DST or CT.

Coding Example

In this example, the current SE contains a digital signature template with an algorithm ID and a key reference. The command is going to change the value of the key reference but leave the value of the algorithm ID unchanged and is to be sent without secure messaging. The command APDU is as follows:

CLA	INS	P1	P2	Lc	Data
00h	22h	41h	B6h	06h	Data

Where:

Data is coded as follows:

80h is the tag of the algorithm ID

01h is the length of the algorithm ID

11h is the value of the algorithm ID (meaning SHA–1 hash; RSA padding according to ISO 9796–2)

84h is the tag of the key reference

01h is the length of the key reference

02h is the value of the key reference (Key ID #2 in this example)

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
67h	00h	Incorrect length. Lc > 30
69h	82h	Security status not satisfied, error during secure messaging
6Ah	80h	Incorrect Lpv, Lcg or Le when using secure messaging in application phase
6Ah	86h	Incorrect P1, P2

MUTUAL AUTHENTICATE

This command enables the card and the terminal to authenticate each other, by verifying the presence of the two secret keys, K_{ENC} and K_{MAC}. For added security, the values of K_{ENC} and K_{MAC} stored in the card can be diversified.

Command Processing

1. The terminal sends the following data to the card in the **Mutual Authenticate** command:
   - Encrypted message The encrypted message S′ is $3DES_{CBC}$ [S]<K_{ENC}>
   - MAC A MAC computation of the encrypted message S′ using the retail MAC algorithm and the K_{MAC} key.

   Where:

S	= RND.IFD || SN.IFD || RND.ICC || SN.ICC || TRnd
RND.IFD	is an eight–byte challenge generated by the terminal
SN.IFD	is the terminal serial number
RND.ICC	is an eight–byte challenge generated by the card
SN.ICC	is the chip serial number
TRnd	is a 32–byte random number generated by the terminal.

2. The card verifies the MAC, decrypts the message and verifies SN.ICC and RND.ICC. It also generates the session keys K_{SK}ENC and K_{SK}MAC to be used in secure messaging.
3. The card uses its own K_{ENC} key to encrypt the same data as the terminal but in a different order. This is SS′. It performs a MAC computation on this encrypted data using its own K_{MAC} key. It returns the encrypted data and the MAC to the terminal.
4. The card verifies the MAC, then decrypts the data and verifies it. If the verification is successful, the terminal and the card both know that the other possesses the same K_{ENC} and K_{MAC} keys, and therefore each has authenticated the other.

For more details about these steps and parameters, refer to “Chapter 8 - Mutual Authentication”.

Conditions of Use and Security

Usage

The command is available only in the application phase, that is when the GemSafe v2 applet is in the life cycle state PERSONALIZED.

A **Get Challenge** command must have been issued prior to the **Mutual Authenticate** command in order to obtain the value RND.ICC.

The **Mutual Authenticate** command cannot be issued if an authentication session is already in progress.

Security

The authentication ratification counter is decremented each time that the **Mutual Authenticate** command fails. When the command is successful, the counter is reset to its initial value. The initial value of the authentication ratification counter is set with the **Install** command, see Table 18 on page 25.

The command does not use secure messaging, since this is the command that performs mutual authentication.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data	Le
80h	82h	00h	00h	00h or 48h	Data	48h

Where:

Lc — is the length of the data (48h = 72 bytes) or 00h. Use 00h to find out if the **Mutual Authenticate** command has already been successfully run. If it has, the card returns 90h 00h – if it has not, the card returns 63h CXh where X is the number of remaining attempts allowed, or 63h 00h if the ratification counter is set to A5, that is no limit to the number of tries allowed.

Data — is S′ ‖ MAC

Le — is the length of the response (always 72 bytes (48h))

Response

Response	SW1	SW2

Where:

Response is SS′ ‖ MAC. The Response is followed by the SW1, SW2 status codes. Possible status code values are:

SW1	SW2	Description
90h	00h	Command processed without error
63h	00h	Authentication failed—further attempts allowed (no limit)
63h	Cxh	Authentication failed—”x” attempts remaining
67h	00h	Incorrect length
69h	85h	Command conditions not satisfied
6Ah	80h	Data not found
6Ah	86h	Incorrect P1, P2

PSO–COMPUTE DIGITAL SIGNATURE

The **PSO–Compute Digital Signature** command is in fact the **Perform Security Operation (PSO)** command used to compute a digital signature (from either a message or a hash computation). Since the **PSO** command has different uses, each use is described in this reference manual as though it were a separate command.

A message is always hashed before signing. This hash can be performed by GemSafe v2, externally, or both. In the last case, the data is hashed externally but the last block is hashed by the card. In all cases, GemSafe v2 pads the hash to create digital signature input (DSI) and signs this DSI by using the **PSO–Compute Digital Signature** command. For a more detailed explanation, refer to “Performing a Digital Signature” on page 14.

The current SE indicates the keys and algorithms to use for the signature and, if applicable, the hash algorithm. For more details see “Security Environments” on page 47, and in particular “CRT” on page 49.

Conditions of Use and Security

Usage

The command is also available in the personalization phase.

The command must be preceded by a **PSO–Hash** command which provides the hash or sets the hashed data if the hash is performed entirely externally.

Note: For applications to be CC–compliant, there must not be any commands in between the **PSO–Hash** and the **PSO–Compute Digital Signature**.

None of the following actions must have taken place in between the execution of the **PSO–Hash** command and the **PSO–Compute Digital Signature** command otherwise the hash data is lost:

- Select GemSafe v2 application
- Another completed hash computation
- An **MSE–SET** or **MSE–Restore** command

The current SE must contain a DST which refers to a private key. This key must be a signature key and must have been initialized.

Caution: The key modulus value must be greater than the value of the padded message to be signed. Remember that messages padded with ISO 9796–2 padding begin with a padding byte of 6Ah, so any modulus value beginning with a value less than 6Ah is not allowed. In such a case, the **PSO–Compute Digital Signature** does not return an error code in its status bytes, but generates a signature that has no meaning.

Security

In the application phase any security attributes set for the private key regarding the **PSO–Compute Digital Signature** must be fulfilled. Refer to “Access Mode Byte”—“For Data Objects” on page 34 for more details. The security attributes are not checked in the personalization phase.

For details on the command format to use with secure messaging see “Secure Messaging During the Application Phase” on page 43.

Note: For clarity, the command is described here with the data in plain text.

For compatibility with other GemSafe applets, and since the instruction byte is the same for all PSO commands, when using secure messaging the structure is the same as for a case 4 command. Consequently, for the **PSO–Compute Digital Signature** command which has no input data, the length of the input data must be zero. This gives: TPv=81h || LPv = 00h || Tcc = 8Eh || Lc = 08h || MAC. (Pv is absent.)

If the signature key is protected by one or more PINs, a correct PIN presentation is valid for one **PSO–Compute Digital Signature** command only. To use the command again, present the correct PINs once more.

If the signature key is protected by one or more PINs, and the “Change PIN before first use” option is active, see Table 19, “Application Options Byte Coding”, on page 25, then these PINs must have been changed at least once since personalization for the signature key to be available. In other words, the “changed” flag for the PIN must be true.

Format

This command is formatted as follows:

CLA	INS	P1	P2	Lc	Data	Le
CLA	2Ah	9Eh	9Ah	Empty	Empty	Le

Where

CLA 00h transmit normally

04h transmit with secure messaging (personalization phase)

0Ch transmit with secure messaging (application phase)

Le is the expected length of the response, that is, the length of the signature with no tag or length bytes.

Note: If the response is greater than 256 bytes including secure messaging (for example with key lengths of 2,048 bits), the card opens a retrieval sequence. The sequence differs according to the communication protocol in use (T=0 or T=1). For details see “Retrieval Sequence for Large Outgoing Data” on page 70.

Response

Response	SW1	SW2

The card returns the value of the digital signature, followed by the SW1, SW2 status codes. The digital signature is not in TLV format.

SW1	SW2	Description
90h	00h	Command processed without error
67h	00h	Incorrect length, Lc. Lc must be empty
68h	84h	The chaining mechanism is not available for the command.
69h	82h	Security conditions not satisfied, for example: • Security attribute not fulfilled for private key • Error during secure messaging • PIN protecting the signature key has not been presented (it is valid for one signature only) • PIN protecting the signature key has not been changed since personalization, and the “Change PIN before first use” option is active. Table 19 on page 25
69h	85h	Conditions of use not satisfied, for example: • No hashed message available from **PSO–Hash** command • No algorithm or private key in the current SE’s DST • Algorithm not recognized by GemSafe v2 • Private key in current SE’s DST is not a digital signature key, that is, bit 1 of the private key usage byte is not set to 1–see Table 66 on page 110 • Private key in current SE’s DST has not been initialized, that is, at least one of the elements has not been initialized using the **Put Data** command
6Ah	86h	Incorrect P1, P2. Must be 9Eh 9Ah

PSO–DECIPHER

This command decrypts a message using a utility key stored in the card (in the GemSafe v2 applet). It returns the message in plain data.

The **PSO–Decipher** command can be used in “chaining” mode. This means that if the data is too large to be sent in a single **PSO–Decipher** command you can send it in two **PSO–Decipher** commands. The first command is sent with CLA = 1Xh. This tells the card to wait for the remaining data. Then issue another **PSO–Decipher** command with CLA= 0Xh. The card concatenates the data from the first command with that of the second and then performs the deciphering operation. You will need to use chaining mode to decipher data that was encrypted using a 2,048–bit key.

When chaining data and using secure messaging, the MAC is computed on the whole of the command header and data field as if it were one command APDU.

Conditions of Use and Security

Usage

The command is also available in the personalization phase.

The private key used to decipher the data must be a utility key and must be stored in the confidentiality template (CT - tag B8h) of the current SE. The encrypted message must be the same length as this key.

If you send the command with CLA = 1Xh to indicate chaining mode, the next command you issue must be another **PSO–Decipher** command with CLA = 0Xh, otherwise the chaining mechanism ends and the data is lost.

The total amount of data that can be sent in chaining” mode is 300 bytes. If this figure is exceeded, the chaining session is broken and the card returns the error 6700h.

The message to be encrypted must have been encrypted using PKCS #1 v2.x padding with block type 2 and be in the format shown in Table 88 on page 166.

Security

If specified, the “Decipher” security attribute for the private key referenced in the CT of the current SE must be fulfilled. For details on the coding of this security attribute, See Access Mode Byte “For Data Objects” on page 34. The security attributes are not checked in the personalization phase.

For details on the command format to use with secure messaging see “Secure Messaging During the Application Phase” on page 43.

Note: For clarity, the command is described here with the data in plain text.

The algo ID must be 1Ah indicating RSA with any padding other than 00h.

Format

This command is formatted as follows:

CLA	INS	P1	P2	Lc	Data	Le
CLA	2Ah	80h	86h	Lc	Data	Le

Where

CLA 00h transmit final or only portion of data normally and execute the command

04h transmit final or only portion of data with secure messaging and execute the command (personalization phase)

0Ch transmit final or only portion of data with secure messaging and execute the command

10h transmit first portion of data in chaining mode normally and wait for final portion

14h transmit first portion of data in chaining mode with secure messaging and wait for final portion (personalization phase)

1Ch transmit first portion of data in chaining mode transmit last or only portion with secure messaging and wait for final portion

Lc is the length of the data portion in the command.

Data is 81h || encrypted message, where the encrypted message is in the following format:

Value	Description	Length (in bytes)
00h	Start byte	1
02h	Block Type (02 for encryption)	1
XXh XXh... XXh	Random (non–zero) padding string	Key length - message length - 3 (Result 8 bytes or more)
00h	Separator	1
XXh XXh... XXh	Message	Message length

Table 88 - PKCS #1 v2.x Format for Encrypting Data

Le is the length of the deciphered message.

Note: The total length of the encrypted message (not including the 81h byte and secure messaging) must be the same length as the key used to decipher it.

Response

Response	SW1	SW2

The card returns the message in plain text, followed by the SW1, SW2 status codes.

SW1	SW2	Description
90h	00h	Command processed without error
67h	00h	Incorrect length, Lc, for example: • Data is not the same length as the private key. • Lc not less than 248 bytes when using secure messaging in application phase. • Total data in chaining mode exceeds 300 bytes.
69h	82h	Security status not satisfied: • Security attribute not fulfilled for private key referenced in CT of current SE • Error during secure messaging
69h	85h	Conditions of use not satisfied, for example: • No algorithm or private key in the current SE's CT • Algorithm ID must be 1Ah • Private key in current SE's CT is not a utility key (that is, bit 2 of the private key usage byte is not set to 1) –see Table 66 on page 110 • Private key in current SE's CT has not been initialized, that is, not all the elements have been updated using the **Put Data** command
6Ah	80h	Incorrect data, for example: • The message before encryption and padding was longer than 512 bytes • The message before encryption and padding was not in the format described by Table 88 on page 166 • Incorrect tag or tag length (including TLVs used in secure messaging)
6Ah	86h	Incorrect P1, P2. Must be 80h 86h

PSO–HASH

This command provides the hashed message to be used as input for the computation of a digital signature. There are three different cases:

- The hash is performed entirely by GemSafe v2
- The hash is performed externally but GemSafe v2 performs the final round of hashing
- The hash is performed entirely outside the card and the **PSO–Hash** command is used to "set" the data in preparation for the **PSO–Compute Digital Signature** command

Conditions of Use and Security

Usage

The command is available in both the personalization phase and the application phase.

If the data is to be hashed entirely in the card and is more than 64 bytes, divide it into blocks of 64 bytes or less and perform the **PSO–Hash** command for each block. The final command has a different format (see "Format of Final Block" on page 169). If the data is 64 bytes or less, use the format described for the final command.

If a command other than **PSO–Hash** is sent, the hash session ends and previously hashed data is lost.

Security

The current SE must contain a DST template and the algo ID in this template must be:

- 11h or 12h if hashing is entirely or partially performed by the card
- 01h, 02h, 03h, 11h, 12h if hashing is performed entirely externally

For details on the command format to use with secure messaging see "Secure Messaging During the Application Phase" on page 43.

Gemplus strongly recommends that this command is issued at least with secure messaging in MAC mode. For CC compliancy this is a requirement.

Note: For clarity, the command is described here with the data in plain text.

Format

This command format varies according to where the hash is performed. Each case is shown separately.

Hashing Performed Entirely by the Card

If the data is more than 64 bytes, the command must be sent for each block. The format for all the blocks except the last one is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	2Ah	90h	80h	Lc	Data

Where:

CLA 00h transmit normally

04h transmit with secure messaging (perso phase)

0Ch transmit with secure messaging (application phase)

Lc is the length of the data to be hashed

Data is a block of data to be hashed (no more than 64 bytes)

Format of Final Block

The format for the final block or only block if the data is not more than 64 bytes is as follows:

CLA	INS	P1	P2	Lc	Data	Le (optional)
CLA	2Ah	90h	A0h	Lc	Data	Final Hash Value

Where:

CLA 00h transmit normally

04h transmit with secure messaging (perso phase)

0Ch transmit with secure messaging (application phase)

Lc is the length of the input template

Data is the final or only block in TLV format as follows: 80h || L || Final data block

Le is the length of the hash result. This is optional and is used if you want to return the final value of the hash in the command.

Where:

L is the length of the final data block

Hashing Performed Partially by the Card

In this case, all the data except the final block is hashed outside the card but the card hashes the final block. The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	2Ah	90h	A0h	Lc	Data

Where:

CLA 00h transmit normally

04h transmit with secure messaging (perso phase)

0Ch transmit with secure messaging (application phase)

Lc is the length of the input data template

Data is the input data template in the following format:

Tag	Length	Value
90h	1Ch	20–byte intermediate hash value || 8–byte counter (number of bits already hashed)
80h	L	Final data block

Table 89 - PSO–Hash Input Data (Hash Performed Partially by the Card)

Where:

L is the length of the final data block

Hashing Performed Entirely Externally

In this case, all the data is hashed outside the card. The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	2Ah	90h	A0h	Lc	Data

Where:

CLA 00h transmit normally

04h transmit with secure messaging (perso phase)

0Ch transmit with secure messaging (application phase)

Lc is the length of the input data template

Data is the input data template in the following format:

Tag	Length	Value
90h	L	Hash value

Table 90 - PSO–Hash–Input Data (Hash Performed Entirely Externally)

Where:

L is the length of the hash value according to the algo ID as follows:

Algo ID	Length of Data to be Hashed
01h or 02h	1–36 bytes
03h	16–36 bytes
11h or 12h	20 bytes

Table 91 - PSO–Hash–Length of Hash Input Data (External Hash)

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
64h	00h	Incorrect checksum for hash value
68h	84h	The chaining mechanism is not available for the command.
69h	82h	Error during secure messaging
69h	85h	Conditions of use not satisfied, for example: • Algo id is not 11h or 12h in DST template of current SE when hash is entirely or partially performed in the card • Length of input hash data not valid for algo ID
6Ah	80h	Incorrect data. Input template in incorrect format

PSO–VERIFY CERTIFICATE

This command is used as part of the asymmetric key mutual authentication process to send the terminal's certificate, C_CV.IFD.AUT to the card. This serves two main purposes:

- It enables the card to verify the certificate, thus proving that the certificate originated from the CA
- It enables the card to retrieve certain data as follows:
  - The terminal's public key for authentication, PK.IFD.AUT, which is used later to authenticate the terminal with an **External Authenticate** command
  - The CHA (which includes the role ID), which is used later by the application as part of the security attributes.

The card must contain the CA's public key, PK.CA.AUT, in order to verify the certificate. This key must be referenced in the current SE.

The format of C_CV.IFD.AUT is defined by GemSafe v2 and must be strictly followed in order for the card to interpret the certificate correctly. The card checks the structure of the certificate and if correct, verifies its contents.

The details of the C_CV.IFD.AUT format are given in "Appendix B - Format of Authentication Certificates".

Conditions of Use and Security

Usage

The command can be used only during the application phase, that is the GemSafe v2 applet must be in the life cycle state PERSONALIZED.

The **PSO–Verify Certificate** is used as part of the asymmetric key mutual authentication process and cannot be used once mutual authentication is successful.

Security

The **PSO–Verify Certificate** does not use secure messaging.

Format

This command is formatted as follows:

CLA	INS	P1	P2	Lc	Data
00h	2Ah	00h	BEh	D1h	Data

Where

Lc is the length of the C_CV.IFD.AUT certificate

Data is the C_CV.IFD.AUT certificate in the format described in "Appendix B - Format of Authentication Certificates"

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
68h	84h	The chaining mechanism is not available for the command.
67h	00h	Incorrect length, Lc
69h	82h	Security conditions not satisfied, for example: • Keys are corrupted • The verification of the hash extracted from the certificate has failed. • CPI and/or OID in C_CV.IFD.AUT is incorrect
69h	85h	Conditions of use not satisfied, for example: • No public key in the current SE's AT • Key referenced in SE's AT is not a public key, or is not an authentication key (that is, the public key usage byte is not 01h – see Table 66, "Example Data Field Structure for Put Data–Private Key Creation", on page 110) or does not exist.
6Ah	80h	Incorrect data. Certificate is not in format defined by GemSafe.
6Ah	86h	Incorrect P1, P2
6Dh	00h	Mutual authentication has already taken place

PUT DATA

This command creates the following data objects in the application phase:

- DES secret keys
- RSA private keys
- RSA public keys
- DH key exchange parameters

The conditions of use and data are different for each type of data object. For clarity, each case is described individually.

Note: The format of this command has been designed for backward compatibility.

PUT DATA–SECRET KEYS

The GemSafe v2 applet uses two 3DES secret keys, K_{ENC} and K_{MAC}. These are used in symmetric key mutual authentication (but not asymmetric key mutual authentication), so must be known by the terminal and must be unique for each card.

The keys must be created and initialized during the personalization phase. In the application phase, the **Put Data** command can be used only to update the value of the keys. For more information about the creation and initialization of secret keys, see “Put Data–Secret Keys” on page 106.

This command is used to create these 3DES secret keys. It can be used in chaining mode, that is, the data can be split over two **Put Data** commands.

When chaining data and using secure messaging, the MAC is computed on the whole of the command header and data field as if it were one command APDU.

Conditions of Use and Security

Usage

In the application phase, the **Put Data** command can be used only to update the key values (as the keys have already been created).

The values of both keys must always be specified, it is not possible to update one of the keys only.

Security

The security attributes for the secret key must be satisfied to update the value of the secret key. GemSafe v2’s built–in security policy means that the command must always be sent with secure messaging in MAC and Encryption mode. For more information refer to “Security Attributes” on page 33, and Table 26, “Built-in Minimum Levels of Secure Messaging”, on page 37.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	DBh	00h	FFh	Lc	Data

Where:

CLA 0Ch transmit final or only portion of data with SM and execute the command

1Ch transmit first portion of data in chaining mode with SM and wait for final portion

Lc indicates the length of the data field

Data is coded in TLV format as follows:

Tag	Length	Value	Description	No. of Bytes
A4h	03h		Key Usage: Authentication Template	2
		83 01 xxh	*where*: xx = key id	3
DF23h	10h	K_{ENC}	Value of K_{ENC}	19
DF24h	10h	K_{MAC}	Value of K_{MAC}	19
TOTAL BYTES				**43**

Table 92 - Data Field Structure for Put Data–Secret Key Update

In this example, the data is 43 bytes, so the value of Lc in the **Put Data** command is 2Bh.

Response

The responses for the four **Put Data** commands are grouped together. See “Response” on page 182.

PUT DATA–PRIVATE KEYS

A private key contains five CRT elements, p, q, dp, dq, and iq. For details of these elements, see “Private Keys” on page 10. A private key is created during the personalization phase by six **Put Data** commands—one to create the key itself and five to initialize the data for each element. The key and each element are identified by their individual tags. For information about using the command in the personalization phase, see “Put Data–Private Keys” on page 108.

In the application phase, the **Put Data** command can be used to update the value of one or more elements. Each element requires a separate **Put Data** command.

This command can be used in chaining mode, that is, the data can be split over two **Put Data** commands.

When chaining data and using secure messaging, the MAC is computed on the whole of the command header and data field as if it were one command APDU.

Conditions of Use and Security

Usage

No more than N private keys can be created in the GemSafe v2 applet, where N is a parameter specified at the time of applet installation by the **Install** command in Install mode, see Table 18, “Option Parameters”, on page 25.

In the application phase, the **Put Data** command can be used only to update the elements (as the key has already been created).

Security

The security attributes for the private key must be satisfied to update the value of the private key. GemSafe v2’s built–in security policy means that the command must always be sent with secure messaging in MAC and Encryption mode. For more information refer to “Security Attributes” on page 33, and Table 26, “Built-in Minimum Levels of Secure Messaging”, on page 37.

For details on the command format to use with secure messaging see “Secure Messaging During the Application Phase” on page 43.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	DBh	00h	FFh	Lc	Data

Where:

CLA 0Ch transmit final or only portion of data with secure messaging and execute the command

1Ch transmit first portion of data in chaining mode transmit last or only portion with secure messaging and wait for final portion

Lc indicates the length of the data field

Data as described in “Data for Private Keys”.

Data for Private Keys

The data field is coded in TLV format.

Example Data Field for Updating a Private Element (1,024–bit keys):

Tag	Length	Value			Description	No. of Bytes
A4h or B6h	03h				AT (authentication key SK.ICC.AUT) DST (digital signature and/or utility key)	2
		84h	01h	XXh	TLV of private key ID (XXh is key ID value)	3
7Fh 48h	XXh				Tag and length of private elements	3
		Tag	**Length**	**Value**		
		92h–96h			Tag of private key element (p, q, iq, dp, or dq) (for values see “Data for Private Keys” on page 178).	1
			40h		Length of element (all five elements are 64 bytes for 1,024–bit keys, 40h)	1
				Value	Value of element	64
					TOTAL BYTES	**74**

Table 93 - Example Data Field Structure for Put Data–Writing Private Key Elements

In this example, one element only is updated, making a total of 74 bytes, so the value of Lc in the **Put Data** command is 4Ah. Add 66 bytes for each additional element (one for the tag, one for the length, 64 for the value).

More than one element can be updated in the same **Put Data** command. If the total exceeds FFh bytes in length, the command must be used in chaining mode.

Response

The responses for the four **Put Data** commands are grouped together. See “Response” on page 182.

PUT DATA–PUBLIC KEYS

RSA public keys are created during the personalization phase. For details on public key creation, see “Put Data–Public Keys” on page 112.

In the application phase, the **Put Data** command can be used to update the value of one or more elements.

When chaining data and using secure messaging, the MAC is computed on the whole of the command header and data field as if it were one command APDU.

Conditions of Use and Security

Usage

The commands to initialize the modulus, N and the public exponent, e can be in any order.

Security

The security attributes for the public key must be satisfied to update the value of the public key. GemSafe v2’s built–in security policy means that the command must always be sent with secure messaging in at least MAC mode. For more information refer to “Security Attributes” on page 33, and Table 26, “Built-in Minimum Levels of Secure Messaging”, on page 37.

For details on the command format to use with secure messaging see “Secure Messaging During the Application Phase” on page 43.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	DBh	00h	FFh	Lc	Data

Where:

CLA 0Ch transmit final or only portion of data with secure messaging and execute the command

1Ch transmit first portion of data in chaining mode transmit last or only portion with secure messaging and wait for final portion

Lc indicates the length of the data field as follows:

Data as described in “Data for Public Keys”

Data for Public Keys

The data field is coded in TLV format. Examples are given for each **Put Data** command.

Data Field Example for Writing Exponent Element:

Tag	Length	Value			Description	No. of Bytes
A4h or B6	03h				Tag and length of AT or DST Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	06h				Tag and length of public key template	3
		Tag	**Length**	**Value**		
		82h	04h	Prime	TLV of exponent (value coded on 4 bytes)	6
					TOTAL BYTES	**14**

Table 94 - Example Data Field Structure for Put Data–Writing Exponent Value

In this example, the data is 14 bytes, so the value of Lc in the **Put Data** command is 0Eh. The exponent can be any value but must be a prime number. The number cannot exceed 8 bytes in length.

Data Field Structure for Writing Modulus Element:

Tag	Length	Value			Description	No. of Bytes
A4h or B6	03h				Tag and length of AT or DST Template	2
		83h	01h	KeyID	TLV of public key	3
7Fh 49h	L				Tag and length of public key template	3
		Tag	**Length**	**Value**		
		81h	81h 80h	var	TLV of modulus (value coded on 128 bytes)	131
					TOTAL BYTES	**139**

Table 95 - Data Field Structure for Put Data–Writing Modulus Value

In this example for a 1,024–bit key, the data is 139 bytes, so the value of Lc in the **Put Data** command is 8Bh.

Response

The responses for the four **Put Data** commands are grouped together. See “Response” on page 182.

PUT DATA–DH KEY EXCHANGE PARAMETERS

The Diffie–Hellman key exchange parameters are created during the personalization phase. For details on their creation, see “Put Data–DH Key Exchange Parameters” on page 115.

In the application phase, the **Put Data** command can be used to update the value of one or more elements.

When chaining data and using secure messaging, the MAC is computed on the whole of the command header and data field as if it were one command APDU.

Conditions of Use and Security

Usage

The exchange parameters can be initialized in any order.

Security

The security attributes for the DH key exchange parameters data object must be satisfied to update the value of the parameters. GemSafe v2’s built–in security policy means that the command must always be sent with secure messaging in at least MAC mode. For more information refer to “Security Attributes” on page 33, and Table 26, “Built-in Minimum Levels of Secure Messaging”, on page 37.

For details on the command format to use with secure messaging see “Secure Messaging During the Application Phase” on page 43.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	DBh	00h	FFh	Lc	Data

Where:

CLA 0Ch transmit final or only portion of data with secure messaging and execute the command

1Ch transmit first portion of data in chaining mode transmit last or only portion with secure messaging and wait for final portion

Lc indicates the length of the data field as follows:

Data as described in “Data for DH Key Exchange Parameters”

Data for DH Key Exchange Parameters

The data field is coded in TLV format. Examples are given for each **Put Data** command.

Data Field Structure for Writing DH Key Exchange Parameters

Tag	Length	Value			Description	No. of Bytes
A6h	03h				Tag and length of KAT Template	2
		83h	01h	KeyID	TLV of Key ID for DH key exchange parameters object	3
7Fh 49h	var				Tag and length of DH key exchange parameters template	3
		Tag	**Length**	**Value**		
		86h 87h 88h	81h 80h 20h 81h 80h	var	TLV of DH:p element TLV of DH:q element TLV of DH:g element	
					TOTAL BYTES	

Table 96 - Data Field Structure for Put Data–Writing DH Key Exchange Parameters

The three DH parameters can be combined in one command as shown or sent as three separate **Put Data** commands. If they are combined, and the total length of the parameters exceeds FFh, the command must be sent in chaining mode.

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
67h	00h	Incorrect length, Lc. Lc must be less than 248 bytes when using secure messaging in application phase.
69h	82h	Security status not satisfied, for example: • The security attributes are not satisfied • Error during secure messaging
69h	85h	Conditions of use not satisfied, for example: • Public or private key does not exist (when writing an element) • Incorrect applet life cycle state (Command invalid for PINs or SEs or to create public or private keys in the applet life cycle state PERSONALIZED) • Incorrect AM byte in data field (only the **Put Data** access mode bit is allowed for a private key)
6Ah	80h	Incorrect data, for example • Incorrect tag or tag length • Incorrect Lpv, Lcg or Le when using SM in application phase
6Ah	86h	Incorrect P1, P2
6Ah	88h	Reference data not found (Tag specified in P1, P2)

READ BINARY

This command reads all or part of an EF.

There are two ways of referencing the EF:

- Implicit selection. In this case, the EF is under the current DF, and is specified by its short file identifier (SFI). P1 specifies the SFI and P2 specifies the start address for the read (called the offset). The command cannot be used for files with an SFI of 1Fh.
- Direct selection. In this case the EF must have been previously selected using the **Select File** command. The data is read directly from the current EF. In this case the offset is specified over P1 and P2.

The command reads the number of bytes specified in Le from the offset, up to a maximum of 255 bytes (including secure messaging data if used).

Conditions of Use and Security

Usage

The command is available only in the application phase, that is when the GemSafe v2 applet is in the life cycle state PERSONALIZED.

The command can be used on any activated EF. It is not valid for deactivated EFs, for DFs or for data objects.

In the case of implicit selection, the EF becomes the current EF after execution of the command.

Security

If the EF containing the data to be read is protected by a security attribute, then this security attribute must be fulfilled in order to perform the **Read Binary** command. For more details see “Security Attributes” on page 33.

During the application phase, use of the command depends on the security attribute. The security attribute specifies if prior mutual authentication is necessary, and if so if secure messaging is also necessary.

For details on the command format to use with secure messaging see “Secure Messaging During the Application Phase” on page 43.

Note: For clarity, the command is described with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Le
CLA	B0h	P1	P2	Le

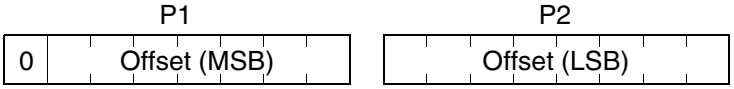
Where:

CLA 00h transmit without secure messaging

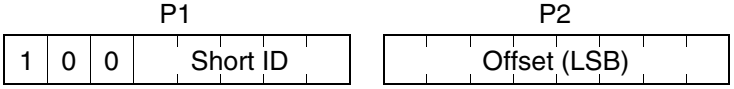
0Ch transmit with secure messaging

P1, P2 are the parameter bytes.

To read from the currently selected EF, P1 and P2 specify the offset, in bytes, from the beginning of the file to the first byte to read. P1 and P2 have the following format:



To select and read from an EF, P1 specifies the SFI of the file containing the data to be read. P2 specifies the offset, in bytes, from the beginning of the file to the first byte to read. P1 and P2 have the following format:



Le indicates the number of bytes to read. If empty or zero, the command reads until the end of the file, up to a maximum of 255 bytes (including data for secure messaging if SM specified).

Response

Data	SW1	SW2

Data is the data read from the card. It is followed by the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
62h	82h	End of file reached (offset + Le is higher than the end of the file)
64h	00h	Execution error (file descriptor checksum error)
67h	00h	Incorrect length, Lc (if using secure messaging)
69h	82h	Security status not satisfied, for example: • The security attributes are not satisfied • Error during secure messaging
69h	86h	Command not allowed (no current EF)
6Ah	80h	Incorrect Lpv, Lcg or Le when using secure messaging in application phase
6Ah	81h	Function not supported: SM in ENC/MAC mode specified for the response and command was issued with no SM.
6Ah	82h	File not found (b8 of P1 = 1 indicating implicit selection, but no SFI given)
6Ah	86h	Incorrect P1, P2 (check that the offset is inside the EF and that the SFI is not 1Fh)
6Bh	00h	Incorrect P1, P2 (offset outside EF)

RESET RETRY COUNTER

The **Reset Retry Counter** command unblocks a PIN. PINs are blocked when the number of allowed consecutive incorrect attempts is exceeded. You can optionally specify a new value for the reference PIN. PINs are protected by a security attribute (the unblocking PIN reference in the PIN attributes stored in the PIN data object itself). This unblocking PIN reference indicates the PIN reference of the unblock PIN. If it is set to zero, the reference PIN can never be unblocked.

Note: The reference PIN is the PIN that is to be unblocked. The unblock PIN is the PIN that must be verified to grant the access rights to unblock the reference PIN.

If the **Reset Retry Counter** command is successfully executed, following a successful verification of the unblock PIN, the following actions take place:

- The reference PIN Try Counter is set to PIN Try Limit
- The reference PIN validated flag is set to false (because the PIN is not yet verified)
- The reference PIN unblocking counter is decremented by 1, unless its value is A5h (no limit to the number of times reference PIN can be unblocked) or 00h (already been unblocked maximum allowed number of times).
- The unblock PIN Try Counter is set to unblock PIN Try Limit
- The unblock PIN validated flag is set to false
- The unblock PIN usage counter is decremented by 1 unless its value is FFh (no limit to the number of times unblock PIN can be used) or 00h (already been used maximum allowed number of times).
- If a new value for the reference PIN is specified, the reference PIN value is updated with the value sent in the **Reset Retry Counter** command
- If a new value for the reference PIN is specified, the reference PIN changed flag is set to “false”

If the **Reset Retry Counter** command fails, the following actions take place:

- The unblock PIN Try Counter is decremented by 1
- The unblock PIN validated flag is set to false

For more information about the structure of PINs and their security attributes, see “PIN Management” on page 37.

Conditions of Use and Security

Usage

The command is available only in the application phase, that is when the GemSafe v2 applet is in the life cycle state PERSONALIZED.

Security

The PIN Try Counter and the Usage Counter for both the reference PIN and the unblock PIN must not be zero.

The use of secure messaging is determined by the security attribute of the PIN.

Note: For clarity, the command is described with the data in plain text. For information on how to send the command with secure messaging, refer to “Secure Messaging During the Application Phase” on page 43.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	2Ch	Mode	P2	Lc	Unblock PIN || New PIN (optional)

Where:

CLA	00h	Transmit normally
	0Ch	Transmit with secure messaging
Mode	00h	Reset retry counter and replace value of reference PIN
	01h	Reset retry counter only
P2	is the reference of the reference PIN (81h–8Fh)	
Lc	10h–20h	If replacement value for reference PIN (P1 = 00h)
	08h–10h	If no replacement value for reference PIN (P1 = 01h)
Data	If P1 = 00h	Unblock PIN || New PIN (both PINs are coded on 8–16 bytes)
	If P1 = 01h	Unblock PIN (8–16 bytes)

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
63h	Cxh	Unblock PIN not verified.”x” attempts remaining for unblock PIN
67h	00h	Incorrect length, Lc. Lc must be less than 248 bytes when using secure messaging
69h	82h	Security conditions not satisfied (error during secure messaging or security attribute for the reference PIN is never)
69h	83h	Authentication method blocked (the unblock PIN needed to unblock the reference PIN is blocked itself)
69h	84h	Referenced data invalidated (PIN Unblocking Counter for reference PIN or PIN Try Counter or PIN Usage Counter for unblock PIN has reached zero)
6Ah	80h	Incorrect Lpv, Lcg or Le when using secure messaging in application phase
6Ah	86h	Incorrect P1, P2
6Ah	88h	Referenced data not found

SELECT FILE

The **Select File** command selects a DF (either PKCS#15 ones or issuer ones) or an EF.

If P1, P2 and Lc are all zero, the command selects the root but returns no data. To select the root and return the FCI information, you must select it by its file ID (3F00h).

Conditions of Use and Security

Usage

The root and DFs can be selected by their file identifier, by their path or by their name, but not by partial name. EFs can be selected by their file identifier or by their path.

The **Select File** command is available during the personalization phase and the application phase. For information on how to use the command in the personalization phase, see “Select File” on page 121.

The command cannot be used to select a deactivated file.

Security

In the application phase, the command can be used before or after mutual authentication. Secure messaging is optional. For details on the command format to use with secure messaging see “Secure Messaging During the Application Phase” on page 43.

Note: For clarity, the command is described here with the data in plain text.

Format

This command is formatted as follows:

CLA	INS	P1	P2	Lc	Data	Le
CLA	A4h	Ref Ctrl	Ref Ctrl	Lc	Data	Le

Where:

CLA 00h Transmit normally

0ChTransmit with secure messaging

P1 is a reference control parameter. The bit settings are as follows:

P1	b8	b7	b6	b5	b4	b3	b2	b1
RFU	0	0	0	0	x	x	x	x
Select by path from root					1	0	0	0
Select DF or root by name					0	1	0	0
Select EF by file ID under current DF					0	0	1	0
Select root, DF or EF by File ID					0	0	0	0

Note: When P1 is 00h, (file selection by file ID), but not under the current DF, GemSafe v2 searches all the files in the GemSafe v2 applet in the order of creation. It selects the first file that matches the file ID specified in the data field, which might not be the file intended.

P2 is the reference control parameter that specifies the required response. The bit settings are as follows:

P2	b8	b7	b6	b5	b4	b3	b2	b1
RFU	0	0	0	0				
FCI option					x	x		
Return FCI template					0	0		
Return FCP template					0	1		
No response					1	1		
First or only file							0	0

Lc is the length of the data field as follows:

02h for file selection by ID

01h–10h for selection by DF or root name

02h or 04h for file selection by path

Data is the file reference and is one of the following:

file ID is the file identifier coded in the file descriptor see Table 2, “FCP for DFs”, on page 6.

DF name is the name of the DF coded in the file descriptor Table 2, “FCP for DFs”, on page 6.

path this is the concatenation of the file IDs from the root or current DF to the file ID of the file to be selected, not including the file ID of the current DF or root. For more detailed information about paths, see “Referencing by Path” on page 9.

Le is the length of the data returned, which is determined by the P2 setting.

Response

Response	SW1	SW2

The card returns the data requested by the P2 reference control parameter, followed by the SW1, SW2 status codes. The response data is either in the FCI template, the FCP template or does not exist (P2 = 0Ch).

The FCP template contains the file control parameters that were specified when creating the file. For DFs, this data is as shown in Table 51, “Example of DF Creation”, on page 78 but does not include the FDB (TLV= 82h 01h 38h). For EFs the data is as shown in Table 52, “Example of EF Creation”, on page 78.

The FCI information returned by the command (P2 = 00h) is shown in “Table 97” and “Table 98”.

Caution: When the command is sent with secure messaging, the tag (62h for FCP, 6Fh for FCI) and length of the FCP or FCI template are absent. Two trailer bytes with the value 00h are appended to the end of the FCP/FCI data. This is true for DF and EF selection.

For DFs:

Offset	Data	Description
0	6Fh	Tag of FCI template
1	L	Length of FCI data
2	83h	Tag of File ID
3	02h	Length of File ID
4–5	File ID	Value of File ID
6	8Ch	Tag of security attributes
7	L	Length of security attributes
8	AM	Access mode byte
9–(8+X)	SC	Security condition bytes (X)
9+X	84h	Tag of DF Name
10+X	L	Length of DF Name
11+X –	DF name	Value of DF name (up to 16 bytes)

Table 97 - FCI for DFs

For EFs

Offset	Data	Description
0	6Fh	Tag of FCI template
1	L	Length of FCI data
2	81h	Tag of File Size
3	02h	Length of File Size
4–5	File Size	Value of File Size.
6	82h	Tag of FDB
7	01h	Length of FDB
8	FDB	Value of FDB
9	83h	Tag of File ID
10	02h	Length of File ID
11–12	File ID	Value of File ID
13	8Ah	Tag of Life Cycle Status byte for file
14	01h	Length of Life Cycle Status byte for file
15	Var	Value of Life Cycle Status byte for file
16	8Ch	Tag of security attributes
17	L	Length of security attributes
18	AM	Access mode byte
19–(18+X)	SC	Security condition bytes (X)

Table 98 - FCI for EFs

Possible values for the SW1, SW2 status codes are as follows:

SW1	SW2	Description
90h	00h	Command processed without error. No FCI data returned
62h	83h	Deactivated file
64h	00h	Execution error (file descriptor checksum error)
67h	00h	Incorrect length, Lc. Lc must be less than 248 bytes when using secure messaging in application phase.
69h	82h	Security status not satisfied, error during secure messaging
6Ah	80h	Incorrect Lpv, Lcg or Le when using secure messaging in application phase
6Ah	82h	File not found
6Ah	86h	Incorrect P1, P2 parameter

UPDATE BINARY

This command updates all or part of an EF.

There are two ways of referencing the EF:

- Implicit selection. In this case, the EF is under the current DF, and is specified by its short file identifier (SFI). P1 specifies the SFI and P2 specifies the start address for the update (called the offset). The command cannot be used for files with an SFI of 1Fh.
- Direct selection. In this case the EF must have been previously selected using the **Select File** command. The data is updated directly in the current EF. In this case the offset is specified over P1 and P2.

The command updates the number of bytes specified in Lc from this offset.

Conditions of Use and Security

Usage

The **Update Binary** command is available during the personalization phase and the application phase. For details on its use in the personalization phase, see “Update Binary” on page 125.

The command can be used on any activated EF. It is not valid for deactivated EFs, for DFs or for data objects.

In the case of implicit selection, the EF becomes the current EF after execution of the command.

Security

If the EF containing the data to be updated is protected by a security attribute, then this security attribute must be fulfilled in order to perform the **Update Binary** command. For more details see “Security Attributes” on page 33.

During the application phase, use of the command depends on the security attribute. The security attribute specifies if prior mutual authentication is necessary, and if so if secure messaging is also necessary.

For details on the command format to use with secure messaging see “Secure Messaging During the Application Phase” on page 43.

Note: For clarity, the command is described here with the data in plain text.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	D6h	P1	P2	Lc	Data

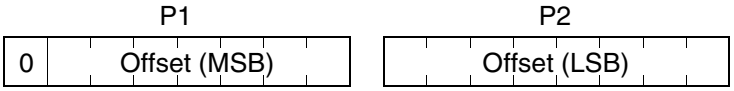
Where:

CLA 00h Transmit normally

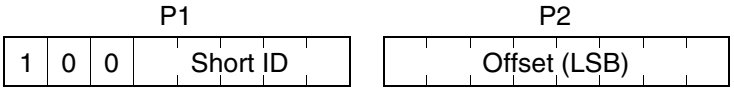
0Ch Transmit with secure messaging

P1, P2 are the parameter bytes.

To update the currently selected EF, P1 and P2 specify the offset, in bytes, from the beginning of the file to the first byte to be updated. P1 and P2 have the following format:



To select and update an EF, P1 specifies the SFI of the file containing the data to be updated. P2 specifies the offset, in bytes, from the beginning of the file to the first byte to be updated. P1 and P2 have the following format:



Lc indicates the length of the data field up to a maximum of 248 bytes

Data contains the value of the data to be updated in the card

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
62h	82h	End of file reached (offset + Lc is higher than the end of the file)
64h	00h	Execution error (file descriptor checksum error)
67h	00h	Incorrect length, Lc. Lc must be less than or equal to 248 bytes when using secure messaging in the application phase.
69h	82h	Security status not satisfied, for example: • The security attributes are not satisfied • Error during secure messaging
69h	86h	Command not allowed (no current EF)
6Ah	80h	Incorrect Lpv, Lcg or Le when using secure messaging in application phase
6Ah	82h	File not found (b8 of P1 = 1 indicating implicit selection, but no SFI given)
6Ah	86h	Incorrect P1, P2 (check that offset is inside the EF and SFI is not 1Fh)
6Bh	00h	Incorrect P1, P2 (offset outside EF)

VERIFY

The **Verify** command authenticates a user by comparing the PIN entered in the command (the verification PIN) with the reference PIN.

If the **Verify** command is successful, the following actions take place:

- The reference PIN Try Counter is set to PIN Try Limit
- The reference PIN validated flag is set to true
- The reference PIN usage counter is decremented by 1 unless its value is FFh (no limit to the number of times the PIN can be used) or 00h (already been used maximum allowed number of times).

If the **Verify** command fails, the following actions take place:

- The reference PIN Try Counter is decremented by 1
- The reference PIN validated flag is set to false

For more information about the structure of PINs and their security attributes, see “PIN Management” on page 37.

Conditions of Use and Security

Usage

The command is available only in the application phase, that is when the GemSafe v2 applet is in the life cycle state PERSONALIZED.

The length of the entered PIN must be equal to length of the PIN in the card (L_{PIN}), otherwise the **Verify** command fails.

Sending the command with Lc = 00h has a special meaning. It enables you to know if the PIN has already been successfully presented during the current session. If the PIN has been successfully presented already, then the **Verify** command returns a status code of 9000h. If the PIN has not been successfully presented already, the **Verify** command returns a status code of 63 Cxh, where x is the number of remaining presentation attempts allowed.

Security

The PIN Try Counter and the Usage Counter for the reference PIN must not be zero.

The use of secure messaging is determined by the security attribute of the PIN. If secure messaging is specified, the verification PIN provided by the terminal must be of the same length as the reference PIN and its value must match.

Note: For clarity, the command is described with the data in plain text. For information on how to send the command with secure messaging, refer to “Secure Messaging During the Application Phase” on page 43.

Format

The command format is as follows:

CLA	INS	P1	P2	Lc	Data
CLA	20h	00h	P2	Lc	Verification PIN

Where:

CLA	00h	Transmit normally
	0Ch	Transmit with secure messaging
P2	is the reference of the reference PIN (81h–8Fh)	
Lc	00h	To find out if PIN has already been successfully presented.
	08h–10h	Must be equal to L_{PIN}
Data	verification PIN value (8–16 bytes)	

Response

The card returns the SW1, SW2 status codes. Possible status code values are as follows:

SW1	SW2	Description
90h	00h	Command processed without error
63h	Cxh	Reference PIN not verified.”x” attempts remaining
67h	00h	Incorrect length, Lc. Lc must be less than 248 bytes when using secure messaging.
69h	82h	Security conditions not satisfied (error during secure messaging)
69h	83h	Authentication method blocked (reference PIN blocked)
69h	84h	PIN Try Counter or PIN Usage Counter for reference PIN has reached zero
6Ah	80h	Incorrect Lpv, Lcg or Le when using secure messaging in application phase
6Ah	86h	Incorrect P1, P2
6Ah	88h	Referenced data not found

A

Status Bytes

SW1	SW2	Description
62h	82h	End of file reached
62h	83h	Deactivated file
63h	00h	Authentication process blocked—too many failed attempts
63h	Cxh	Authentication failed—”x” attempts remaining
64h	00h	Execution error, such as corrupt data, EEPROM checksum verification error.
67h	00h	Incorrect length. For example: • Incorrect Lc (Lc must be less than 248 bytes when using secure messaging in application phase) • New PIN value is not same length as previous PIN for a **Change Reference Data** command.
68h	00h	Instruction byte is not compatible with class byte
68h	84h	The chaining mechanism is not available for the command.
69h	82h	Security status not satisfied, for example: • The security attributes are not satisfied • Error during secure messaging • MAC check failed • The verification of the hash extracted from the certificate has failed. **PSO–Verify Certificate** command • CPI and/or OID in the certificate C_CV.IFD.AUT is incorrect in the data field for the **PSO–Verify Certificate** command • PIN protecting the signature key has not been changed since personalization, and the “Change PIN before first use” option is active.
69h	83h	PIN blocked (either change, reference or unblock PIN)
69h	84h	Referenced data invalidated (PIN Try Counter or PIN Usage Counter for either the reference PIN or the change PIN has reached zero)

Table 99 - Response Status Bytes

SW1	SW2	Description
69h	85h	Conditions of use not satisfied, for example: • No previous **Initialize Update** command • Incorrect applet life cycle state • Incorrect AM byte in data field (**Create File** command) • Incorrect tag or length (**Create File** command) • Attempt to create a PIN with a PIN ID that already exists (**Put Data** command) • PIN to be created is protected by a PIN that has not yet been created (**Put Data** command) • Instruction byte is not allowed for current DF or application • Algo id is not 11h or 12h in DST template of current SE when hash is entirely or partially performed in the card (**PSO–Hash** command) • Length of input hash data not valid for algo ID
69h	86h	Command not allowed, for example, no current EF
6Ah	80h	Incorrect data, for example: • Incorrect tag or tag length (including TLVs used in secure messaging) • Key usage value does not correspond to key type • Incorrect key length • Referenced file not found • C_CV.IFD.AUT is not in correct format (**PSO–Verify Certificate** command) • Incorrect LPV, LCG or LE when using secure messaging in application phase • Data not found
6Ah	81h	Function not supported, for example: • Command specifies current file, but no file currently selected • DF identifier specified for a command that cannot be used on DFs • SM in ENC/MAC mode specified for response in case 2 command and command issued with no SM.
6Ah	82h	File or data object not found
6Ah	84h	Insufficient memory. (Allowed maximum number of private keys exceeded).
6Ah	86h	Incorrect P1, P2
6Ah	88h	• Reference data not found • Tag specified in P1, P2 not found • No message DO in data field (**PSO–Compute Digital Signature** command)
6Bh	00h	Incorrect P1, P2, for example Offset outside EF
6Dh	00h	Instruction not supported, for example: • Incorrect instruction byte • Command not allowed for applet life cycle state • Mutual authentication has already taken place (**PSO–Verify Certificate** command)
6Eh	00h	Class not supported
90h	00h	Command processed without error. No FCI data is returned.

Table 99 - Response Status Bytes (continued)

B

Format of Authentication Certificates

C_CV.IFD.AUT

This is the certificate of the terminal used in asymmetric key mutual authentication and must be coded in the format shown in this appendix. The certificate tells the card (the GemSafe v2 applet) that the terminal is trusted by the CA and also gives the card the value of the terminal public key for authentication and the terminal's role ID.

Tag	Length	Tag	Length	Value	Description	No. of Bytes
7Fh 21h	81h CDh				Tag and length of C_CV.IFD.AUT	2+2=4
		5Fh 37h	81h 80h		Tag and length of signature	2+2=4
Signature (that is, this data encrypted using the CA's private key)				6Ah	Padding byte according to ISO 9796–2	01h
				0Dh	Value of the certificate profile identifier	01h
				"CAR"	Value of certification authority reference	08h
				"CHR"	Value of certificate holder reference (IFD)	0Ch
				"CHA"	Value of certificate holder authorization	07h
				2Bh 24h 07h 02h 01h 02h	Value of object identifier (OID)	06h
				"PK.IFD.AUT _part1"	Value of "PK.IFD.AUT_part1" (72 MSB of modulus)	48h
				"Hash"	Value of 20–byte hash	14h
				BCh	Padding byte according to ISO 9796–2	01h
		5Fh 38h	3Ch		Tag and length of public key remainder (remainder of modulus and exponent)	2+1=3
				"PK.IFD.AUT _part2"	Value of remainder of public key modulus (56 LSB)	38h
				00h 01h 00h 01h	Value of public key exponent	04h
		42h	08h	"CAR"	TLV of CAR	1+1+8=10

Table 100 - C_CV.IFD.AUT Format

The public key is too large to fit in the signature and so it is divided into two parts. The most significant part of the modulus (pk_part 1) is in the signature. The rest of the modulus (pk_part 2) and the exponent are stored in the public key remainder.

The format of this certificate is defined in *CEN/ISSS WS/E-Sign Draft CWA Group K—Application Interface for Smart Cards Used as Secure Signature Creation Devices—Part 1 -Basic Requirements*.

Certification Authority Reference (CAR)

This value is defined by the card manufacturer and must be listed separately because it is hidden by the signature.

Certificate Holder Reference

These 12 bytes are used to identify the PK.IFD.AUT. Its value is:

Filler (0–4 bytes) || SN.IFD (8–12 bytes)

Where the filler bytes have the value 00h.

Certificate Holder Authorization

This describes the access rights of the certificate holder for a given application. It is seven bytes and is structured as follows:

AID Prefix (6 bytes) || Role ID (1 byte)

Where:

AID is the application identifier

Role ID corresponds to the role of the terminal (IFD) in the application.

Hash

The 20–byte hash is computed as follows:

SHA–1 [0Dh || “CAR” || “CHR” || “CHA” || “OID” || PK.IFD.AUT_part 1 || PK.IFD.AUT_part 2 || 00h 01h 00h 01h]

These values are all contained in C_CV.IFD.AUT, see Table 100 on page 199.

C.ICC.AUT

This is the card’s certificate used in asymmetric key mutual authentication. The terminal reads this in step “13.” of the mutual authentication process in order to authenticate the card. For more details refer to page 62.

GemSafe v2 does not impose a format for this certificate but it must be able to be interpreted by the terminal and must contain at least PK.ICC.AUT (the card’s public key used for authentication) and the chip serial number.

C

Generating Session Keys with the Diffie–Hellman Algorithm

This appendix describes the generation and use of the data elements K_{IFD} and K_{ICC}, the shared secret data $K_{IFD/ICC}$ and the session keys K_{SK}ENC and K_{SK}MAC.

Deriving the Shared Secret Data, $K_{IFD/ICC}$

Here the terminal is denoted by A and the card, that it the GemSafe v2 applet is denoted by B. A and B both share the public Diffie–Hellman key exchange parameters p, g and q where:

p is the modulus – a 1,024–bit prime number

g is a generator number such that $g^p = 1 \bmod p$

q is a 160–bit prime number.

The principle is as follows:

1. A chooses a random number, x, such that 1 <= x <= q-1
   B chooses a random number, y, such that 1 <= y <= q-1

Where:

<= means “less than or equal to”

Note: x and y remain secret throughout the process.

2. B generates a data element, K_{ICC}. $K_{ICC} = g^y \bmod p$ and sends it to A
   A generates a data element, K_{IFD}. $K_{IFD} = g^x \bmod p$ and sends it to B
3. A generates the secret value $K_{IFD/ICC}$. $K_{IFD/ICC} = (K_{ICC})^x \bmod p$
   B generates the secret value $K_{ICC/IFD}$. $K_{ICC/IFD} = (K_{IFD})^y \bmod p$
4. A destroys the random number x
   B destroys the random number y

We have said that $K_{IFD/ICC} = (K_{ICC})^x \bmod p$

Substituting $K_{ICC} = g^y \bmod p$ gives $K_{IFD/ICC} = (g^y)^x \bmod p$

$(g^y)^x \bmod p = (g^x)^y \bmod p = (K_{IFD})^y \bmod p = K_{ICC/IFD}$

Therefore $K_{IFD/ICC} = K_{ICC/IFD}$

This means that both A and B have derived the same number from two random numbers x and y without ever exchanging x and y. $K_{IFD/ICC}$ is then shared secret data.

Deriving the Session Keys

The card (the GemSafe v2 applet) and the terminal both derive the session keys $K_{SK}ENC$ and $K_{SK}MAC$ from the shared secret data $K_{IFD/ICC}$

$K_{SK}ENC$

Hash = SHA-1[$K_{IFD/ICC}$ || 00h 00h 00h 01h]

This gives a result on 20 bytes.

MSB		LSB
Bytes 20-13	Bytes 12-5	Bytes 4-1
Ka	Kb	Not Used

$K_{SK}ENC$ = Ka || Kb

$K_{SK}MAC$

Hash = SHA-1[$K_{IFD/ICC}$ || 00h 00h 00h 02h]

This gives a result on 20 bytes.

MSB		LSB
Bytes 20-13	Bytes 12-5	Bytes 4-1
Ka	Kb	Not Used

$K_{SK}MAC$ = Ka || Kb

D

Triple DES Computations

The GemSafe v2 applet uses the triple DES (3DES) algorithm to perform various computations. This appendix shows the details of these computations for the different modes of 3DES.

3DES in ECB Mode

The triple DES algorithm is used in its electronic codebook (ECB) mode to compute Auth/Enc session keys in the personalization phase from keys with the algorithm reference 81h (see Table 29 on page 41).

The computation is shown in the following figure:

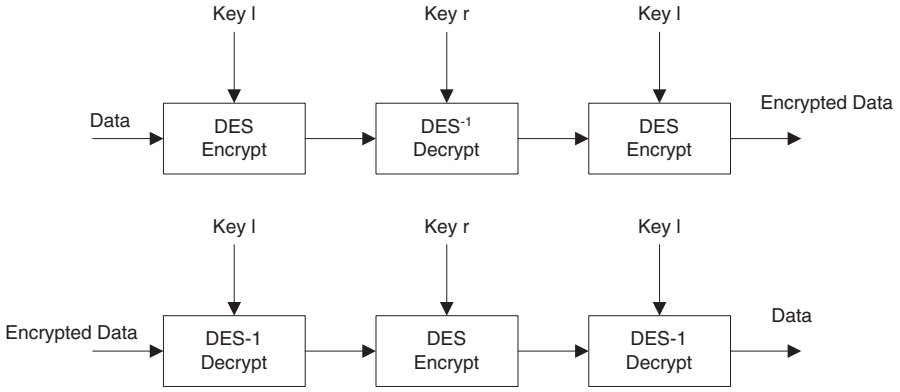


Figure 12 - 3DES Algorithm

Where:

Key r and Key l are the right and left half of the key used in the encryption, such that the whole key is Key r || Key l.

3DES in CBC Mode

The triple DES algorithm in cipher block chaining (CBC) mode is used to:

- Compute MAC session keys in the personalization phase from keys with the algorithm reference 82h (see Table 29 on page 41).
- Compute a MAC when sending a command with secure messaging in MAC mode during the personalization phase (see “Commands Sent in MAC Mode” on page 41).
- Encrypting data when sending a command with secure messaging in encryption mode during the personalization phase (see “Commands Sent in Encryption Mode” on page 42).
- Compute cryptograms that are used when performing commands using secure messaging during the application phase (see “Secure Messaging During the Application Phase” on page 43).

In cipher block chaining mode, the data is padded and separated into 8-byte blocks and encrypted one at a time. The result of each encrypted block is used as input to encrypt the following block. The following figure illustrates this mechanism:

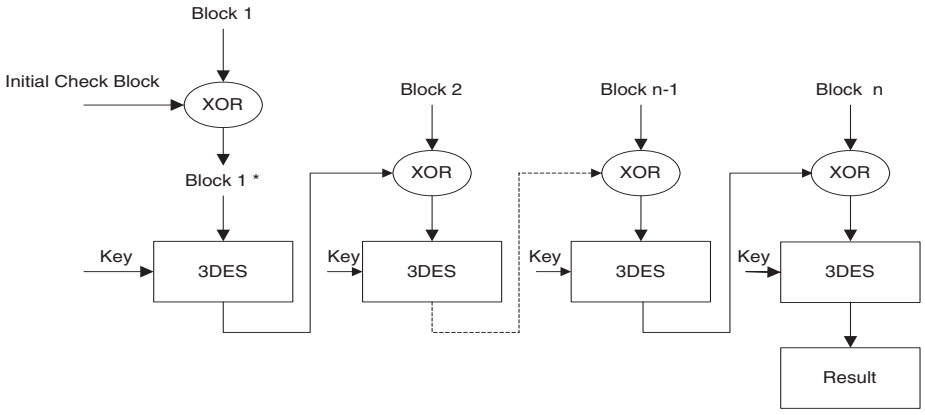


Figure 13 - 3DES in CBC Mode

If the initial check block is null, then Block 1* = Block 1.

E

Security Environment Coding Examples

Example 1

In this example, the SE is being used to control the access to a file. Suppose you want to protect a file in such a way that a PIN must be correctly presented in order to use the **Activate File** command, and that the PIN to use is defined by SE4. In this case, the security attribute for the file could be coded like that in Table 25, “Security Attributes Coding Example”, on page 36, where the security byte with the value 14h indicates PIN protection in SE4.

Suppose there are two PINs, PINA and PINB and either PIN allows access. Each PIN must be coded in a separate AT.

SE4 could be coded as follows:

- SEID = 04h
- LCS = 04h, meaning OPERATIONAL (DEACTIVATED)
- CRT = A4h, meaning authentication template (AT)

Tag	Length	Value			Description
80h	01h	04h			Tag, length and value of SEID (SE#4)
8Ah	01h	04h			Tag, length and value of LCS (04=OPERATIONAL (DEACTIVATED))
A4h	06h				Tag and length of authentication template
		Tag	**Length**	**Value**	
		83h	01h	PIN Ref	TLV of PINA (Value of AT)
		95h	01h	08h	CRT Usage qualifier (value 08h indicates user authentication)
A4h	06h				Tag and length of authentication template
		83h	01h	PIN Ref	TLV of PINB (Value of AT)
		95h	01h	08h	CRT Usage qualifier (value 08h indicates user authentication)

Example 2

In this example, the SE is used to protect files or data objects by specifying that a symmetric key mutual authentication must have taken place in order to use the **Update Binary** command, and that the key to use is defined by SE5. In this case, the security attribute for the file could be coded:

- 02h AM Byte (see Table 23, “Access Mode Byte Coding – For EFs”, on page 34)
- 25h Security Byte (meaning mutual authentication with key ref in SE5)

SE5 could be coded as follows:

- SEID = 05h
- LCS = 04h, meaning OPERATIONAL (DEACTIVATED)
- CRT = A4h, meaning authentication template (AT)

<table>
<tr><th>Tag</th><th>Length</th><th colspan="3">Value</th><th>Description</th></tr>
<tr><td>80h</td><td>01h</td><td colspan="3">05h</td><td>Tag, length and value of SEID (SE#5)</td></tr>
<tr><td>8Ah</td><td>01h</td><td colspan="3">04h</td><td>Tag, length and value of LCS (04=OPERATIONAL (DEACTIVATED))</td></tr>
<tr><td>A4h</td><td>06h</td><td colspan="3"></td><td>Tag and length of authentication template</td></tr>
<tr><td colspan="2" rowspan="3"></td><td>Tag</td><td>Length</td><td>Value</td><td></td></tr>
<tr><td>83h</td><td>01h</td><td>Key Ref</td><td>TLV of Key ref</td></tr>
<tr><td>95h</td><td>01h</td><td>80h</td><td>CRT Usage qualifier (value 80h indicates mutual authentication)</td></tr>
</table>

Example 3

In this example, the SE is used to protect files or data objects by specifying that an asymmetric key mutual authentication must have taken place in order to use the **Get Data** command, and that the key to use is defined by SE3. In this case, the security attribute for the file could be coded:

- 01h Access Mode Byte (see Table 24, “Access Mode Byte Coding – For Data Objects”, on page 34)
- 23h Security Byte (meaning mutual authentication with key ref in SE3)

SE3 could be coded as follows:

- SEID = 03h
- LCS = 05h, meaning OPERATIONAL (ACTIVATED)
- CRT = A4h, meaning authentication template (AT)

Tag	Length	Value			Description
80h	01h	03h			Tag, length and value of SEID (SE#3)
8Ah	01h	05h			Tag, length and value of LCS (05=OPERATIONAL (ACTIVATED))
A4h	0Dh				Tag and length of authentication template
		Tag	**Length**	**Value**	
		5F4Ch	07h	CHA	TLV of CHA
		95h	01h	80h	CRT Usage (80h for mutual authentication)

Example 4

In this example, the tag 84h is used in an authentication template (AT) as part of an **MSE–Set** command. The command specifies the card’s private authentication key SK.ICC.AUT to be used with the **Internal Authenticate** command.

The format of the MSE–Set command would be:

CLA	INS	P1	P2	Lc	Data
CLA	22h	41h	A4h	Lc	84h || 01h || 02h

Example 5

In this example, the terminal selects the Diffie–Hellman key exchange parameters and sends the value K_{IFD} that it computed to the GemSafe v2 applet

The SE stores the DH parameters in a KAT that is coded as follows:

- SEID = 02h
- LCS = 05h, meaning OPERATIONAL (ACTIVATED)
- CRT = A6h, meaning key agreement template (KAT)

Tag	Length	Value			Description
80h	01h	02h			Tag, length and value of SEID (SE#2)
8Ah	01h	05h			Tag, length and value of LCS (05=OPERATIONAL (ACTIVATED))
A6h	86h				Tag and length of KAT
		Tag	**Length**	**Value**	Value of AT
		83h	01h	Key ID	TLV of Key ID for DH key exchange parameters data object
		91h	81h80h	K_{IFD}	TLV of the K_{IFD} (a random data object)

Example 6

In this example, the SE is used to protect files or data objects by specifying that MAC secure messaging must be used in the command and response when using the **Activate File** command. SE8 is used to specify this secure messaging.

The security attribute for the file is coded as follows:

- 10h Access Mode Byte (meaning only the Activate File SC byte is present)
- 49h Security Byte (meaning secure messaging specified in SE9)

SE9 could be coded as follows:

- SEID = 09h
- LCS = 04h, meaning OPERATIONAL (DEACTIVATED)
- CRT = B4h, meaning cryptographic checksum template (CCT)

Tag	Length	Value			Description
80h	01h	09h			Tag, length and value of SEID (SE#9)
8Ah	01h	04h			Tag, length and value of LCS (04=OPERATIONAL (DEACTIVATED))
B4h	03h				Tag and length of CCT
		Tag	**Length**	**Value**	Value of CCT
		95h	01h	30h	TLV of CRT Usage qualifier. 30h indicates SM is necessary for both the command and response (see Table 35, “CRT Usage Qualifier Values”, on page 50 for possible values)

Example 7

In this example, the SE is used to set a context and provide GemSafe v2 with the information necessary to perform a **PSO: Compute Digital Signature** command. The SE specifies the type of padding to use on the hash to be signed, and the reference of the private key to use. The **PSO: Compute Digital Signature** takes these settings from the current SE. The SE has the following values:

- SEID = 05
- LCS = 05h, meaning OPERATIONAL (ACTIVATED)
- CRT = B6, meaning digital signature template (DST)

Tag	Length	Value			Description
80h	01h	05h			Tag, length and value of SEID (SE#5)
B6h	06h				Tag and length of digital signature template
		Tag	**Length**	**Value**	
		84h	01h	03h	TLV of key reference (key ID = 3)
		80h	01h	11h	TLV of algo ID (11h=SHA–1 with ISO padding)

Example 8

In this example, the SE is used to identify the CA public key before verifying a certificate with the **PSO: Verify Certificate** command. The **PSO: Verify Certificate** command takes these settings from the current SE. The SE has the following values:

- SEID = 05
- LCS = 05h, meaning OPERATIONAL (ACTIVATED)
- CRT = B6, meaning digital signature template (DST)

Tag	Length	Value			Description
80h	01h	05h			Tag, length and value of SEID (SE#5)
B6h	03h				Tag and length of digital signature template
		Tag	**Length**	**Value**	
		83h	01h	04h	TLV of key reference (key ID = 4)

Example 9

In this example, the SE is used to set a context and provide GemSafe v2 with the information necessary to perform a **PSO: Decipher** command. The SE specifies the type of padding used when encrypting the hash, and the reference of the private key to use. The **PSO: Decipher** takes these settings from the current SE. SE has the following values:

- SEID = 06
- LCS = 05h, meaning OPERATIONAL (ACTIVATED)
- CRT = B8, meaning confidentiality template (CT)

Tag	Length	Value			Description
80h	01h	06h			Tag, length and value of SEID (SE#6)
B8h	06h				Tag and length of confidentiality template
		Tag	**Length**	**Value**	
		84h	01h	41h	TLV of key reference (key ID = 41h)
		80h	01h	1Ah	TLV of algo ID (No hash, RSA, PKCS padding)

Example 10

In this example, the SE is used to protect files or data objects by specifying that MAC and Encryption secure messaging must be used in the command but not the response when using the **Activate File** command. SE9 is used to specify this secure messaging.

The security attribute for the file is coded as follows:

- 10h Access Mode Byte (meaning only the Activate File SC byte is present)
- 49h Security Byte (meaning secure messaging specified in SE9)

SE9 could be coded as follows:

- SEID = 09h
- LCS = 04h, meaning OPERATIONAL (DEACTIVATED)
- CRT = B8h, meaning confidentiality template (CT)

<table>
<tr><th>Tag</th><th>Length</th><th colspan="3">Value</th><th>Description</th></tr>
<tr><td>80h</td><td>01h</td><td colspan="3">09h</td><td>Tag, length and value of SEID (SE#9)</td></tr>
<tr><td>8Ah</td><td>01h</td><td colspan="3">04h</td><td>Tag, length and value of LCS (04=OPERATIONAL (DEACTIVATED))</td></tr>
<tr><td>B8h</td><td>03h</td><td colspan="3"></td><td>Tag and length of CT</td></tr>
<tr><td rowspan="2" colspan="2"></td><td>Tag</td><td>Length</td><td>Value</td><td>Value of CT</td></tr>
<tr><td>95h</td><td>01h</td><td>10h</td><td>TLV of CRT Usage qualifier (10h indicates SM in command only)</td></tr>
</table>

F

Common Criteria Certification Requirements

This appendix provides a summary of the requirements that must be respected if the application using the GemSafe V2 applet is to conform to common criteria (CC) certification.

Applet Installation

For CC certification, there are two requirements that must be respected concerning the “application specific parameters” during applet installation. For general information about these parameters, refer to “GemSafe v2–Specific Parameters” on page 24.

The two requirements concern:

- The authentication ratification counter
- The Change PIN before first use byte option

For information about these two parameters, see “Option Parameters” on page 25.

Authentication Ratification Counter

This counter decreases by 1 each time a mutual authentication attempt fails. When a mutual authentication attempt succeeds, the counter returns to its maximum value.

The counter is the second byte of the data object with tag DFh 0Ah shown in “Table 16 - GemSafe v2–Specific Parameters” on page 24 and the first option parameter shown in “Table 17 - Authentication Key Option Byte Coding” on page 24.

CC Requirement

The counter must not be set to a value higher than 0Fh.

Change PIN Before First Use Option

This option is bit 1 of the applications option byte. Refer to “Table 19 - Application Options Byte Coding” on page 25 for details.

The applications option byte is the sixth byte of the data object with tag DFh 0Ah shown in “Table 16 - GemSafe v2–Specific Parameters” on page 24 and the fifth option parameter shown in “Table 17 - Authentication Key Option Byte Coding” on page 24.

CC Requirement

Bit 1 of the applications option byte must be set to 1 indicating the PIN must be changed before first use.

Application Personalization

For CC certification, there are some requirements concerning the key pairs that are dedicated to certified signatures. If your application must be CC certified, then the requirements must be respected when personalizing your applet with the **Put Data** command.

Note: These requirements are only for key pairs dedicated to certified signatures. There are no CC requirements for other PINs and keys.

PINs

The PINs that protect the certified signature keys must respect the following requirements:

- The PIN Try Limit must not be more than 03h. This is coded in the PIN attributes (tag DFh 21h). For details about PINs, refer to “PIN Contents” on page 38 and for the PIN Try Limit, “Table 28 - PIN Attributes” on page 39.
- The PIN must contain at least six digits. If less than eight bytes, the application must pad the PIN to make it a length of eight bytes. The length of the PIN is coded in the PIN value object (tag DFh 22h). It is L_{PIN} in “Table 27 - Structure of a PIN Data Object” on page 38.

RSA Keys

The RSA key pairs used for certificate signatures must respect the following requirements:

- The key pairs must be used for digital signatures only, so the private key usage value (tag DFh 26h) must be 01h
- The key pairs must be at least 1,024–bits long, so the “maximum length” value (tag DFh 25h) must be at least 04h 00h.

Note: For details about the private key usage and maximum length, see “Table 66 - Example Data Field Structure for Put Data–Private Key Creation” on page 110)

- The security attributes for the public portion of the key pair, must specify secure messaging of at least MAC for the **Get Data** command. This means that bit 1 of the access mode byte must be set to 1 (see “Table 24 - Access Mode Byte Coding – For Data Objects” on page 34) and bit 7 of the security condition byte must be set to 1 (see “Security Condition Byte” on page 35).
- The security attributes for the private of the key pair, must specify secure messaging of at least MAC + User Authentication for the **PSO–Compute Digital Signature** command. This means that bit 4 of the access mode byte must be set to 1 (see “Table 24 - Access Mode Byte Coding – For Data Objects” on page 34) and bits 8, 7 and 5

of the security condition byte must be set to 1 (see “Security Condition Byte” on page 35).

Note: The security attributes are the object with the tag 8Ch when using the **Put Data–Private Keys** command, as shown in Table 66, “Example Data Field Structure for Put Data–Private Key Creation”, on page 110.

Signature Requirements

For CC certification, there are two additional requirements that must be respected when computing digital signatures. They are:

- The **PSO–Compute Digital Signature** command must be immediately preceded by a **PSO–Hash** command.
- This preceding **PSO–Hash** command must be sent with secure messaging (at least MAC level).

Terminology

Abbreviations

3DES	triple DES
AID	application identifier
AM	access mode
APDU	application protocol data unit
AT	authentication template
ATR	answer to reset
AUT	authentication (key)
b	binary
CA	certification authority
CAD	card acceptance device
CAR	certification authority reference
CBC	cipher block chaining
CC	common criteria
CCT	cryptographic checksum template
CEN	European Committee for Standardization
CG	cryptogram
CGA	certification generation application
CH	command header
CHA	certificate holder authorization
CHR	certificate holder reference
CPI	certificate profile indicator
CPLC	card production life cycle
CRDO	control reference data object
CRnd	card random number

CRT	control reference template
	Chinese remainder theorem (CRT elements)
CRYCKS	cryptographic checksum
CT	confidentiality template
CWA	CEN workshop agreement
DES	data encryption standard
DF	dedicated file
DH	Diffie–Hellman
DS	digital signature (key)
DSI	digital signature input
DST	digital signature template
EA	external authentication
ECB	electronic code book
EF	elementary file
ENC	encryption
FCI	file control information
FCP	file control parameters
FDB	file descriptor byte
FID	file identifier
h[ ]	hash function
h	hexadecimal
IC	integrated circuit (chip)
ICC	integrated circuit card
ID	identity
IFD	interface device (terminal)
ISO	International Organization for Standardization
KAT	key agreement template
$\mathbf{K_{ENC}}$	key used for encryption in symmetric key mutual authentication
$\mathbf{K_{ICC}}$	computation made by the card used with K_{IFD} to derive the asymmetric key used in mutual authentication
KID	key identifier
$\mathbf{K_{IFD}}$	computation made by the terminal used with K_{ICC} to derive the asymmetric key used in mutual authentication
$\mathbf{K_{MAC}}$	key used for MAC computation in symmetric key mutual authentication
$\mathbf{K_{SK}ENC}$	session key used for encryption when using secure messaging in MAC and ENCRYPTION mode

K_{SK}MAC	session key used for MAC computation when using secure messaging in MAC mode
LCS	life cycle status
Le	expected length
LIFO	last in first out
lsb	least significant bit(s)
LSB	least significant byte(s)
MAC	message authentication code
msb	most significant bit(s)
MSB	most significant byte(s)
MSE	manage security environment
OID	object identifier
PB	padding byte(s)
PI	padding indicator
PIN	personal identification number
PKCS#15	public key cryptography standard#15
PKI	public key infrastructure
PPSSCD	protection profile—secure signature–creation device
PSO	perform security operation
PV	plain value
RFU	reserved for future use
RSA	Rivest, Shamir, Adleman (creators of algorithm)
SC	security condition
SCA	signature creation application
SE	security environment
SEID	security environment identifier
SFI	short file identifier
SHA	secure hash algorithm
SM	secure messaging
SN	serial number
SN.ICC	chip serial number
SN.IFD	terminal serial number
SO	security officer
SSC	send sequence counter
SW	status word

TLV	tag, length, value
TPDU	transmission protocol data units
UT	utility (key)

Glossary

access mode (AM) byte	This byte is part of a security attribute and defines the commands against which the file or data object is to be protected.
applet	In Java Card terminology, a Java Card applet is an independent Java application loaded into a Java Card.
application	The implementation of a well-defined and related set of functions that perform useful work on behalf of the user. It may consist of software and or hardware elements and associated user interfaces.
application identifier (AID)	A byte string that identifies a package or an application in a card, in correspondence with the naming scheme defined by ISO 7816-5. It contains a registered application provider number.
application options byte	This byte codes an option for Identrus compatibility, the “Change PIN before first use” option is coded in b1. If activated, it means that if the signature key used by the **PSO–Compute Digital Signature** command is protected by one or more PINs, then these PINs must have been changed at least once since personalization for the signature key to be usable.
application phase	The phase that begins after the **End Personalization** command is issued.
application privileges	These are coded on one byte and are defined in GP 2.1.1. They define characteristics for an applet, such as it being selected by default following a reset. GemSafe v2 does not require any particular privileges.
application protocol data unit (APDU)	Standard communication messaging protocol between a card acceptance device and a smart card.
authentication template (AT)	A specific type of CRT used to contain data objects involved in mutual authentication.
card manager	An applet with a set of APDU commands which manages card initialization and loading, installation and selection of applets.

card manager/terminal authentication	Process where the Card Manager applet and the terminal authenticate each other in the personalization phase. This is done by the **Initialize Update** and **External Authenticate (Perso)** commands. If authentication is successful, a secure channel for communication is established.
certificate	Data, digitally signed by a Certification Authority and containing the certificate holder's public key. There are many different types of certificates (the most common being X 509 V3).
certificate holder authorization	Used in C_CV.IFD.AUT. This describes the access rights of the certificate holder for a given application. It is seven bytes and is structured as follows: AID Prefix (6 bytes) || Role ID (1 byte).
certificate holder reference	Used in C_CV.IFD.AUT. It is coded on 12 bytes and if it is less than 12 bytes, it is preceded by filler bytes to make it 12. Its value is as follows: Filler (0–4 bytes) || SN.IFD (8–12 bytes) where the filler bytes have the value 00h.
certificate profile indicator	Used in C_CV.IFD.AUT. In GemSafe v2, this always has the value 0Dh.
certification authority	The organization that issues certificates and takes liability associated with the validity of the holder's identity.
certification authority reference	Part of the data in C_CV.IFD.AUT but not interpreted by GemID.
Chinese remainder theorem	A theorem used in public key cryptography, first discovered by the first century Chinese mathematician Sun Tse. It uses prime numbers to find unique solutions to equations. These unique solutions are used to optimize calculations for signing and deciphering data.
command header	The first four bytes of a command APDU (CLA, INS, P1, P2).
communication protocol	A set of rules and procedures governing interchange of information between a smart card and a reader. The ISO defines several protocols, including T=0, T=1.
confidentiality template (CT)	A specific type of CRT used to contain data objects involved in data decryption.
control reference data object	As the name suggests, these data objects are stored in control reference templates. They are a reference to a PIN, key, or an algorithm.
control reference template (CRT)	The CRT is the part of the security environment that contains the control reference data objects.

CPLC (card production life cycle) data	This data is related to the card manufacture, such as fabrication date, chip serial number and so on.
CRT usage qualifier	A particular data object that defines the use of a control reference template.
cryptographic checksum	See MAC.
cryptogram	Used in secure messaging, it is the result of an encryption made by a symmetric algorithm. In GemSafe v2, it is computed using a session key and the 3DES algorithm.
cryptographic checksum template (CCT)	A specific type of CRT used to specify the level of secure messaging (command, response or both) necessary for a command to access a data object
cryptography	The science of ensuring that messages are secure. Cryptographic systems are based on the concepts of authentication, integrity, confidentiality and non-repudiation.
CWA standard	A standard developed to support the EU directive on electronic signatures.
data objects	Information seen at the interface which consists of a tag, a length and a value.
dedicated file	File that holds groups of EFs. It is similar to a directory.
Diffie–Hellman algorithm	Algorithm used to enable two entities such as a card and a terminal to derive a shared secret (a common piece of data) without having to exchange sensitive information. Typically, this shared secret is used by both entities to derive common session keys.
digital signature	(Not to be confused with a digital certificate). An electronic signature created using a public-key algorithm. A digital signature can be used by the recipient to authenticate the identity of the sender and to ensure the integrity of the message.
digital signature template (DST)	A specific type of CRT used to contain data objects involved in signature creation.
direct selection	When a command (for example, **Read Binary**) is performed using direct selection, it is performed directly on the currently selected EF.
elementary file (EF)	A file that contains data. All EFs in GemSafe v2 are transparent EFs.
exponent	One of the elements of an RSA public key.

external authentication

The procedure to authenticate the external world (a terminal or a user) to a card.

In the personalization phase, this is done by the Card Manager applet (see card manager/terminal authentication on).

In the application phase, this can be asymmetric key or symmetric key mutual authentication according to the application. In asymmetric key mutual authentication this is performed by the **External Authenticate** command. In symmetric key mutual authentication external authentication is combined with internal authentication in a single command: **Mutual Authenticate**. Also see mutual authentication.

file control parameter template

This is the data specified at the time of file creation in the **Create File** command.

file descriptor byte

Element generated at file creation and used to specify the type of file as follows:

01h: EF
38h: DF

file identifier

This is allocated when the file is created. The 5 least significant bits are known as the SFI. See also, short file identifier.

GemXpresso Pro R3.x

The name of a Gemplus smart card that contains the GemSafe v2 applet.

GlobalPlatform

The organization that manages, promotes and evolves the Open Platform specifications.

hashing algorithm

The process whereby a variable-length input string is converted to a fixed-length (generally smaller) output string. SHA–1 is such an algorithm.

host

A central computer or server in a network.

Identrus

An international body that has developed a standard for electronic identification.

implicit selection

When a command (for example, **Read Binary**) is performed using implicit selection, it is performed on an EF under the currently selected DF. In this case the SFI of the EF must be specified in the command.

initializing

The writing of the initial value to an empty data object such as a key. The values can later be updated.

instance

An object of a particular class. In programs written in the Java programming language, an instance of a class is created using the new operator followed by the class name.

internal authentication	The procedure to authenticate the card to the external world (a terminal or host). In the personalization phase, the **Initialize Update** command performs this as part of the Card Manager/ terminal authentication. In the application phase, this can be asymmetric key or symmetric key mutual authentication according to the application. In asymmetric key mutual authentication this is performed by the **Internal Authenticate** command. In symmetric key mutual authentication internal authentication is combined with external authentication in a single command: **Mutual Authenticate**. Also see mutual authentication.
key agreement template (KAT)	A specific type of CRT used to contain the reference to the Diffie–Hellman key data objects involved in asymmetric key mutual authentication. It also stores K_{IFD}, the computation made by the terminal as part of the asymmetric key derivation.
load file	A complete block of data transferred to a GlobalPlatform card containing certification data and a Load File Data Block.
load file data block	A block of data containing one or more applications and libraries or support information for the applications as required by the specific platform.
message authentication code (MAC)	Sometimes referred to as a cryptographic checksum. A symmetric cryptographic transformation of data that guarantees the integrity of a message. Used in secure messaging. It is not the same as a cryptogram.
modulus	One of the elements of an RSA public key.
mutual authentication	Process whereby the terminal and card authenticate each other in the application phase.
offset	The distance from the start of a file. Its value is added to a base value to derive the actual value.
on board key generation	The process where the card generates values for a public key pair. The GemSafe v2 applet has a function, **Generate Public Key Pair**, that updates the values of an existing key pair in the card. The value of the public key is also returned to the terminal in the response APDU. It cannot create a new key pair
open platform	The industry standard for managing a smart card based multiple application program. Includes card specifications, terminal specifications, development tools and back office support. Also see GlobalPlatform.

package	A group of types (classes or interfaces)
padding	One or more bits appended to a message in order to ensure that it contains the required number of bits or bytes.
path	The concatenation of two–byte file identifiers starting with the root or current DF and ending with the file itself. It includes the file identifier of the parent DF. In this way, any file can be selected unambiguously from the current DF. As the path does not include the current DF or the root in GemSafe v2, it is reduced to the two–byte file ID for DFs and EFs under the root. For EFs under a DF, the path becomes the concatenation of the DF file ID and the EF file ID, a total of four bytes.
personalization	The creation of files and/or data objects necessary for an application.
personalization device	The device that writes the personalization data to the applet.
personalization phase	The first phase for a GemSafe v2 applet in which personalization takes place. The phase ends after the execution of the **End Personalization** command and moves to the application phase.
PIN ID	Identity number of a PIN (in the range 1h–Fh). See also PIN reference
PIN reference	The reference of a PIN. In GemSafe v2 this is always “8X” where X is the PIN ID. See also PIN ID.
public key infrastructure	The software and/or hardware components necessary to manage and enable the effective use of public key encryption technology, particularly on a large scale.
private key	The half of an RSA public key pair that is private and is used for signatures and decryption. It is stored as a data object in the GemSafe v2 applet.
public key	The half of an RSA public key pair that is public and is used for signature verification and encryption. It is stored as a data object in GemSafe v2 when the corresponding private key is present. Other public keys can be stored in the applet in transparent EFs.
reference PIN	The PIN stored in the GemSafe v2 applet that can be used to identify a cardholder and protect data. The PIN entered by a user is compared against the reference PIN.

role ID

Extracted from the C_CV.IFD.AUT during external authentication. In GemSafe v2, the role ID is a number, which in itself has no significance other than it must match the value in the current SE in order to allow access to a data object.

RSA algorithm

Rivest, Shamir, and Adleman; a highly secure encryption method that uses a two-part key. The private key is kept by the owner; the public key is published. Data is encrypted by using the recipient's public key, which can only be decrypted by the recipient's private key. RSA is also used for authentication. You can verify who you are with a digital signature by encrypting a message with your private key and letting others decrypt your message with your public key. This requires the sender to compute a hash value of the message being sent, which is encrypted along with the message. If they match, the signature is authenticated.

secure channel

Card Manager/terminal authentication consists of opening a secure channel that determines the level of secure messaging during the personalization phase. The three levels used in GemSafe v2 are:

- No secure messaging
- MAC
- MAC and Encryption

secure messaging

Process used to protect command transmissions between GemSafe v2 and the terminal. This can be done either by encrypting the data that is sent to the card or by sending three bytes of a cryptographic checksum with the command or both (in which case the MAC computation is made on the encrypted data).

security attribute

The condition(s) that must be fulfilled to grant access to a file or data object. Security attributes are made up of two parts:

- One access mode (AM) byte.
- 0–4 security condition bytes.

See also the entries for “access mode (AM) byte” and “security condition (SC) byte”.

security condition (SC) byte

Part of a security attribute. The number of security condition bytes is determined by the preceding access mode byte.

security environment (SE)

Mechanism to specify to the card system the security functions that are available to provide protection to commands and data for a specific application of the card.

session key	A temporary cryptographic key that the card uses during secure messaging sessions.
SHA–1 (secure hash algorithm)	A hash algorithm developed by the National Institute of Standards and Technology and the National Security Agency.
short file identifier	The Short File Identifier corresponds to the 5 least significant bits of the file identifier. It is used to reference a file in a command.
status codes	Values, standardized in ISO 7816-4, which indicate the result (status) of a command APDU.
tag, length, value (TLV) format	The components that make up a data object, where tag is an identifier, length is the length of the data and value is the data itself.
transparent file	An unstructured file as defined in ISO 7816–4. The EF is seen at the interface as a sequence of bytes.
triple DES	Encryption/decryption algorithm based on 16-byte keys; a variation of the DES algorithm, consisting of a triple encryption.
try counter	Part of a PIN object. Stores the number of remaining attempts that can be made to present the correct PIN before it is blocked. The initial value of this is equal to the value of Try Limit. It is decremented for each incorrect PIN verification. When this value reaches zero, the PIN is blocked
try limit	Part of a PIN object. Stores the maximum number of consecutive incorrect PIN presentations that can be made before the PIN is blocked (max value is 0Fh).
unblocking counter (PINs)	Part of a PIN object. Stores the number of times that the PIN can be unblocked. This counter can be set to maximum of 0Fh and is decremented each time that the PIN is unblocked. Once this counter reaches zero, the PIN can no longer be unblocked. Setting this counter to FFh means that the PIN can always be unblocked.
unblocking PIN reference	Part of a PIN object. It gives the PIN ID of the unblock PIN. If set to zero, the PIN can never be unblocked.

updating	In GemSafe v2, updating values implies that an initial value is already present. Writing the first value to an empty data object is referred to as initializing
usage counter (PINs)	Part of a PIN object. Stores the number of times that the PIN can be verified. This counter can be set to maximum of 0Fh and is decremented each time that the PIN is verified. Once this counter reaches zero, the PIN can no longer be verified. Setting this counter to FFh means that the PIN can always be verified. This feature is useful to limit the use of an unblock PIN for example.

For More Information

Standards and Specifications

References are made to the following standards and documents:

- *CWA 14890 2004—Application Interface for smart cards used as Secure Signature Creation Devices—Part 1 -Basic requirements. March 2004.*
- *Protection Profile—Secure Signature–Creation Device Type 2, version 1.04, EAL 4+*, 25 July 2001. Prepared by E–Sign Workshop Expert Group F. Prepared for CEN/ISSS.
- *Protection Profile—Secure Signature–Creation Device Type 3, version 1.05, EAL 4+*, 25 July 2001. Prepared by E–Sign Workshop Expert Group F. Prepared for CEN/ISSS.
- *Java Card 2.1.1 Virtual Machine Specification from Sun Microsystems*, Revision 1.0, May 18 2000
- *Java Card 2.1.1 Runtime Environment Specification from Sun Microsystems*, Revision 1.0, May 18 2000
- *Java Card 2.1.1 Application Programming Interface from Sun Microsystems*, Revision 1.0, May 18 2000
- *GlobalPlatform Card Specification 2.1.1*, March, 2003
- *ISO/IEC 7816-3: Electronic signals and transmission protocols.*
  Defines the characteristics of the electronic signals exchanged between the card and the terminal, and the two possible low-level communication protocols that can be used. (T = 0 is an asynchronous half-duplex character transmission protocol; T = 1 is an asynchronous half-duplex block transmission protocol).
- *ISO/IEC 7816-4: Interindustry commands for interchange.*
  Defines a set of standard commands for smart cards, as well as a hierarchical file system structure for cards. These commands are the base of most existing card protocols.
- *ISO/IEC 7816-6: Interindustry data elements.*
  This standard defines the TLV data element system for cards.
- *ISO/IEC 7816-8: Security related interindustry commands.*
  Defines a set of standard commands for cryptographic operations for smart cards.
- *ISO/IEC 7816-9: Additional interindustry commands and security attributes.*
  Defines the coding of the life cycle of the card and of the security attributes of card related objects.

- *ISO/IEC 7816-15: Cryptographic Information Application.*
  This standard is based on PKCS#15 but is specific to IC card requirements.
- *ISO/IEC 9796— Information Technology - Security Techniques - Digital Signature Schemes Giving Message Recovery - Part 2: Integer Factorization Based Mechanisms*
- *ISO/IEC 9797—Information Technology—Security Techniques—Data Integrity Mechanism Using a Cryptographic Check Function Employing a Block Cipher*
- *ISO/IEC 11770–3—Information Technology—Security Techniques—Key Management Part 3:* Mechanisms Using Asymmetric Techniques
- *PKCS#1 v2.0: RSA Cryptography Standard*, RSA Laboratories, Sept. 4, 1998
- *ANSI X9.19—Financial Institution Retail Message Authentication*, Jan. 1, 1996
- *Key Storage Mechanism Requirements*, version 3.0, Identrus

Other Gemplus Documentation

The GemSafe v2 applet resides on GemXpresso Pro R3.x cards. For information about these cards, refer to the *GemXpresso Pro R3.x Reference Manual.*

If you own one of the Gemplus software tools, RAD III or JCard Manager 4.0, these are supplied with the following useful documents:

RAD III:

- *RAD III Getting Started*
- *RAD III User Guide*

JCard Manager 4.0:

- *JCard Manager 4.0 Getting Started*

Recommended Reading

Applet Development

For more information about Java applet development for smart cards, see:

- *Java Card Applet Developer's Guide, Java Card Version 2.1 from Sun Microsystems*, Version 1.0, August 30, 1999
- *Guidelines for Developing Java Card Applications on GlobalPlatform cards*, Version 1.0, December 2002

Cryptography

B. Schneier. *Applied Cryptography, Protocols, Algorithms and Source Code in C*. John Wiley & Sons, Inc., 1996.

A major reference about cryptography. Contains an introduction to all the important concepts in cryptography, as well as a description of the most widely used algorithms, from Enigma to PGP. Plus source code for all of the algorithms!

Web References

The JavaSoft Home Page (http://www.java.sun.com)

The home page for the latest developments in Java. Maintained by Sun.

The Java Card home page (http://java.sun.com/products/javacard)

The Java Card home page, maintained by Sun. The right place to get the latest version of the API specification, at all times.

GlobalPlatform (http://www.globalplatform.org)

Site contains interesting specifications for downloading including an overview of the GlobalPlatform Card Specification.

The Identrus home page (http://www.identrus.com)

Index

A

access mode (AM) bytes 6, 8, 33, 36
  for data objects 34
  for DFs 34
  for EFs 34
Activate File
  in application phase 129
  in perso phase 73
AID
  applet 23
  application instance 23
  load file 23
  package 23
algorithm
  ID 15, 53
  keys xii
  notation xii
APDU
  body fields 68
  command format 67
  header fields 68
  response format 68
applet xiii
  AID 23
  personalization data 147, 151
  version number 89, 93, 147
application xiii
  commands 127
  instance AID 23
  options byte 25, 163
  privileges 23
application protocol data unit. See APDU
asymmetric key mutual authentication 11, 13, 35, 57–62, 155
  AT templates 49
  External Authenticate command 141
  main steps 59
AUTH/ENC session key 41
authentication
  asymmetric key mutual 13
  external 50
  key option byte 24
  keys xii
  mutual 35, 36, 43
  ratification counter 25
  templates (ATs) 49
  user 9, 35, 49

C

C.ICC.AUT 27
C_CV.IFD.AUT 58, 172, 199
card
  certificate 27
card manager/terminal authentication 40
certificate
  card 27
  holder authorization (CHA) 50, 58, 199, 200
  holder reference (CHR) 199, 200
certificates 199
  verifying 49, 172
certification authority (CA)
  public key 11, 27, 49, 58
  reference 199, 200
chaining commands 68, 101
change PIN before first use. See application options byte
Change Reference Data 40, 132
chip serial number (SN.ICC) 64, 200
command
  format 67, 69
  header 67

commands
Activate File (application phase) 129
Activate File (perso phase) 73
Change Reference Data 40, 132
Create File 76, 134
Deactivate File (application phase) 135
Deactivate File (perso phase) 80
End Personalization 82, 147
Erase Binary (application phase) 138
Erase Binary (perso phase) 84
External Authenticate (Application) 35, 61, 141
External Authenticate (Perso) 21, 40, 87
Generate Public Key Pair 19, 34, 144
Get Challenge 146
Get Data (application phase) 147
Get Data (perso phase) 89
Get Response 96, 154
Initialize Update 21, 97
Internal Authenticate 155
list of application 127
list of personalization 71
MSE-Restore 157
MSE-Set 158
Mutual Authenticate 64, 160
PSO-Compute Digital Signature 14, 25, 34, 162
PSO-Decipher 18, 34, 165
PSO-Hash 168
PSO-Verify Certificate 60, 172
Put Data (application phase) 174
Put Data (perso phase) 100
Put Data-DH Key Exchange Parameters (application phase) 181
Put Data-DH Key Exchange Parameters (perso phase) 115
Put Data-PINs 102
Put Data-Private Keys (application phase) 177
Put Data-Private Keys (perso phase) 108
Put Data-Public Keys (application phase) 179
Put Data-Public Keys (perso phase) 112
Put Data-Secret Keys (application phase) 175
Put Data-Secret Keys (perso phase) 106
Put Data-Security Environments 101
Read Binary 118, 183
Reset Retry Counter 39, 40, 186
Select File (application phase) 188
Select File (perso phase) 121
Update Binary (application phase) 192
Update Binary (perso phase) 125
Verify 194
common criteria certification requirements 213–215
common secret data. See shared secret data
Compute Digital Signature
see PSO-Compute Digital Signature
computing
digital signatures 49
MACs 46
session keys in application phase 65
session keys in personalization phase 41
confidentiality templates (CTs) 49, 165
control reference templates (CRTs) 49
CPLC data 89, 93, 147
Create File 76, 134
creating
digital signatures 49
files 27, 28, 31, 76, 134
PINs 37, 102
secret keys 30
CRT
elements 10
usage qualifiers 50
cryptograms 44
cryptographic checksum templates (CCTs) 49
cryptographic checksums. See MACs

D

data
decryption 18, 49
encryption 18
objects 3, 9
structure 3
Deactivate File
in application phase 135
in perso phase 80
Decipher
see PSO-Decipher
decrypting data 18, 49
dedicated files (DFs) 6
access mode bytes 34
DF name 6, 78
file control parameters (FCP) 6, 78
file descriptor 6, 7
file descriptor byte (FDB) 6, 78
file ID 6, 78
nested 6
referencing by DF name 9
delegated management 23
deriving
session keys 202
DF name 6, 9, 78, 121, 122
partial 121, 122
DF. See dedicated files (DFs)

Diffie-Hellman
  algorithm 201
  key exchange parameters 11, 27, 58, 115, 201
digital signature
  input (DSI) 14
  keys xii
  templates (DSTs) 49
digital signatures 199
  computing 14, 49, 162
  performing 14
  verifying 17
Display Message EF 62

E

EFs. See elementary files (EFs)
elementary files (EFs) 7
  access mode bytes 34
  Display Message 62
  file control parameters (FCP) 7, 78
  file descriptor byte (FDB) 7, 78
  file ID 7, 78
  file size 7, 78
  transparent 8
  types 8
encrypting data 18
encryption mode (secure channels) 40, 42
End Personalization 82, 147
Erase Binary
  in application phase 138
  in perso phase 84
exponent 11, 145, 199
  initializing 112, 114, 179, 180
External Authenticate (Application) 35, 61, 141
External Authenticate (Perso) 21, 40, 87
external authentication 50

F

FDB
  for DFs 6, 78
  for EFs 7, 78
file
  descriptor byte. See FDB
  hierarchy 5
  path 9
  size for EFs 7, 78
file control information
  for DFs 123
  for EFs 124
file control parameters (FCP) 77
  DFs 6, 78
  EFs 7, 78
file ID
  DFs 6, 78
  EFs 7, 78
file selection 121
  methods 84
files 5
  creating 27, 28, 31, 76, 134
  dedicated 6
  Display Message 62
  referencing by DF name 9, 121, 122
  referencing by file ID 8, 74, 81, 121, 122
  referencing by path 74, 81, 121, 122
  transparent 8

G

GemSafe v2 applet
  installing 21–25
  personalizing 27–32
Generate Public Key Pair 19, 34, 144
generating keys 19, 144
Get Challenge 146
Get Data
  in application phase 147
  in perso phase 89
Get Response 96, 154

H

hash
  padding the 14–16, 53
hashing 14, 53
header of commands 67

I

identifying RSA keys 13
Initialize Update 21, 97
initializing
  private keys 10
  public keys 11
Install 23
installing the GemSafe v2 applet 21–25
interface device xiii
Internal Authenticate 155
ISO 9796-2 padding 15, 53, 162

K

key
  agreement templates (KATs) 49
  exchange parameters (Diffie-Hellman) 27
  indexes 98
  notation xii
  pairs 13
  usage xii
keys
  authentication xii
  digital signature xii
  generating on board 19, 144
  private 10
  public 11
  secret 10
  utility xii, 165

L

life cycle states 36
  applet 55
  card 40
  security environments 48
life cycle status
  files 73, 77, 80
  for EFs 7, 78
load file AID 23

M

MAC
  and encryption mode (secure channels) 40, 43
  computation 46
  mode (secure channels) 40, 41
  session key 41
modes of secure channels 40
modulus 11, 199
  conditional value for digital signatures 162
  initializing 112, 114, 179, 180
MSE-Restore 157
MSE-Set 158
multiple instances 2
Mutual Authenticate 64, 160
mutual authentication 35, 36, 43, 57–66
  asymmetric key 11, 13, 49, 57–62, 141, 155
  card manager/terminal 40, 87, 97
  main steps 63
  symmetric key 10, 49, 63–65

N

no secure messaging (secure channel mode) 40
notation for keys xii

O

on board key generation 19, 144

P

package AID 23
padding
  a hash 14–16
  indicator 44
  ISO 9796-2 15, 53, 162
  PKCS #1 v2.x 15, 53, 165
  RFC 2409 15, 53
partial DF name 121, 122
path (for files) 9, 74, 81
performing digital signatures 14
personalization 27–32
  commands 71
  PKCS#15 profiles 32
PINs 35, 37–40
  changing the value of 40
  creating 37, 102
  try counter 39, 132, 186, 194
  try limit 39
  unblocking 39, 186
  unblocking counter 39
  unblocking PIN reference 39
  updating 37, 102
  usage counter 39, 132, 186, 194
  value 40
  verifying 39, 194
PK.CA.AUT 27, 58, 172
PK.ICC.AUT 155
PKCS #1 v2.x padding 15, 53, 165
PKCS#15
  personalization profiles 32
private keys 10
  creating 108
  CRT elements 10
  initializing 10
  purpose 13
  storing 10
  updating 144
PSO-Compute Digital Signature 14, 25, 34, 162
PSO-Decipher 18, 34, 165
PSO-Hash 168
PSO-Verify Certificate 60, 172

public keys 11
  CA 11
  creating 112
  exponent 11
  initializing 11
  maximum number of 25, 112
  modulus 11
  purpose 13
  retrieving value 148
  storing as data objects 11
  updating 144
Put Data
  in application phase 174
  in perso phase 100
Put Data-DH Key Exchange Parameters
  in application phase 181
  in perso phase 115
Put Data-PINs 102
Put Data-Private Keys
  in application phase 177
  in perso phase 108
Put Data-Public Keys
  in application phase 179
  in perso phase 112
Put Data-Secret Keys
  in application phase 175
  in perso phase 106
Put Data-Security Environments 101

R

Read Binary 118, 183
referencing files 8
  by DF name 121, 122
  by file ID 8, 74, 81, 121, 122
  by path 9, 74, 81, 121, 122
Reset Retry Counter 39, 40, 186
response format 68, 69
RFC 2409 padding 15, 53
role IDs 49, 50, 58
root 5
  selection 121
RSA key pairs 13
  maximum number of 25
RSA keys
  identifying 13
RSA padding 53

S

secret data 60, 65, 141
secret keys 10
  creating 30, 106
  maximum number of 25
secure channels 87, 97
  encryption mode 40, 42
  MAC and encryption mode 40, 43
  MAC mode 40, 41
  modes 40
  no secure messaging mode 40
  setting 40
secure messaging 40–46
  in application phase 43–46
  in personalization phase 40–43
security attributes 6, 7, 8, 33–36, 47, 78
  example 36
security condition (SC) bytes 6, 8, 33, 35, 36
security environments 47–54
  control reference templates 49
  creating 101
  identifiers 48
  life cycle state 48
  maximum number of 47
  modifying 101
  purpose 9, 47
  retrieving value 150
  structure 48
Select File
  in application phase 188
  in perso phase 121
selecting files
  direct selection 84
  implicitly 84
  the root 121
SEs. See security environments
session keys 61, 141
  computation in personalization phase 41
  deriving 202
  for secure messaging in personalization phase 41
setting secure channels 40
SFI. See short file identifier (SFI)
SHA-1 algorithm 53
shared secret data 60, 65, 141, 201
short file identifier (SFI) 7, 8
signatures. See digital signatures
SK.ICC.AUT 27, 58, 155
SK.ICC.DS 27
SK.IFD.AUT 58
status bytes 68
storing
  private keys 10

symmetric key mutual authentication 10, 35, 63–65
  AT templates 49

T

transparent EFs 8
triple DES (3DES) 203
  in CBC Mode 204
  in ECB Mode 203
try counter (PINs) 39, 132, 186, 194
try limit (PINs) 39

U

unblocking PINs 39, 186
  counter 39
  reference 39
Update Binary
  in application phase 192
  in perso phase 125
updating
  PINs 37, 102
  private keys 144
  public keys 144
usage counter (PINs) 39, 132, 186, 194
user authentication 9, 35, 49
utility keys xii, 165

V

Verify 194
verifying
  certificates 49, 172
  digital signatures 17
  PINs 39, 40, 194